DOORS Software Design Description

Data Orchestration and On Demand Reporting System

Reverse Engineered Design Baseline for User Interface Master API Agent API and Governed Data Services

Document attribute	Value
Document type	Software Design Description
Product baseline	DOORS 1.0.9
Document version	Final Review 1.3
Prepared date	27 October 2026
Design basis	Current doors-ui, doors-master-api and doors-agent-api source on branch doors-write-feature
Audience	HOG, product owners, architects, developers, DBAs, operations, security, QA and comprehensive audit teams
Classification	Internal project document

Document Control

Version	Date	Owner	Change summary	Status
1.0	26 October 2026	DOORS project team	Initial reverse-engineered design baseline covering UI, Master API, Agent API, read, write, document, reporting, subscriptions, AIra, security and operations	Audit Draft
1.1	27 October 2026	DOORS project team	Added complete routed-screen wireframe catalogue and controlled-state approval workflow swimlanes	Audit Draft
1.2	27 October 2026	DOORS project team	Added inbound request and outbound response encryption, decryption, signing, key-wrapping and failure-path sequence diagrams	Audit Draft
1.3	27 October 2026	DOORS project team	HoD design-assurance review: completed quality attributes, assumptions, capacity, retention, availability, decisions, risks, traceability and final acceptance controls	Final for Approval

Approval Record

Role	Name	Decision	Date
Project owner		Pending	
GePNIC technical authority		Pending	
Database authority		Pending	
Security authority		Pending	
Quality assurance lead		Pending	

How to Use This Document

This description explains how the implemented DOORS system is structured and how its components cooperate at runtime. Reviewers should read the Executive Design Summary first, Sections 3 to 7 for architecture, Sections 8 to 17 for feature design, Sections 18 to 23 for security and operations, Section 24 for acceptance evidence, and the appendices for implementation catalogues and assurance records. The SRS remains the authority for required behaviour; this SDD records the implemented design used to satisfy it.

Document Status and Approval Condition

This is the final technical review copy for DOORS 1.0.9. It becomes an approved controlled baseline only when the named authorities complete the Approval Record and release evidence confirms the same source, database, configuration and deployment baseline. Blank names or dates are not implicit approvals. Any material deviation after approval requires change control and an updated design or recorded design decision.

Executive Design Summary

DOORS separates governance from execution so that centrally approved data services can operate against independently controlled GePNIC environments without moving database authority into the application tier. The Master API owns identity, policy, approval, client admission, cryptography, orchestration, reporting and audit. The Agent API executes only approved, bounded operations near the source database. Browser and external machine interfaces use separate request planes with independent admission controls.

The design is acceptable for final approval subject to environment evidence for certificates, secrets, proxy rules, database grants, backup and recovery objectives, capacity limits, monitoring thresholds and successful representative tests. Principal residual risks are configuration drift, certificate or secret expiry, Agent unavailability, incomplete operational retention values and divergence between generated database packages and deployed catalogues. Appendices I to K make the decisions, risks and acceptance conditions explicit.

Contents

1 Purpose Scope and Design Basis

2 Design Drivers and Constraints

3 System Context

4 Deployment and Request Planes

5 Logical Component Architecture

6 User Interface Design

7 Master API Design

8 Agent API Design

9 Identity Authentication and Session Design

10 Authorization and Governance Design

11 Read Service Design

12 Write Service Design

13 Document Service Design

14 Reporting and Export Design

15 API Client Protocol and SDK Design

16 Subscription Rate Limit and Usage Design

17 AIra Design

18 Data and Persistence Design

19 Interface Design

20 Security Design

21 Audit Logging and Observability

22 Error Handling Resilience and Recovery

23 Configuration Build and Deployment

24 Verification Testability and Traceability

- Appendix A Component Catalogue
- Appendix B Endpoint Families

- Appendix C Configuration Catalogue
- Appendix D State Models
- Appendix E Acronyms and Terms
- Appendix F Source Baseline
- Appendix G User Interface Screen Catalogue and Wireframes
- Appendix H Controlled State Workflow Swimlanes
- Appendix I Architecture Decision Register
- Appendix J Residual Risk and Operational Dependency Register
- Appendix K Final Design Acceptance Checklist

1 Purpose Scope and Design Basis

This document records the current software design of DOORS as implemented in the Vue user interface, Spring Boot Master API, Spring Boot Agent API, Flyway migrations and generated database packages. Its purpose is to make design decisions, runtime responsibilities, trust boundaries and operational dependencies understandable without relying on the original developers.

1.1 Scope

- Interactive administration and self-service functions exposed through the DOORS UI.
- Master API governance, routing, security, persistence, reporting, SDK and documentation services.
- Agent API query, document and write execution inside a GePNIC environment.
- Read services, write services, document delivery, subscriptions, API clients and AIra.
- Authentication, authorization, encryption, audit, rate limiting, error handling and deployment.
- Logical data design and its relationship to the separate Database Design Document.

1.2 Design Authority

The deployed source and database catalogue are the authority for an environment. This SDD uses the current source baseline and versioned migrations. Configuration values, secrets, certificates and infrastructure addresses remain environment-specific and are intentionally described by purpose rather than copied into this document.

1.3 Related Documents

Document	Purpose
Software Requirements Specification	Required behaviour, actors, controls and acceptance criteria
Database Design Document	Physical and logical database structures, roles, grants and verification queries
Complete Functional User and Administrator Manual	Operational procedures and screen-level instructions
Generated OpenAPI and SDK documentation	Client-specific machine interface and onboarding details
Deployment runbooks	Environment-specific service, proxy, certificate, backup and rollback steps

1.4 Assumptions Exclusions and Change Control

Category	Statement
Assumption	Enterprise DNS, TLS termination, time synchronization, certificate issuance, protected secret delivery, database backup and infrastructure monitoring are supplied by the approved hosting environment.
Assumption	Each Agent is mapped to one controlled GePNIC database context and receives only locally configured database credentials and target profiles.
Exclusion	This SDD does not publish production secrets, private keys, passwords, certificate contents, infrastructure addresses, personal data samples or unrestricted SQL.
Exclusion	Detailed screen operating steps belong to the functional manual; physical database definitions and grants belong to the Database Design Document.
Change control	Changes to trust boundaries, cryptographic protocol, authorization model, database function authority, request

Category	Statement
	planes or approval states require architecture, security and QA review.
Evidence rule	Where this SDD states a configurable limit or objective, approved environment configuration and release evidence provide the binding value.

2 Design Drivers and Constraints

Driver or constraint	Design response
GePNIC databases remain under DBA control	Master never performs live DDL; approved SQL is downloaded and run by the DBA
Data stays near the source	Agent executes approved queries, resolves documents and writes through approved functions
Human and machine access differ	Interactive sessions use captcha, password, MFA and CSRF; machine APIs use client credentials, IP policy, signatures and encryption
Independent governance is required	Proposers cannot approve their own query, contract or write-service revision
Existing read services must remain stable	Write-service routes, permissions, tables and runtime identities are independent
Writes require immutable history	Each changed JSON snapshot receives a server-managed record version; identical JSON returns UNCHANGED
One Agent maps to one GePNIC database	Write mappings are Agent based; stored instance context is retained for compatibility and consolidation
Audit investigations require usable evidence	Trace, actor, client, API key, parameters, endpoint, Agent, outcome and timing are retained subject to secret exclusions
No secret may be exposed in browser code	Private keys, passwords, transit secrets and database credentials remain in protected service configuration

2.1 Quality Attribute Objectives

Attribute	Design objective	Verification evidence
Security	Fail closed at every external and inter-service trust boundary; preserve least privilege and separation of duties	Negative authorization, route-plane, signature, encryption, CSRF, IP, replay and database-grant tests
Integrity	Bind governed definitions, generated SQL and runtime requests to immutable versions and hashes	Definition-hash comparison, migration checksum and audit correlation
Availability	Isolate failure by plane and Agent; bound downstream calls; preserve durable governance and write receipts	Health checks, timeout tests, restart tests and environment recovery exercise
Performance	Apply row, payload, pool, timeout, export and rate bounds before resource exhaustion	Environment load profile and percentile latency or throughput evidence
Scalability	Scale stateless processing within database, key-store and downstream limits	Capacity test at approved concurrency and payload profile
Maintainability	Use versioned APIs, layered services, generated contracts, Flyway and reproducible builds	Build pipeline, automated tests, ownership and decision records
Auditability	Correlate actor, policy, request, target, outcome and timing without retaining secrets	Trace walk-through across UI, Master, Agent and database evidence
Recoverability	Restore consistent Master metadata, GePNIC write history,	Approved RPO and RTO, backup evidence and recovery

Attribute	Design objective	Verification evidence
	receipts, grants and deployment versions	rehearsal

3 System Context

DOORS sits between human users or external applications and independently operated GePNIC data sources. The Master API owns governance and client admission. The Agent API owns execution inside the destination environment. GePNIC databases and document repositories remain the system of record.

3.1 DOORS System Context

The context diagram identifies the principal actors, application boundary, execution Agent and supporting services. It also shows that browser and machine requests enter through different controls while both are governed by the Master API.

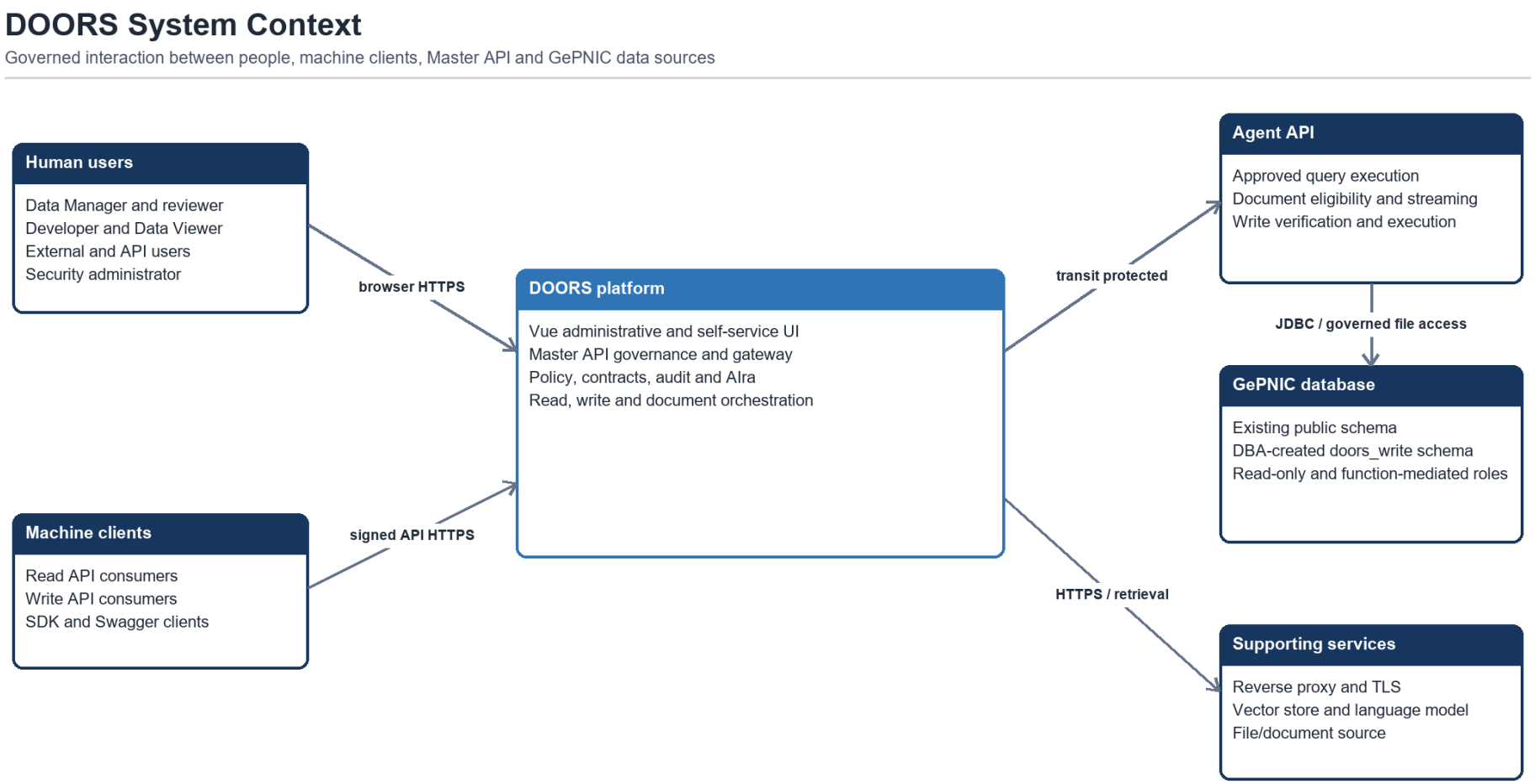


Figure 3.1 DOORS System Context

4 Deployment and Request Planes

The deployment separates Admin, External and API request planes by hostname and route policy. The reverse proxy terminates TLS and forwards requests to private application ports. PlaneRoutingFilter and related policies fail closed when a host or route does not belong to the expected plane.

4.1 Deployment and Request Planes

Each request plane exposes only the functions intended for its audience. Authentication on one plane does not authorize routes on another plane.

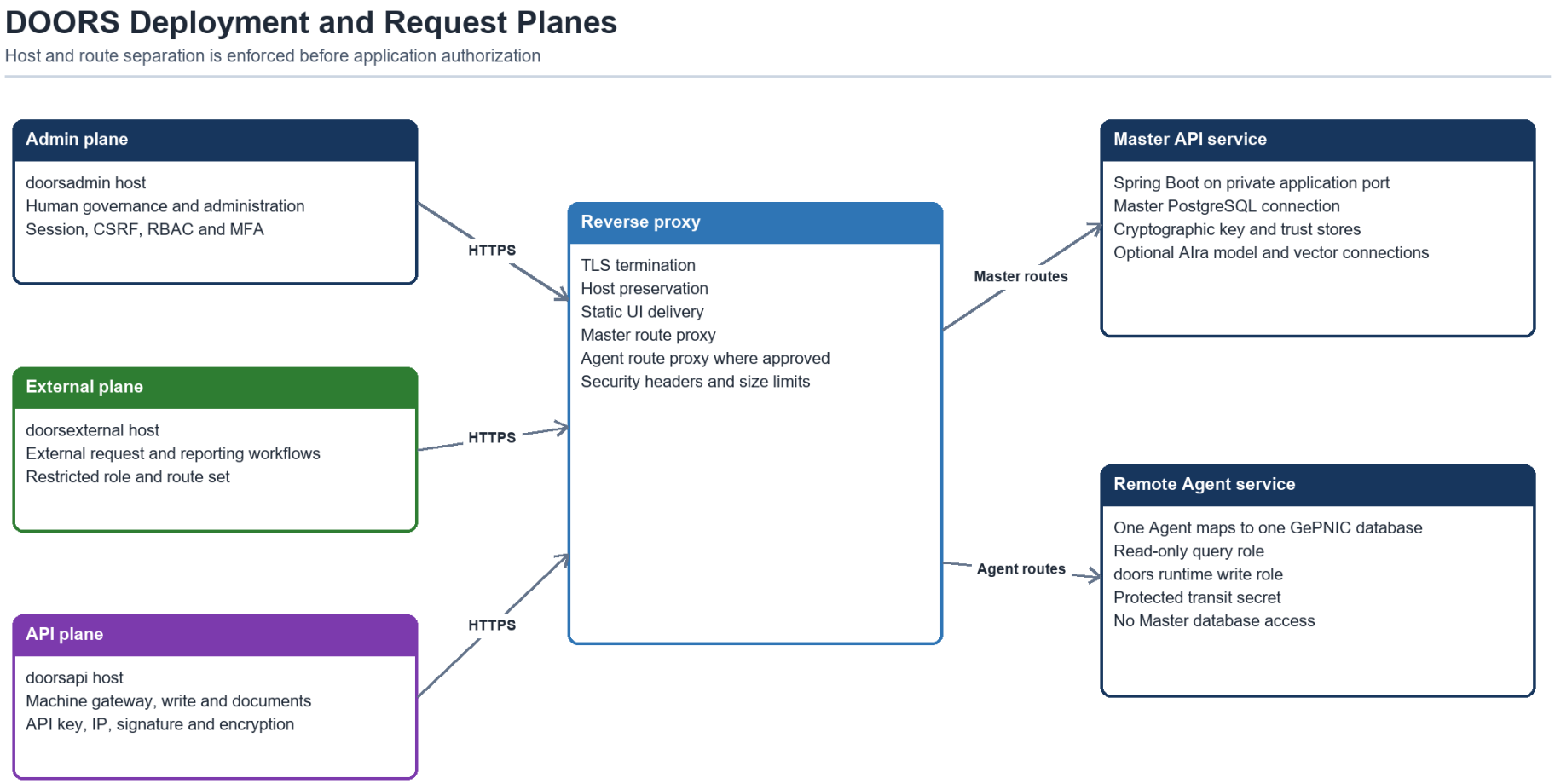


Figure 4.1 Deployment and Request Planes

4.2 Runtime Nodes

Node	Runtime	Responsibility	Persistent dependencies
Browser	Modern browser	Vue UI, session cookies, CSRF token and user interaction	No trusted secret storage
Reverse proxy	Nginx or approved enterprise proxy	TLS, static UI, host routing, headers, body limits and upstream proxying	Certificate and proxy configuration
Master API	Java 21 and Spring Boot 3.2.5	Governance, gateway, client policy, reporting, AIra, audit and SDK generation	Master PostgreSQL, key stores and optional vector/model services
Agent API	Java 21 and Spring Boot 3.5.10	Bounded query execution, document controls and governed writes	GePNIC PostgreSQL and document source
Database	PostgreSQL	Master metadata or GePNIC operational and doors_write data	DBA-managed backup and recovery
AI services	Configured model and vector services	Embedding, retrieval and language generation	Approved knowledge only

4.3 Network Trust Boundaries

- The proxy is the only intended public entry point; application ports remain private.
- Forwarded headers are accepted only from configured trusted proxies.
- Master-to-Agent routes use HTTPS outside local development and an inbound transit secret.
- Agent database connections use least-privilege roles defined for read, sandbox or live write operations.
- The API plane does not rely on an interactive browser login for machine requests.

4.4 Environment Separation and Promotion

Concern	Required design control
Environment identity	Development, test, UAT, staging and production use distinct hostnames, databases, credentials, certificates and key identifiers.
Promotion	Only versioned source, migrations, generated database packages and configuration templates move forward; runtime data and private keys do not.
Production access	Administrative access uses approved network, MFA, privileged-role and audit controls; direct database access remains DBA controlled.
Test identities	UAT certificates and wallets are visibly identified and technically prevented from becoming production trust anchors.
Configuration drift	Release evidence compares effective plane rules, Agent targets, migrations, grants, certificates and artefact hashes with the approved baseline.
Rollback	A rollback restores a compatible application and configuration set; irreversible data changes require an approved forward-fix strategy.

5 Logical Component Architecture

5.1 Component Design

The Master API applies cross-cutting admission controls before invoking feature services. Persistence and cryptographic operations remain behind service boundaries. The Agent has no connection to the Master database.

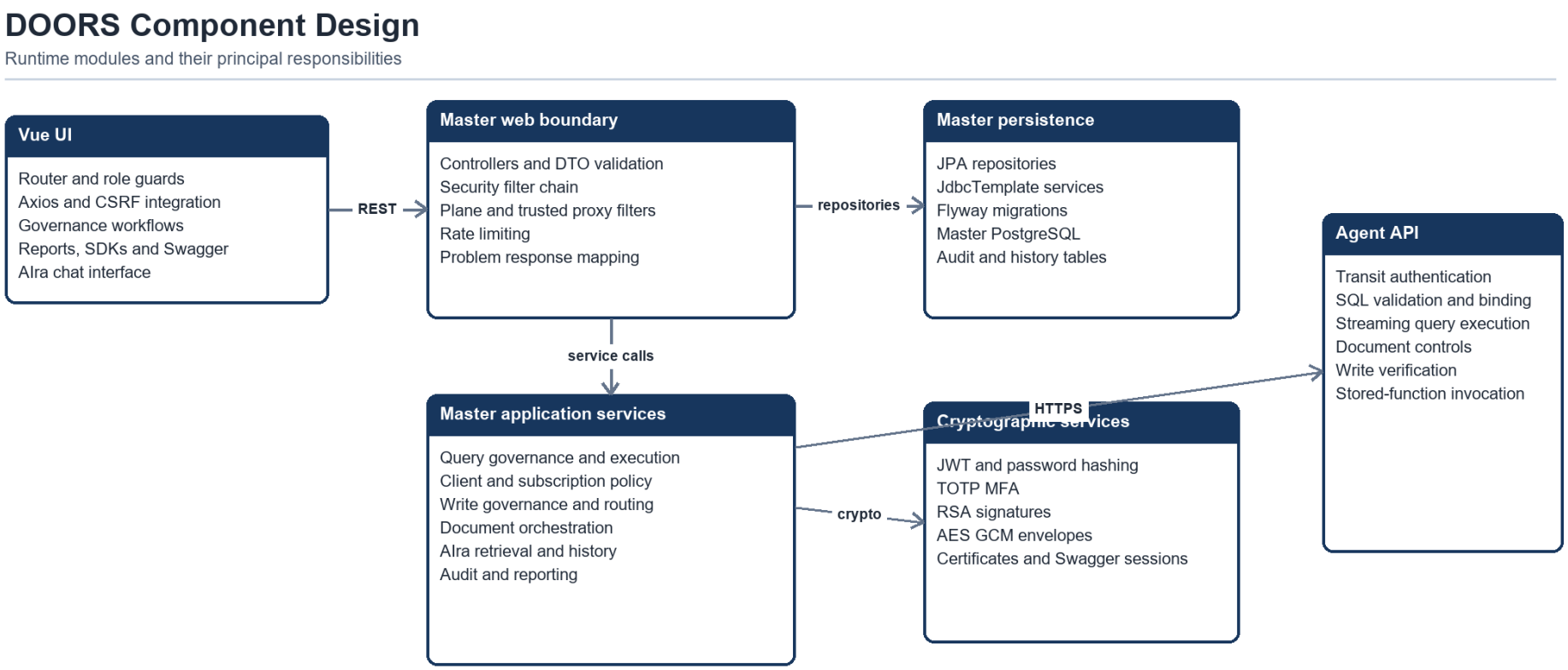


Figure 5.1 Component Design

5.2 Layer Responsibilities

Layer	Principal elements	Design responsibility
Presentation	Vue views, components, router, Pinia and Axios	Role-scoped navigation, forms, validation feedback and secure API invocation
Web boundary	Spring controllers, filters and exception handlers	Route exposure, request validation, authentication context and safe responses
Application	Governance, execution, client, document, subscription and AIra services	Use-case coordination and transaction boundaries
Domain and policy	Contracts, mappings, roles, state transitions and validation rules	Business invariants independent of presentation
Persistence	JPA repositories, JdbcTemplate and Flyway	Durable state, migration and query execution
Integration	Agent HTTP clients, cryptographic services, model/vector clients and export services	External protocol adaptation and failure translation

5.3 Architectural Invariants

- The Master does not receive caller-supplied database credentials or execute live write DDL against a GePNIC database.
- The Agent does not access the Master database and does not decide governance approval.
- A browser role guard is never treated as an authorization control; the server independently authorizes every protected operation.
- An external client cannot select an arbitrary Agent, SQL statement, database object, filesystem path or write function.
- A governed revision is identified by immutable version and definition hash; material change produces a new revision.
- Plaintext, passwords, private keys, AES keys and transit secrets are excluded from audit and ordinary application logs.
- Retries of a write use the same request identifier and unchanged logical payload; receipts prevent duplicate mutation.

6 User Interface Design

6.1 Technology and Structure

The UI uses Vue 3, TypeScript, Vite, Vue Router, Pinia, Axios and Element Plus. Feature views are grouped by authentication, governance, external self-service, reporting and role-specific dashboards. Shared layout, footer, AIra chat and security utilities provide consistent behaviour.

6.2 Route Guard Design

Guard	Decision
Authentication	Unauthenticated users are redirected to login; authenticated users cannot remain on login or registration routes
Password reset	Users marked for forced password change are redirected to the change-password route
Role	Route metadata names one role or an allowed-role list; unauthorized roles return to their permitted dashboard

Guard	Decision
Security administrator	SecurityAdmin is directed to its dedicated user-management surface
API user	ApiUser is directed to subscription and client self-service functions
Session failure	401 or an invalid session clears local identity state and returns to login

6.3 Major View Groups

View group	Primary views	Purpose
Authentication	Login, registration and password change	Captcha, account registration, MFA continuation and credential lifecycle
Read governance	Query submission, pending approvals, query library and template contracts	Create, test, approve and publish SELECT services
Write governance	Write workflow, request approvals and write library	Analyse JSON, govern contracts, provision and verify destinations
Client registry	Read and Write Services tabs	Credentials, IPs, mappings, permissions, versions, SDKs and Swagger
Documents	Dashboard, service registry, policies and logs	Configure and monitor governed document delivery
Reporting	Report library and viewer	Execute approved templates and export bounded results
Administration	Users, mappings, Agents, categories, signing keys, subscriptions and audit	Operate identities, infrastructure and platform policy
Alra	Global chat component	Grounded feature, procedure and authorised activity assistance

6.4 Client-Side Security

- The UI does not contain database passwords, private signing keys or transit secrets.
- Axios sends credentials only to the configured same-origin API and carries the CSRF token for state-changing interactive requests.
- DOMPurify is used where rich or generated content must be rendered.
- The browser prepares Swagger write envelopes only after the user selects a PKCS12 identity and enters its password; the password is not persisted.
- Role guards improve usability but do not replace server-side authorization.

6.5 Screen and Workflow Documentation Standard

Every user-facing screen shall have an approved wireframe documenting its layout, controls, authorized roles, validation rules, navigation paths and significant UI states. Every workflow involving approval or another controlled state transition shall have a swimlane diagram identifying the responsible actors and system components and covering, as applicable, the normal flow, approval, rejection, return for correction, resubmission, authorization checks, notifications, audit events, exception handling and final state. Appendices G and H provide the controlled baseline.

7 Master API Design

7.1 Technology

The Master API is a Java 21 Spring Boot application using Spring MVC, Spring Security, Spring Data JPA, Spring JDBC, WebFlux clients, Flyway, PostgreSQL, JJWT, Nimbus JOSE, Bucket4j, springdoc OpenAPI, PDFBox and LangChain4j integrations.

7.2 Controller Domains

Domain	Controllers	Responsibility
Identity	AuthController and AdminUserController	Captcha, login, MFA, registration, password and privileged account administration
Infrastructure	AgentController, InfrastructureController and UserAgentMappingController	Agent registry, health, ownership and user scope
Read governance	TemplateController, QueryGovernanceController and TemplateContractGovernanceController	Submission, dry run, approval, mapping and JSON contract governance
Machine clients	ApiClientController, ApiUserSelfServiceController and SwaggerSessionController	Client credentials, IPs, query mappings, protocol policy and test sessions
Write services	WriteGovernanceController, WriteClientController, InboundAdminController and WriteIntegrationController	Contract lifecycle, Agent verification, client authorization, SDK and OpenAPI
Gateway	ExternalGatewayController and InboundGatewayController	Read orchestration and encrypted write admission
Documents	Policy, registry, gateway, reporting and audit controllers	Eligibility, streaming, configuration and evidence
Reports	ReportViewerController, ReportController and PublicReportApiController	Template discovery, execution and export
Subscriptions	ApiSubscriptionController and ApiSubscriberDashboardController	Licences, periods, schedules, notices and usage
Alra	AlraController	Grounded chat, history, ratings, analytics and knowledge reindex
Documentation	SecureDocumentationController and SwaggerPageController	Role-aware documents, walkthroughs and interactive API guides

7.3 Service Boundaries

Service group	Key services	Design notes
Execution	QueryExecutionService, AgentExecutionService and OrchestrationRequestService	Resolve approved definitions and coordinate bounded Agent calls
Governance	QueryApprovalService, TemplateContractService and GovernanceReviewRules	Enforce state, independent review and revision rules
Client policy	ApiClientService, MappingService, GatewayProtocolService and ClientSpecificDataSegregationService	Intersect identity, query, Agent, protocol and response scope
Write	WriteGovernanceService, WriteClientService, WriteIntegrationService and InboundGatewayService	Separate governance, onboarding artefacts and runtime delivery
Documents	Manifest, grant, policy, registry, orchestration and audit services	Keep policy and file transfer responsibilities separate
Alra	Context, knowledge, operational answer, history and orchestration services	Prefer deterministic grounded answers before model generation
Security	Captcha, TOTP, certificate, signing and response-encryption services	Centralize cryptographic and authentication operations
Audit	AuditService, SecurityAuditService, AuditAspect and AuditDiagnostics	Correlate actions and retain safe troubleshooting context

7.4 Transaction Boundaries

Governance state transitions and persistence updates execute inside service transactions. External Agent calls are bounded and their results are recorded after policy decisions. Write data transactions occur inside the GePNIC database store function so that history, head and receipt remain consistent even if the network response is lost.

8 Agent API Design

8.1 Responsibility

The Agent is the only DOORS component that connects directly to a GePNIC operational database or document source. It accepts only Master-authorized operations, applies local validation and uses configured target profiles. It does not contain governance state and does not connect to the Master database.

8.2 Components

Component	Responsibility
AgentQueryController	Execute approved parameterized queries, stream results, run dry runs and evaluate document eligibility
AgentStatusController	Report service and database readiness
InboundGovernanceController	Sandbox a write definition or verify installed schema, ownership, function settings and grants
InboundWriteController	Receive the internal write request and return the stored outcome
JsonWriteEngine	Validate the contract and invoke the approved store function using bound parameters
AgreementProvisioning	Support agreement-specific provisioning and compatibility validation
JsonColumnMapRowMapper	Convert JDBC results to deterministic JSON while preserving database value types
InboundProperties	Bind inbound feature, target profile, transit secret and sandbox configuration

8.3 Query Safety

- Only SELECT statements are accepted for governed read execution.
- Named inputs use the p_ prefix and are bound as values rather than concatenated into SQL.
- SQL parsing distinguishes parameters from PostgreSQL format strings such as HH:MI.
- Row, time, statement and payload limits are applied before or during execution.
- The configured database identity remains read only for existing business schemas.

8.4 Write Safety

- Live writes use a configured target profile and the doors runtime database role.
- The Agent validates the approved definition hash, function signature and installed objects.
- The runtime role executes a SECURITY DEFINER store function and has no table DDL or direct table DML.
- Sandbox uses an isolated temporary schema and rolls back test objects.
- Verification reports the exact failing object, role or privilege in Agent logs while the external response remains bounded.

9 Identity Authentication and Session Design

9.1 Interactive Authentication Flow

The interactive authentication flow separates identity proof, optional second factor and authorization. Captcha, IP and plane checks reduce abuse before a protected session is issued.

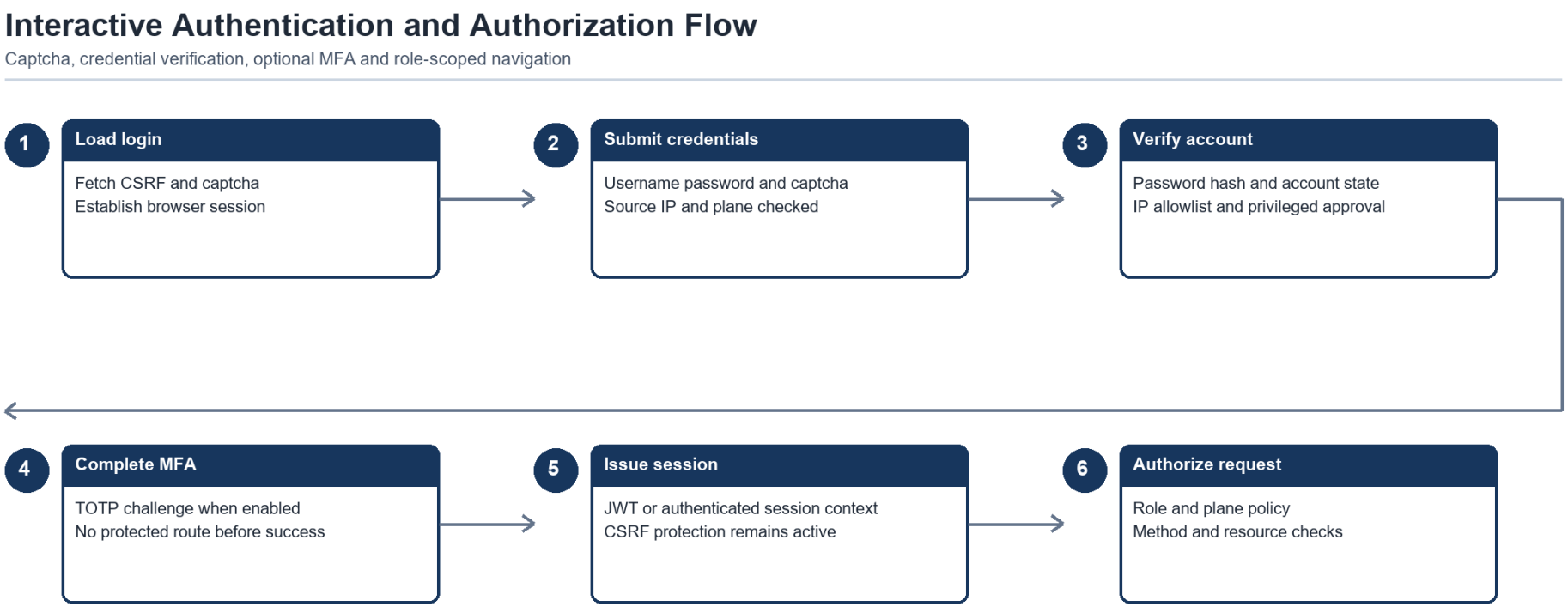


Figure 9.1 Interactive Authentication Flow

9.2 Account Lifecycle

State or event	Control
Registration	Creates a pending account with the requested role and profile subject to administrative approval
Approval	Authorized administrator activates the account and records approval evidence
First or reset login	password_reset_required forces a password change before ordinary functions
MFA enrollment	TOTP secret is encrypted at rest and verified before privileged access completes
VPN or IP policy	Configured addresses or CIDRs are checked for protected users
Session revocation	Logout, password reset, administrator action or session replacement invalidates the active context
Rejection or disablement	Authentication fails closed and the state is retained for audit

9.3 Password and MFA Controls

Password encoding is provided by the Spring Security password configuration. Raw passwords are never stored or logged. Administrative reset issues a controlled reset state rather than exposing the existing password. MFA uses time-based one-time passwords and protected secret storage. Exact password length, complexity, lockout and expiry values remain configuration-controlled and must be stated in the security policy register for each deployment.

10 Authorization and Governance Design

10.1 Role Model

Role	Primary authority
DataManager	Govern queries, write contracts, clients, Agents, documents, subscriptions, reports and audit
SecurityAdmin	Administer approved security-sensitive identity functions without general data-governance access
Developer	Propose development artefacts and use permitted documentation and test functions
DataViewer	Run approved reports and view permitted documentation
External	Submit and track external data requests and run explicitly exposed reports
ApiUser	Manage assigned clients, subscription information and approved self-service testing
Machine client	Invoke only mapped services from permitted IPs using configured credentials and protocol

10.2 Effective Permission

Effective access is an intersection, not a single flag. A successful request may require an active account or client, correct request plane, permitted source IP, valid schedule or subscription, approved query or contract, client mapping, Agent mapping, service permission, valid protocol, cryptographic verification and an available target. Failure of any required condition denies the request.

10.3 Separation of Duties

- A proposer cannot approve their own governed query or write contract.
- A changed query or contract must be dry-run or sandbox tested before approval.
- The DBA, not the application runtime, creates or migrates doors_write objects.
- Schema verification is distinct from client authorization.
- Read permission does not imply write permission, and write permission does not grant direct table modification.

11 Read Service Design

The sequence shows policy resolution before Agent execution and response protection after bounded results return.

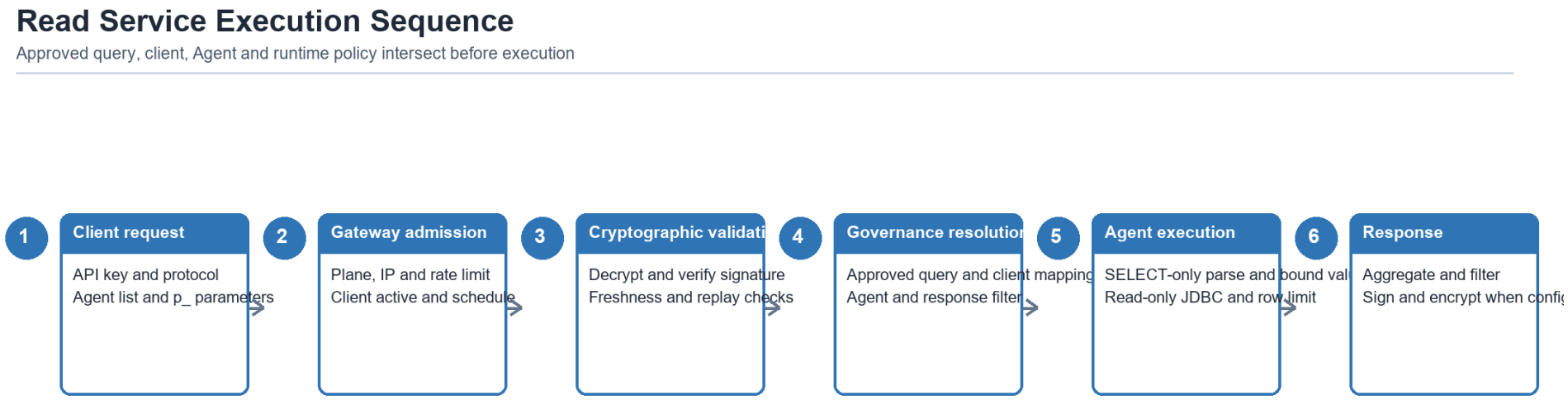


Figure 11 Read Service Design

11.1 Query Governance Lifecycle

Stage	Design action	Persisted evidence
Submission	Validate SELECT-only SQL and p_ parameters; capture title, purpose and classification	Draft query, proposer and version
Dry run	Execute against an authorized Agent with supplied test values	Agent, parameters, result summary and timestamp
Review	Allow reviewer edits, require re-test when SQL changes and record justification	Reviewer, decision, notes and tested SQL
Approval	Activate the version and authorized Agent mappings	Approved state and audit event
Client mapping	Join an active client to the approved query and optional response filter	Client-query grant and segregation rule
Execution	Resolve policy and send bound values to selected Agents	Trace, timings, count and outcome

11.2 Parameter Rules

Input parameters are explicitly named with the p_ prefix. The UI and Agent parser use the same rule so PostgreSQL date and time format fragments, casts and ordinary colons are not interpreted as inputs. Values are bound through JDBC. The approved SQL text is never replaced by caller-supplied SQL.

11.3 Response Handling

- Results from multiple Agents are correlated with their source and aggregated according to the request.
- Configured client segregation filters restrict rows or fields after approved execution.
- Structured responses preserve supported database value types through deterministic JSON mapping.
- Large report exports run through bounded export jobs rather than an unlimited interactive response.
- Encrypted clients receive the response through the configured cryptographic protocol.

12 Write Service Design

The top row is the governed onboarding lifecycle. The bottom row is the runtime write path after the schema and client permission are ready.

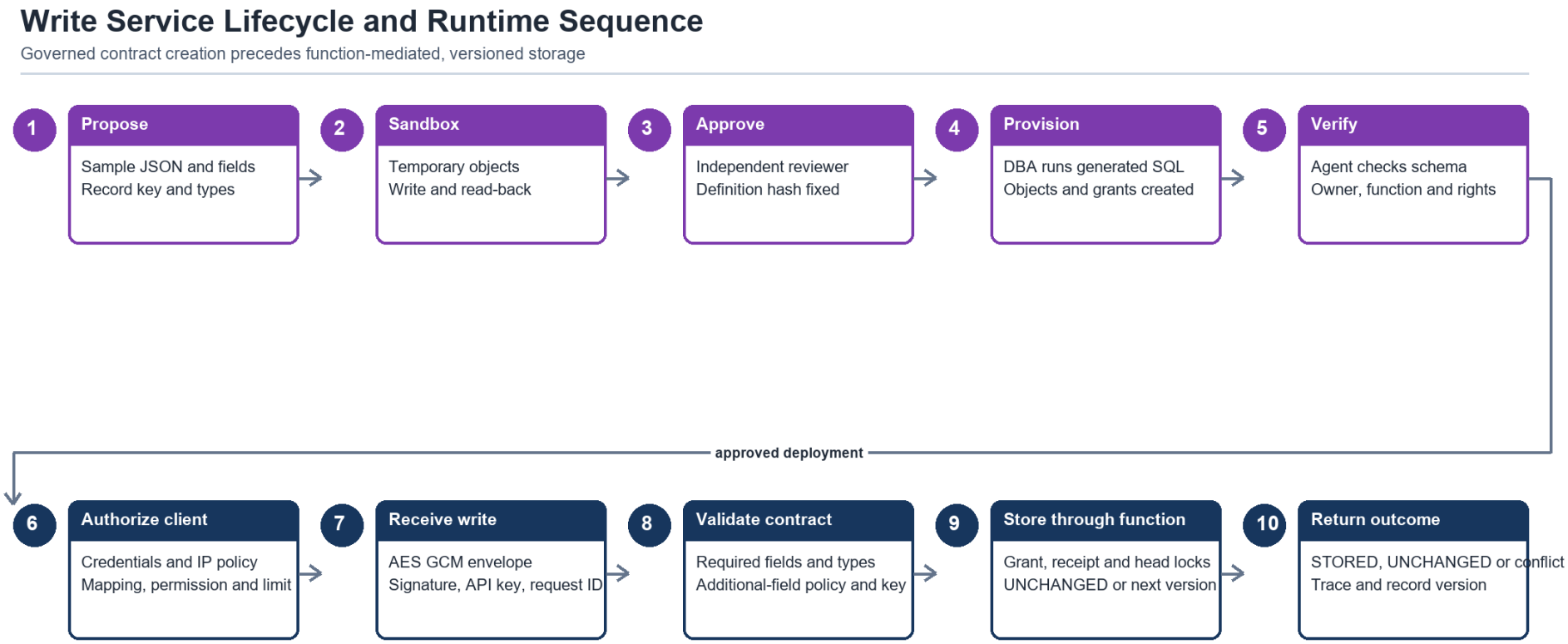


Figure 12 Write Service Design

12.1 Contract Model

A write contract contains a stable lowercase service code, human title and purpose, representative JSON, approved field definitions, required flags, PostgreSQL projections, record-key fields, additional-field policy, contract version and SHA-256 definition hash. The complete received JSON is stored as payload; selected scalar projections support operational queries.

12.2 Contract Revision

- Create New Revision copies the approved fields, types, keys and rules into a new draft.
- Added, removed or changed fields are edited in the revision and tested through the same sandbox flow.
- Approval fixes a new definition hash and contract version.
- The generated migration SQL checks the expected prior definition hash before preserving and changing existing objects.
- Existing record history and permissions are retained unless the approved migration explicitly changes them.

12.3 Runtime Validation

Validation	Outcome
Client, mapping or service permission missing	403-style access denial before Agent invocation
Signature, key, IP or encryption invalid	Request rejected without database access
Required field missing	422 contract validation failure
Projected value incompatible with approved PostgreSQL type	422 before store function invocation where determinable; database conversion remains a final defence
Unapproved additional field	Behaviour follows the contract rollout policy: warn or reject according to the approved mode
Same request ID and same fingerprint	Stored response returned with replayed true
Same request ID and changed payload	IDEMPOTENCY_CONFLICT
Same business key and identical canonical JSON	UNCHANGED; no new record version
Changed JSON within client limit	STORED with the next automatic record version
Changed JSON beyond client limit	409 with administrator-contact guidance

12.4 Database Transaction

The Agent calls the approved SECURITY DEFINER function. The function validates the definition hash and service grant, locks or creates the receipt claim, locks the business-key head row, compares the latest payload, enforces the client version limit, appends the immutable snapshot, advances the head and stores the response receipt in one transaction. A network timeout can therefore be resolved by retrying the same request ID and unchanged payload.

12.5 Latest JSON Read

The latest view exposes one current payload per business key together with the record version and approved metadata. In a database dedicated to one GePNIC instance, consumers may filter by the approved business key such as tender_id and agreement_no. instance_id remains available for compatibility or consolidated storage.

13 Document Service Design

Document delivery combines client admission, policy eligibility, safe Agent resolution, bounded streaming and integrity evidence.

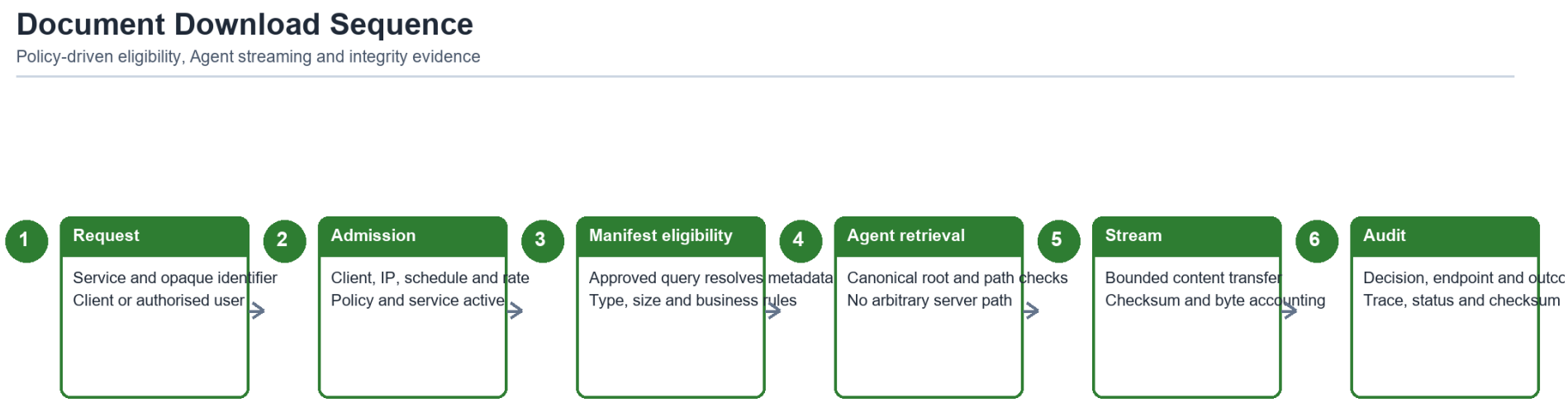


Figure 13 Document Service Design

13.1 Design Objects

Object	Purpose
Document download policy	Defines accepted identifiers, document types, eligible clients, limits, validity and grant rules
Document service registry	Maps a service code to an Agent endpoint, manifest query, policy and governed content root
Document access grant	Provides a time-bound entitlement with eligibility metadata
Manifest query	Returns approved document metadata without exposing an arbitrary filesystem path
Download audit	Records decision, source, endpoint, status, bytes, checksum and trace without storing document content

13.2 Path and Content Controls

- The caller supplies an opaque identifier defined by policy rather than a server filesystem path.
- The Agent resolves the canonical path under the configured content root and rejects traversal or escape.
- Allowed types, file size, count and request frequency are policy bounded.
- Streaming avoids loading the full document into the Master heap.
- Checksum and byte-count evidence support integrity investigation.

14 Reporting and Export Design

14.1 Report Catalogue

Approved read templates marked for reporting appear in the report library according to role and mapping. The viewer discovers required p_ parameters, permitted Agents and presentation metadata before execution.

14.2 Execution

- The viewer submits only approved parameter values and target Agents.
- The Master uses the same governed read path as machine orchestration.
- Table and chart rendering occurs from the returned structured response.
- Exports create a bounded job, expose status and allow authorized download when complete.
- Public report APIs expose only explicitly approved catalogue items and response shapes.

15 API Client Protocol and SDK Design

15.1 Client Security Record

Attribute	Use
API key	Passport credential identifying the client; rotated and audited by administrators
Public key	Verifies client signatures and may protect responses or session material
IP allowlist	Restricts network origins using exact addresses or CIDRs
Activation	Fail-closed switch for the client
Service mappings	Names approved read queries or verified write destinations
Service permission	Separately enables a mapped write service
Protocol or version entry	Publishes applicable API guide, SDK, OpenAPI and Swagger test
Schedule and subscription	Restricts dates, times, periods, Agents and service availability

15.2 Secure Transport Overview

The standard machine transport uses HTTPS plus a signed AES-256-GCM application envelope. It provides confidentiality, integrity, source authentication, request correlation and fail-closed processing across the external-client trust boundary. The application envelope does not replace TLS. A fresh symmetric key and IV are generated for each protected message; private keys never leave their owning trust boundary.

15.3 Inbound Encryption and Decryption Flow

The inbound flow protects client data before it leaves the SDK or browser test surface. The Master admits the client before decryption, verifies the registered client signature, unwraps the transient key, authenticates and decrypts the AES-GCM payload, validates the governed contract, and sends only an authorized protected request to the Agent.

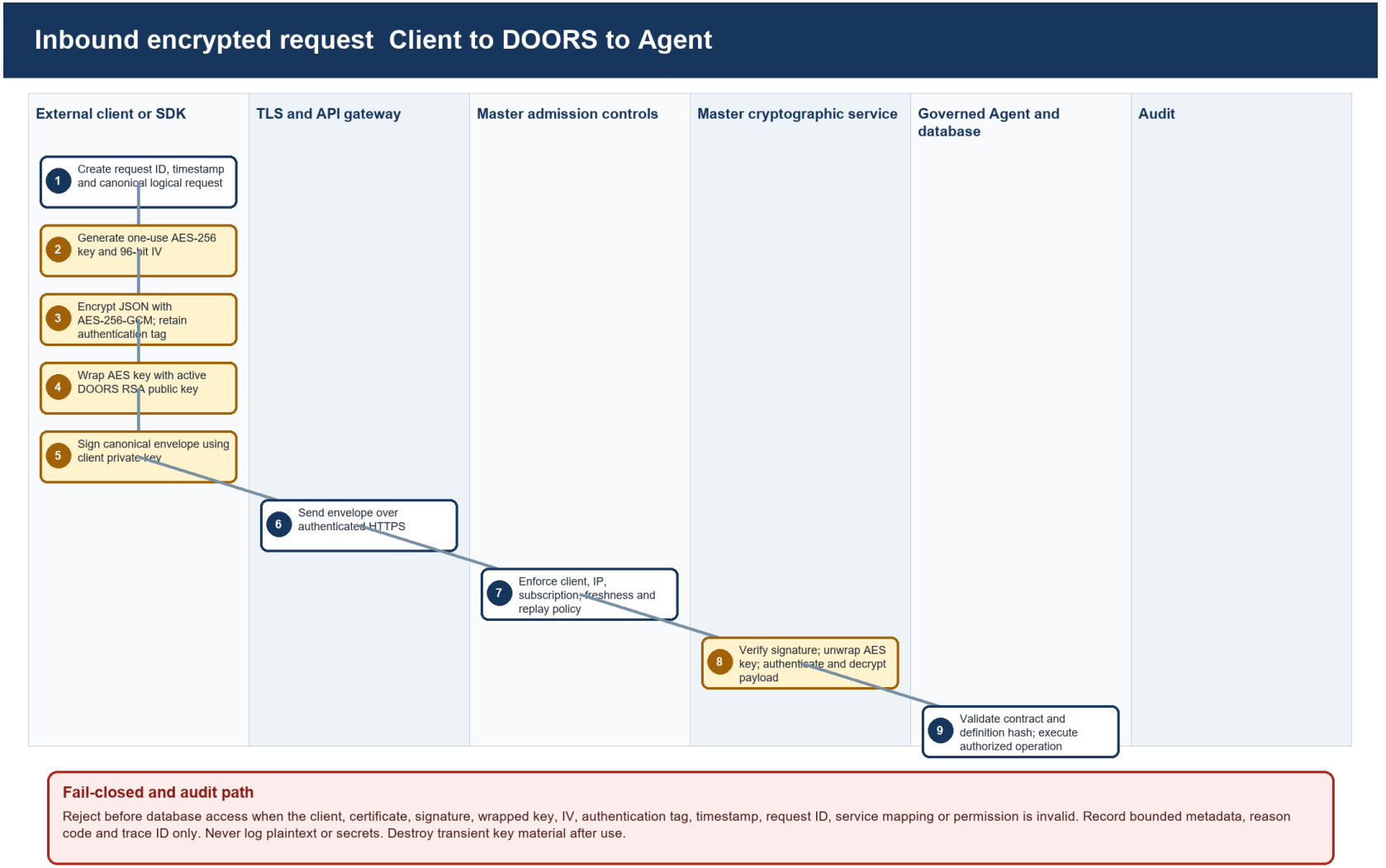


Figure 15.3 Inbound Encryption and Decryption Flow

15.3.1 Inbound Envelope Controls

Stage	Mandatory control	Failure outcome
Envelope creation	Canonical request, UUID request ID, protocol version and freshness value	Client does not transmit an incomplete envelope
Content encryption	Fresh AES-256 key, unique 96-bit IV and authenticated GCM tag	Encryption failure produces no request
Key protection	AES key wrapped with the active DOORS RSA public key	Unknown or inactive key identifier is rejected
Client authentication	Canonical envelope signed by client private key; registered certificate selected by client and key identity	Invalid signature or certificate is rejected before payload use
Admission	Client status, API key, IP, subscription, service mapping, permission, timestamp and replay checks	Request is rejected before Agent or database access
Decryption	DOORS private key unwraps AES key; AES-GCM tag authenticates ciphertext before plaintext is accepted	Unwrap, IV or tag failure discards the message
Governed execution	Contract, service, definition hash, schema and payload rules are validated	Validation failure returns a bounded error without execution
Secret disposal	Transient AES key, plaintext working buffers and key passwords are not persisted and are cleared when practical	Secrets and plaintext are excluded from logs and audit

15.4 Outbound Encryption and Decryption Flow

The outbound flow applies authorization and client-specific segregation before encryption. The Master encrypts the canonical response using a fresh AES-GCM key, wraps that key for the registered client, signs the envelope with the DOORS signing identity, and returns it over HTTPS. The client verifies trust and signature before authenticating and decrypting the content.

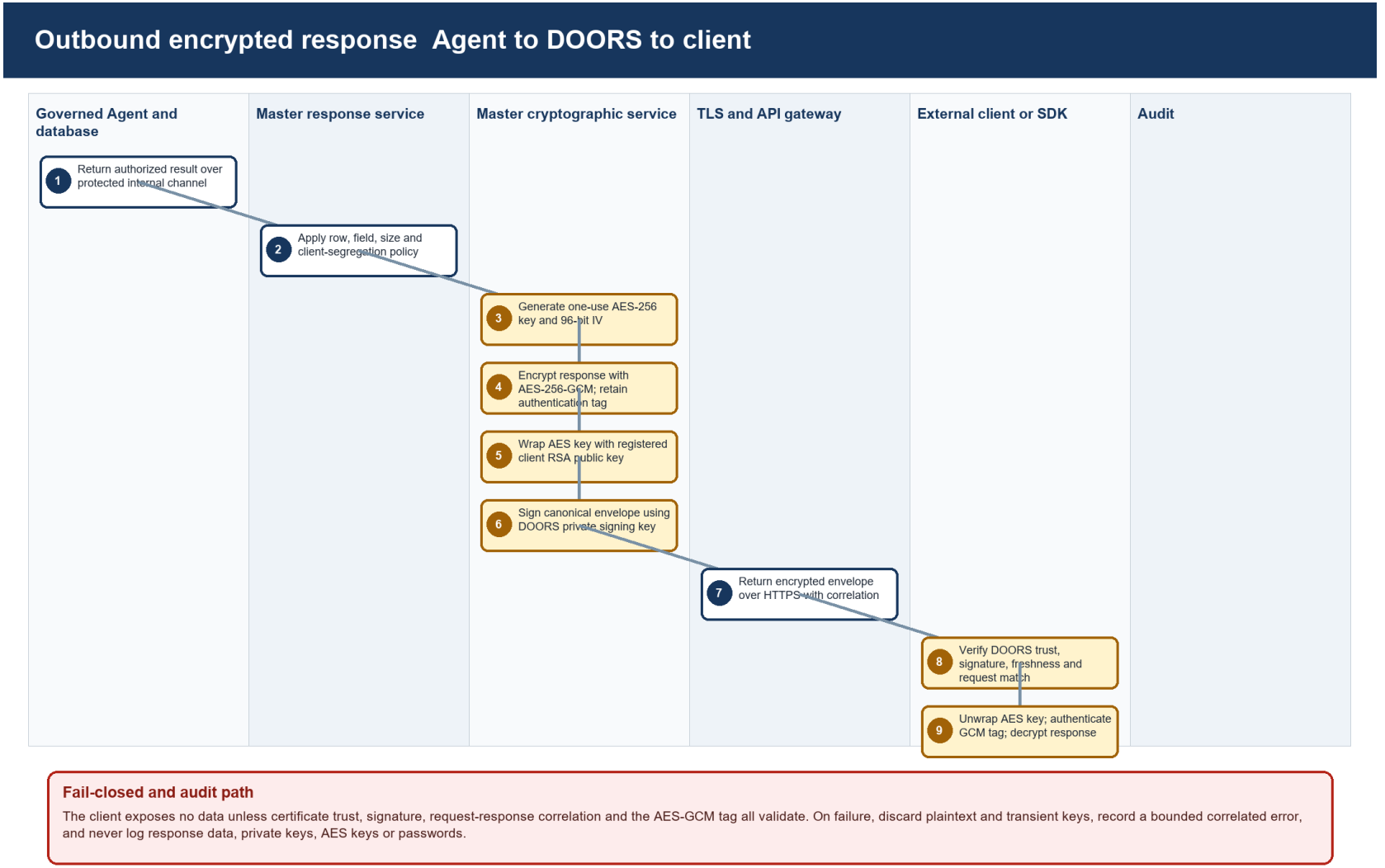


Figure 15.4 Outbound Encryption and Decryption Flow

15.4.1 Outbound Envelope Controls

Stage	Mandatory control	Failure outcome
Response preparation	Only authorized rows, fields and bounded results enter the canonical response	Unauthorized or oversized output is withheld
Content encryption	Fresh AES-256 key, unique 96-bit IV and authenticated GCM tag	Encryption failure returns no plaintext response
Recipient binding	AES key wrapped with the active registered client RSA public key	Missing, expired or revoked recipient key fails closed
DOORS authentication	Canonical response envelope signed using the protected DOORS signing key	Unsigned response is never released as trusted data
Transport	HTTPS carries protocol, key, IV, ciphertext, tag, request ID and trace correlation	Transport failure permits bounded retry under protocol rules
Client verification	Client validates DOORS certificate, signature, freshness and request-response correlation before decryption	Invalid or replayed response is discarded
Client decryption	Client private key unwraps AES key; AES-GCM tag is authenticated before plaintext is exposed	Unwrap, IV or tag failure exposes no data
Audit and disposal	Metadata and reason codes are recorded; transient keys and plaintext are not logged	Audit remains useful without retaining sensitive content

15.5 Cryptographic Audit and Error Rules

- Audit records contain the client, service, request ID, trace ID, selected key identifiers, protocol version, decision, bounded reason code and timing; they do not contain plaintext payloads or responses.
- Private keys, AES keys, PKCS12 content, passwords and unwrapped session material are never written to application logs, browser storage or audit tables.
- Signature verification, key unwrapping and GCM authentication occur before decrypted content is trusted or forwarded.
- Invalid certificate status, signature, key identifier, IV, authentication tag, freshness value, request correlation or replay state fails closed.
- Key rotation retains explicit active and retired identifiers so in-flight protocol handling is deterministic; revoked or expired keys cannot authorize new exchanges.

15.6 SDK Profiles

Profile	Contents and operator action
UAT	Includes the dedicated DOORS test identity, matching wallet configuration, trust store, sample JSON, source client, OpenAPI, HTML documents, checksum and email text. It runs with java WriteClient.java sample.json after extraction when environment URLs are correct.
Production	Does not distribute the client's private identity. The client creates or installs its PKCS12 file, protects the password, shares only the public certificate with the DOORS administrator, updates wallet paths and verifies the supplied SHA-256 checksum.
Swagger	Browser-guided test selects a PKCS12 identity, accepts the key password and JSON file or pasted payload, encrypts, posts and displays the response. Credentials remain in the active page memory only.

15.7 Idempotency

The client or SDK sends a UUID request identifier. The SDK generates and persists it automatically so ordinary users run only java WriteClient.java sample.json. A retry after timeout must reuse the same identifier and unchanged logical payload. A new or changed submission uses a new identifier.

16 Subscription Rate Limit and Usage Design

16.1 Subscription Controls

Control	Purpose
User licence	Links an API user to approved clients, Agents and service enablement
Accounting period	Defines independent access or billing periods without overwriting history
Date schedule	Allows client access on approved calendar dates and intervals
Weekly access	Defines recurring day and time windows
Notice	Publishes operator information and records per-user acknowledgement
Egress event	Records response bytes, count, duration and timing for usage accounting

16.2 Rate Limiting

Bucket4j-backed rate limits protect interactive and machine endpoints from bursts and loops before expensive work starts. A depleted bucket returns HTTP 429 and the caller must wait and retry with backoff. Rate limits govern request frequency. Write record-version limits separately govern the number of changed snapshots for one client and business key. Identical JSON consumes neither a new record version nor a version-limit slot.

17 AIra Design

17.1 Answer Strategy

AIra first attempts deterministic answers from the DOORS feature catalogue, operational workflows and authorised transaction aggregates. Only when those sources do not fully answer the question does it use retrieval and language generation. The response includes the supporting knowledge source and does not grant access to data the caller could not otherwise view.

17.2 Components

Component	Responsibility
AiraOperationalAnswerService	Recognize supported feature, workflow and activity questions and return deterministic answers
AiraKnowledgeService	Load approved documents, chunk text, build embeddings, index and retrieve relevant material
AiraContextService	Assemble role and conversation context
AiraService	Coordinate retrieval, generation, citation and safe fallback
AiraChatHistoryService	Persist conversations, ordered exchanges, evidence metadata and ratings
AIraController	Expose chat, refinement, status, history, ratings, analytics and controlled reindex

17.3 Knowledge Reindex

The authenticated administrative reindex endpoint rebuilds approved knowledge from packaged or configured sources. It is an operational action after deployment of changed knowledge content, not a substitute for vector retrieval. Packaged documents provide the current controlled baseline; embeddings and chunks are stored in the configured vector service for retrieval.

18 Data and Persistence Design

The detailed physical data dictionary, ER diagrams, ownership, grants and verification SQL are maintained in the Database Design Document. This section records how application components use those stores.

Store	Writer	Reader	Design rule
Master PostgreSQL	Master services and Flyway role	Master services and authorized reports	Governance, identity, mapping, audit and history only
GePNIC public schema	Existing GePNIC applications	Agent read role and existing applications	DOORS write package does not alter existing business objects
doors_write schema	DBA provisioning and SECURITY DEFINER functions	doors runtime through function; GePNIC reader through latest views	Append-only history and no runtime DDL
Vector store	AIra indexer	AIra retrieval service	Approved knowledge, not transaction truth
Document source	GePNIC application or document system	Agent document service	Master retains metadata and audit, not document bodies

18.1 Migration Strategy

- Master schema changes use ordered Flyway migrations committed with the Master source.
- Write service schemas and revisions use contract-specific downloadable SQL reviewed against the approved definition hash.
- Agent and Master binaries are deployed only when the corresponding runtime behaviour changes.
- Migration tests preserve existing rows, grants and function authority.
- Rollback uses transaction rollback or an approved forward fix; immutable business history is not discarded to simplify deployment.

18.2 Data Classification Retention and Disposal

Data class	Examples	Required treatment
Restricted secrets	Passwords, private keys, AES keys, transit secrets, database credentials and PKCS12 content	Never stored in ordinary tables, logs or browser storage; protected configuration or key store only; rotate and revoke under security procedure.
Sensitive transaction data	Decrypted parameters, JSON payloads, report results and document content	Process only for authorized purpose; minimize persistence; encrypt in transit; apply role, row, field, export and retention controls.
Security and audit evidence	Actor, client, IP, trace, decision, status, key identifier and timing	Append-oriented access, privileged search, integrity protection and approved retention; exclude secrets and unnecessary plaintext.
Governance metadata	Contracts, mappings, revisions, remarks, definition hashes and approvals	Retain as controlled history for traceability; do not overwrite approved evidence.
Operational telemetry	Health, latency, counts, pool use, errors and backup age	Bound cardinality and retention; restrict detailed diagnostics; purge according to operations policy.
Published documentation	Approved manuals, schemas, OpenAPI and SDK guidance	Version, checksum and publish only to authorized audiences.

Retention durations, legal holds, archival media, purge schedules and destruction evidence are environment governance values and must be approved before production. Application defaults must not silently become the retention policy.

19 Interface Design

19.1 Interface Conventions

Convention	Design
Base paths	Versioned REST paths separate auth, admin, master governance, external and inbound APIs
Media types	JSON for control and data envelopes; multipart for selected requests; octet stream for SDKs and document delivery
Correlation	DOORS trace ID accompanies errors and cross-service audit
Errors	HTTP status plus stable code and bounded human message; internal cause is logged
Pagination and bounds	List, report and audit interfaces use bounded result sets
Caching	Credentials, security state and generated SDK responses use no-store; static assets use deployment cache policy
OpenAPI	Machine interfaces publish contract-derived schemas, constraints and examples

19.2 Master to Agent Protocol

The Master resolves the registered Agent URL and operation-specific route. It sends a transit-authenticated HTTPS request with approved SQL or write metadata. The Agent never trusts a caller-supplied database URL, username, schema name or function name. Those values come from local configuration or the approved definition.

19.3 Database Interface

Read inputs are JDBC-bound parameters. Write inputs are arguments to the approved store function. Document manifest queries are approved SELECT definitions. Database exceptions are translated into stable operational outcomes while the exact SQL state and safe diagnostic context remain in service logs.

20 Security Design

20.1 Security Control Matrix

Threat	Control
Credential theft	Adaptive password hashing, MFA, no plaintext storage, API-key rotation and private-key custody
Cross-site request forgery	CSRF token on state-changing interactive requests and same-origin cookies
Host or route confusion	Plane host resolver and fail-closed plane routing
Spoofed proxy headers	Trusted proxy validation before forwarded addresses are used
API replay	Request identifiers, timestamps or protocol freshness and stored gateway/write claims
Payload disclosure	TLS plus configured AES-GCM application envelope
Payload tampering	AES-GCM authentication and RSA signature verification
SQL injection	Approved SELECT text, parser validation and bound p_ parameters
Unauthorized write	Independent contract approval, verified deployment, client permission and database service grant
Path traversal	Opaque document identifiers, canonical root resolution and type policy
Privilege escalation	Server-side RBAC, least-privilege database roles and separation of duties

Threat	Control
Abuse or loops	Rate limiting, schedules, row limits, timeouts and payload limits
Audit evasion	Trace correlation and append-oriented audit/history records

20.2 Cryptographic Key Boundaries

- DOORS signing and encryption private keys remain in protected Master key stores.
- Client private keys remain with the client; production SDKs distribute only instructions and public material.
- UAT identity files are dedicated test credentials and must not be reused in production.
- Transit secrets are shared only between the applicable Master and Agent deployments.
- Database passwords, wallet passwords and key passwords are supplied through protected configuration or service environment, never committed to source.

20.3 Logging Restrictions

Operational logs may contain decrypted request parameters, client name and API key to support authorized Data Manager troubleshooting. Passwords, private keys, AES keys, transit secrets, database credentials and raw PKCS12 content must never be logged. Access to execution logs is therefore privileged and subject to retention and review policy.

20.4 Security Assumptions and Review Triggers

- System clocks are synchronized closely enough for certificate, token, freshness and audit correlation checks.
- Certificate authorities, revocation information and public-key associations are administered outside the application under approved custody.
- Proxy and load-balancer changes cannot introduce an untrusted forwarded identity or bypass plane routing.
- The database SECURITY DEFINER owner is not the runtime login and functions use controlled search paths and grants.
- A new external protocol, cryptographic algorithm, identity provider, document source, AI provider, public route or cross-database capability triggers security architecture review.
- Penetration findings rated critical or high block release until remediated or formally risk accepted by the competent authority.

21 Audit Logging and Observability

21.1 Audit Context

Field group	Examples
Actor	Username, role, client ID, client name and API key where authorized for troubleshooting
Request	Endpoint, method, source IP, plane, parameters, request ID and trace ID
Target	Query, service, Agent, instance context and downstream URI
Outcome	HTTP status, stable code, message, record count, bytes and write outcome
Timing	Start, execution time, downstream duration and completion time
Governance	Proposer, reviewer, definition hash, version, decision and remarks

21.2 Diagnostic Flow

- Start with the client-visible status, stable code and DOORS trace ID.
- Search Execution Logs by trace, API key, client, endpoint or time.

- Confirm client activation, IP, subscription, schedule and service mappings.
- Confirm Agent availability and the corresponding Agent trace or log entry.
- For writes, inspect the receipt using the request ID before retrying.
- Escalate only after preserving the relevant logs, configuration version and deployment hash.

21.3 Health and Metrics

Agent readiness endpoints report application and database status. Master dashboards summarize users, templates, requests, clients and service activity. Subscription egress tables record usage. Alra exposes status and analytics. Infrastructure monitoring should additionally cover process health, TLS expiry, database pools, latency, locks, disk, backup age, rate-limit events and repeated authorization failures.

21.4 Monitoring Ownership and Alert Outcomes

Signal	Minimum outcome
Service or readiness failure	Alert operations, identify affected plane or Agent, preserve trace context and suppress unsafe traffic where required.
Certificate or secret expiry	Warn before the approved renewal window; escalate before service interruption; verify replacement on every dependent component.
Repeated authentication, signature or authorization failure	Correlate by client, account, IP and service; alert security without logging protected payloads.
Database pool, lock, disk or latency saturation	Throttle or fail bounded requests, preserve database safety and trigger capacity investigation.
Backup age or recovery evidence breach	Escalate to service owner and DBA; production approval remains conditional until protection is restored.
Audit pipeline failure	Fail or restrict sensitive operations according to approved policy and restore audit integrity before normal service.

22 Error Handling Resilience and Recovery

Failure	Design response	Operator action
Validation failure	Return 400 or 422 with stable code; do not invoke Agent	Correct request or approved definition
Authentication or authorization failure	Return 401 or 403 without disclosing protected state	Check identity, IP, mapping and policy
Rate limit	Return 429	Wait for the window and use backoff
Agent unavailable	Return bounded gateway/service error and trace	Check network, proxy, Agent service and transit secret
Write timeout after invocation	Return result unknown when receipt cannot be confirmed	Retry same request ID and payload
Database constraint or permission failure	Rollback transaction and log SQL state	DBA compares installed objects and grants with approved SQL
Export still running	Return job status rather than blocking indefinitely	Poll within documented interval
Alra retrieval unavailable	Return deterministic or safe fallback answer	Check knowledge index and configured services
Master restart	Persistent governance and receipts survive; active in-memory sessions may require reauthentication	Restart under service manager and verify health

22.1 Recovery Principles

- Back up Master and GePNIC databases according to approved RPO and RTO.

- Restore write history, head and receipt tables to a mutually consistent point.
- Reconcile Flyway history, generated write definition hash and deployed JAR hashes after recovery.
- Reindex Alra knowledge from approved sources when the vector store is rebuilt.
- Verify read, write, document, audit and unaffected-service smoke tests before reopening access.

22.2 Availability Continuity and Recovery Objectives

Objective	Design position	Approval evidence
RPO	No universal value is embedded in code; Master metadata and each GePNIC database require an approved business RPO.	Backup schedule, successful job history and data-owner approval
RTO	Recovery order is proxy and trust services, databases, Agent, Master, UI and representative verification; the target is environment specific.	Timed recovery rehearsal and operations sign-off
Agent outage	Only services mapped to the unavailable Agent fail; other Agents and governance functions remain isolated where dependencies permit.	Failure-injection or controlled outage test
Master outage	Governance and external orchestration pause; durable metadata and write receipts survive restart.	Restart, database consistency and session behaviour test
Database outage	Requests fail boundedly and do not fall back to an unapproved source or cached protected response.	Connection-loss and recovery test
Key or certificate incident	Affected trust is revoked or rotated; dependent clients and services are revalidated before reopening.	Rotation or revocation rehearsal and audit evidence

23 Configuration Build and Deployment

23.1 Configuration Domains

Domain	Examples	Rule
Server and proxy	Port, forwarded-header strategy, trusted proxies and body limits	Private application ports; proxy is the public boundary
Database	JDBC URL, role, pool and Flyway settings	Secrets externalized; Agent profiles identify the local database
Planes	Admin, external and API hosts and paths	Exact environment hostnames; fail closed
Security	JWT, password, MFA, IP, captcha, rate limits and certificates	Environment-controlled and auditable
Write SDK	UAT identity, wallet and public download base URL	UAT only contains test identity; production requires client identity
Agent inbound	Enabled flag, governance flag, transit secret, targets and sandbox	Target key must match the mapping context expected by the Agent
Alra	Model endpoint, model name, embedding and vector options	Approved local or managed services only
Documents	Roots, timeouts, limits and policies	No caller-controlled root path

23.2 Build Artefacts

Component	Build	Deployable
UI	npm build using the committed Node engine and package lock	dist directory served by the reverse proxy
Master API	Maven package with Java 21	masterapi-0.0.1-SNAPSHOT.jar plus external configuration and key material

Component	Build	Deployable
Agent API	Maven package with Java 21	agent-api-0.0.1-SNAPSHOT.jar plus staging or production application.yml
Database	Flyway migrations or approved generated SQL	Versioned migration scripts with checksum and execution evidence
SDK	Generated per authorized client-service mapping	ZIP, SHA-256 file and email text for UAT or production

23.3 Deployment Order

1. Back up affected databases and export the current schema and privilege baseline.
2. Verify source commit, JAR or dist checksum and configuration compatibility.
3. Run Master database migrations before starting a Master build that depends on them.
4. Deploy the Agent when its route, verification, query, document or write behaviour changed.
5. Deploy the Master API and confirm authentication, CSRF, health and Agent connectivity.
6. Deploy the UI dist and invalidate or version static caches.
7. Run DBA provisioning or migration SQL for each approved write contract, then verify through DOORS.
8. Reindex AIra after knowledge changes.
9. Run smoke tests for login, read, write, document, reports, audit and unaffected routes.

23.4 Capacity Performance and Scalability

Control point	Required bound or design evidence
Proxy	Approved connection, request-body, header, upload and upstream timeout limits for each request plane.
Master API	Thread or virtual-thread behaviour, database and HTTP pool sizes, request timeouts, export workers and memory limits tested together.
Agent API	Per-target connection pool, statement timeout, row limit, payload limit and concurrent execution bound.
Database	Query plans, indexes, lock behaviour, write-function cost, storage growth and maintenance windows reviewed by DBA.
Documents and exports	File size, result rows, concurrent jobs, temporary storage, expiry and download bandwidth are bounded.
Cryptography	Signing, verification, RSA wrapping and AES-GCM throughput tested at the approved payload and concurrency profile.
Acceptance	Record the workload model, dataset size, concurrency, percentiles, error rate, resource saturation point and headroom. Production limits must reflect measured evidence rather than workstation results.

23.5 Release Configuration Baseline

- Record UI, Master, Agent, migration, generated SQL and documentation hashes in one release manifest.
- Record effective host-plane rules, Agent targets, database identities, feature flags, limits, certificate identifiers and trust-store versions without copying secrets.
- Confirm all production placeholders are resolved and all test identities, URLs and permissive flags are absent.
- Store the approved manifest, test evidence, DBA verification, rollback plan and completed approvals with the release record.

24 Verification Testability and Traceability

24.1 Automated Test Areas

Area	Evidence in source

Area	Evidence in source
UI security and workflows	Vitest suites for SQL parameters, PDF guides, write keys, revisions, SDK release, AES Swagger and browser flows
Master security	Spring Security integration tests, trusted routing and HTTP admission tests
Read governance	Query approval, parser and parameter tests
Write governance	Complete flow, sandbox, Agent mapping, PostgreSQL, OpenAPI, SDK, version limit and additional-field tests
Agent execution	Database routing, JSON write engine, automatic versioning, governance logging and PostgreSQL tests
Alra	Knowledge and coverage tests for feature, workflow and activity questions
Documents and reports	Policy, eligibility, mapping, download and structured report tests

24.2 Design Verification Matrix

Design objective	Verification
No self-approval	Negative governance tests and audit review
Only approved SELECT execution	Parser unit tests and Agent dry-run negative tests
No runtime write DDL	Database role inspection and function-only smoke test
Idempotent write retry	Same request and payload returns replayed result without new version
Identical JSON does not version	New request ID and identical JSON returns UNCHANGED
Changed JSON versions automatically	Changed payload stores exactly next record version
Client version limit	Changed payload beyond configured limit returns conflict
Plane isolation	Wrong host or route returns fail-closed response
Document path safety	Traversal and unauthorized type tests
Audit correlation	Trace is visible across Master, Agent and stored audit

24.3 Release Acceptance

A release is acceptable when source, migrations, configuration, deployment hashes and operational evidence identify the same baseline; critical tests pass; roles and grants match the approved design; no secret is committed or exposed; rollback is defined; and representative read, write, document and reporting requests succeed through the intended request planes.

24.4 Feature to Design Traceability

Requirement domain	Principal design sections	Acceptance evidence
Identity and access	6, 9, 10 and 20	Registration, approval, password, MFA, role, IP, session and negative authorization tests
Read services	7, 8, 10, 11 and 19	Parser, bound-parameter, dry-run, approval, Agent routing, segregation and result-bound tests
Write services	8, 10, 12, 15 and 18	JSON analysis, independent approval, sandbox, DBA package, Agent verification, idempotency and version tests
Machine security	15, 16 and 20	Client activation, mapping, subscription, replay, signature, key-wrap, AES-GCM and fail-closed tests
Documents	13, 19 and 20	Eligibility, canonical path, type, size, expiry, consumption, checksum and unauthorized-download tests

Requirement domain	Principal design sections	Acceptance evidence
Reports and exports	14, 16 and 23	Authorized catalogue, bounded execution, export lifecycle, expiry, egress and capacity tests
Audit and operations	21, 22 and 23	End-to-end trace, alert, backup, recovery, rotation, deployment and rollback evidence
User experience	6 and Appendices G-H	Approved wireframes, role navigation, validation, error state and workflow walkthroughs

24.5 Final Design Review Decision

The software design is complete enough to proceed to formal approval and controlled release when every blocking item in Appendix K is evidenced. The document does not waive unresolved production values or approvals. Any unchecked security, database, recovery, capacity or traceability gate remains a release blocker unless the competent authority records a time-bound risk acceptance.

Appendix A Component Catalogue

Repository	Implementation area	Representative components
doors-ui	Presentation	MainLayout, router, AIraChat, governance views, report views, auth views, API guide PDF and write SDK release utilities
doors-master-api	Web and application	Controllers, filters, governance services, gateway services, document services, AIra services, repositories and Flyway
doors-agent-api	Execution	Query, status, inbound governance and inbound write controllers; JSON write engine and JDBC mappers
Generated database package	Destination persistence	Service history, head, receipt, grant, latest view and SECURITY DEFINER store function

Appendix B Endpoint Families

Family	Base route	Operations
Authentication	/api/v1/auth	Captcha, CSRF, login, logout, MFA, registration, current user and password change
Users	/api/v1/admin/users	List, approve, reject, role, MFA reset, password reset and VPN/IP policy
Agents	/api/v1/master/agents and infrastructure routes	Registry, active list, status and mapping
Read templates	/api/v1/master/templates	Submit, list, inspect, map, dry run and status
Query governance	/api/v1/master/governance	Review and approve query changes
Template contracts	/api/v1/admin/template-contracts	Discover, draft, submit, approve, reject, restore and history
API clients	/api/v1/master/api-clients	Register, activate, encryption, Agents, queries, filters, security and audit
Read gateway	/api/v1/master/gateway/orchestrate	Machine read orchestration
Write governance	/api/v1/master/governance/writes	Analyse, propose, sandbox, submit, review, catalogue, deploy, map and verify
Write clients	/api/v1/admin/write-clients	Create, attach, security and key rotation
Write integrations	/api/v1/admin/write-integrations	Guide, OpenAPI, SDK, scope and downloadable assets

Family	Base route	Operations
Write gateway	/api/v1/inbound	Encrypted write submission
Documents	/api/v1/master documents and gateway routes	Policies, registry, grants, manifests, download and reporting
Reports	/api/v1/reports	Templates, Agents, execute, export status and download
Subscriptions	/api/v1/master/api-subscriptions	Licence, account, periods, schedule, weekly access and notices
AIra	/api/v1/master/aira	Chat, refine, status, history, rating, analytics and reindex
Agent query	/v1/agent/query	Execute, stream, dry run and document eligibility
Agent governance	/v1/agent/inbound/governance	Sandbox and schema verification
Agent write	/v1/agent/inbound	Internal governed write

Appendix C Configuration Catalogue

Configuration group	Purpose
server and spring	Ports, virtual threads, uploads, datasource, JPA and Flyway
doors.planes	Admin, External and API host rules
doors.security	JWT, proxy, IP, MFA, passwords, rate limits and security policy
doors.contracts	Schema-contract and governance settings
doors.documents	Document root, streaming, timeouts and limits
doors.aira	Model, embeddings, knowledge and retrieval
doors.write-sdk	UAT identity, wallet and delivery settings
doors.inbound	Master inbound routes, encryption keys and Agent transit configuration
Agent app.security	Master-to-Agent request authentication
Agent inbound.targets	Per-Agent local database profile
Agent inbound.sandbox	Sandbox database owner and temporary-schema settings
logging	Package levels, file appenders, rotation and protected diagnostics

Appendix D State Models

Object	Principal states
User	Pending, approved or active, disabled or rejected, reset required and MFA pending or enabled
Query	Draft, tested, pending, approved, rejected and disabled
Template contract	Draft, pending, approved, rejected and restored revision
Write contract	Draft, sandbox tested, pending, approved, rejected and disabled
Write deployment	Not provisioned, verification pending, verified and failed or disabled
Client	Inactive and active with independent credential and mapping readiness
Document grant	Pending or active, expired, revoked or consumed according to policy

Object	Principal states
Export job	Submitted, running, completed, failed and expired

Appendix E Acronyms and Terms

Term	Meaning
DOORS	Data Orchestration and On Demand Reporting System
HOG	Head of Group
RBAC	Role Based Access Control
MFA	Multi Factor Authentication
CSRF	Cross Site Request Forgery
JWT	JSON Web Token
AES GCM	Advanced Encryption Standard in Galois Counter Mode
PKCS12	Password-protected identity container commonly stored as p12 or pfx
Agent	DOORS execution service deployed near one GePNIC database
Record key	Approved JSON fields that identify one versioned business record
Definition hash	SHA-256 digest that binds generated SQL and runtime requests to the approved contract
Receipt	Stored request fingerprint and response used for deterministic retries
Plane	Host and route boundary for Admin External or machine API traffic

Appendix F Source Baseline

This document was reverse engineered from the current doors-write-feature branch of the UI, Master API and Agent API repositories after commits cbb5726, bc7d5df and a804bdf. The baseline includes Master Flyway migrations through V2026102202, the current Vue router and views, Spring controllers and services, Agent inbound and query components, packaged Alra knowledge, generated write SQL patterns, automated tests, and the related SRS, Database Design Document and functional manual. Environment-specific configuration and the deployed PostgreSQL catalogue must be attached during formal audit to confirm variance from this source baseline.

Appendix G User Interface Screen Catalogue and Wireframes

This appendix is the controlled wireframe baseline for every routed user-facing screen in the current Vue application. A reused component is documented once and all route aliases are shown. Dialogs, tabs, loading, empty, validation, access-denied and error presentations are significant states of the parent screen rather than separate screens. The wireframes describe information architecture and control placement; the deployed UI and approved functional manual remain the authority for final visual styling and operating instructions.

G.1 Screen completeness and approval rule

Every user-facing screen shall have an approved wireframe documenting its layout, controls, authorized roles, validation rules, navigation paths and significant UI states. A screen is complete for design review only when its route, role, principal content, actions, API responsibility and related workflow are traceable.

ID	Screen	Route or alias	Authorized role	Primary purpose
UI-01	Login and MFA	/login	Public; all authenticated roles	Portal choice, username, password, captcha, MFA enrolment or verification
UI-02	Access Registration	/register	Public	Identity, organisation, official email and requested role
UI-03	Change Password	/change-password	Authenticated user with reset requirement	Current password, new password and confirmation
UI-04	Data Manager Dashboard	/dashboard	DataManager	Operational summaries, governance insights and recent activity
UI-05	Data Viewer Dashboard	/dashboard/dataviewer/dashboard	DataViewer	Read-only service and reporting entry points
UI-06	Developer Dashboard	/dashboard/developer/dashboard	Developer	Submission status and query-authoring entry points
UI-07	API User Dashboard	/dashboard/api-user/dashboard	ApiUser	Subscription, licence, schedule, usage and download status
UI-08	API User Client Management	/dashboard/api-user/clients	ApiUser	Assigned clients, credentials, SDK and integration material
UI-09	External User Dashboard	/dashboard/external/dashboard	External	Data-request summary and self-service entry points
UI-10	External Data Request	/dashboard/external-request	External; ApiUser	Request purpose, category, data need and supporting details
UI-11	My Data Requests	/dashboard/my-requests	External; ApiUser	Submitted requests, decisions and fulfilment status
UI-12	Request Submission	/dashboard/query-submission	DataManager; Developer; ADMIN	Approved assignment, SELECT definition or write JSON proposal
UI-13	Submission History	/dashboard/my-submissions	Authenticated author	Own submissions, state, remarks and revision history
UI-14	Pending Approvals	/dashboard/governance/pending	DataManager	Pending read-query or request review queue
UI-15	Query Library	/dashboard/governance/approved	DataManager	Approved and published read services
UI-16	Infrastructure and Agents	/dashboard/governance/infrastructure	DataManager	Agent registry, health, connectivity and ownership
UI-17	API Client Registry	/dashboard/governance/api-clients	DataManager	Read and write clients, credentials, IPs, mappings and security
UI-18	Template Contract Governance	/dashboard/governance/template-contracts	DataManager	Schema discovery, drafts, revisions and decisions
UI-19	API Subscriptions	/dashboard/governance/api-subscriptions	DataManager	Licence, periods, schedules, usage and notices
UI-20	Document Download Policies	/dashboard/governance/document-policies; /dashboard/documents/policies	DataManager	Eligibility, limits, expiry and download controls
UI-21	Write Library	/dashboard/governance/write-library (alias inbound-services)	DataManager	Approved write contracts, revisions, readiness and integration assets
UI-22	Document Service Registry	/dashboard/governance/document-services; /dashboard/documents/services	DataManager	Document definitions, Agent mappings and availability
UI-23	Category Management	/dashboard/governance/categories	DataManager	Request and service categories
UI-24	Data Request Management	/dashboard/governance/data-requests	DataManager	External request review, assignment and decision
UI-25	Signing Key Management	/dashboard/governance/signing-keys	DataManager	Signing certificates, validity and rotation status
UI-26	User Management	/dashboard/governance/users; /dashboard/security/users	DataManager; SecurityAdmin	Pending, active and rejected users with role and access controls
UI-27	User Agent Matrix	/admin/users/matrix	DataManager	User-to-Agent authorization matrix
UI-28	System Documentation	/dashboard/documentation	All authenticated roles	Role-filtered manuals, guides and downloads
UI-29	Document Dashboard	/dashboard/documents/dashboard	DataManager	Document-service counts, policy status and delivery activity
UI-30	Document Download Logs	/dashboard/documents/logs	DataManager	Delivery attempts, outcomes, actors and trace evidence
UI-31	Audit Logs	/dashboard/audit	DataManager	Governance and execution events with status, trace and latency
UI-32	Report Library	/dashboard/reporting/report-library	DataManager; DataViewer;	Authorized report templates and

ID	Screen	Route or alias	Authorized role	Primary purpose
			External; ApiUser	descriptions
UI-33	Report Viewer	/dashboard/reporting/report-viewer	DataManager; DataViewer; External; ApiUser	Parameters, execution status, results and export

G.2 UI-01 Login and MFA

Route: /login **Authorized roles:** Public; all authenticated roles

Design intent: Portal choice, username, password, captcha, MFA enrolment or verification. Principal actions: Sign in; refresh captcha; verify MFA; register.

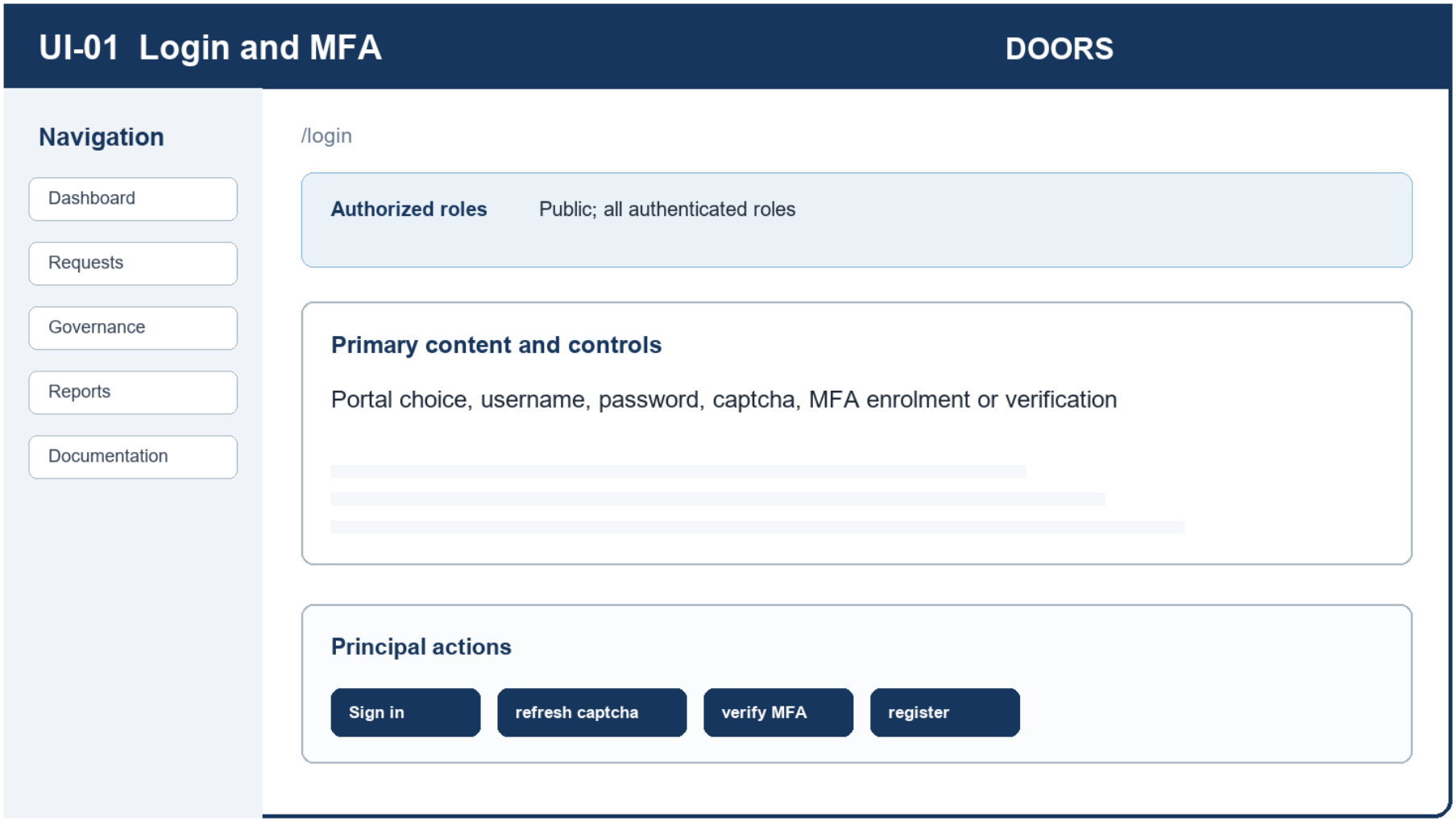


Figure G-1 UI-01 Login and MFA wireframe

G.3 UI-02 Access Registration

Route: /register **Authorized roles:** Public

Design intent: Identity, organisation, official email and requested role. Principal actions: Submit access request; return to login.

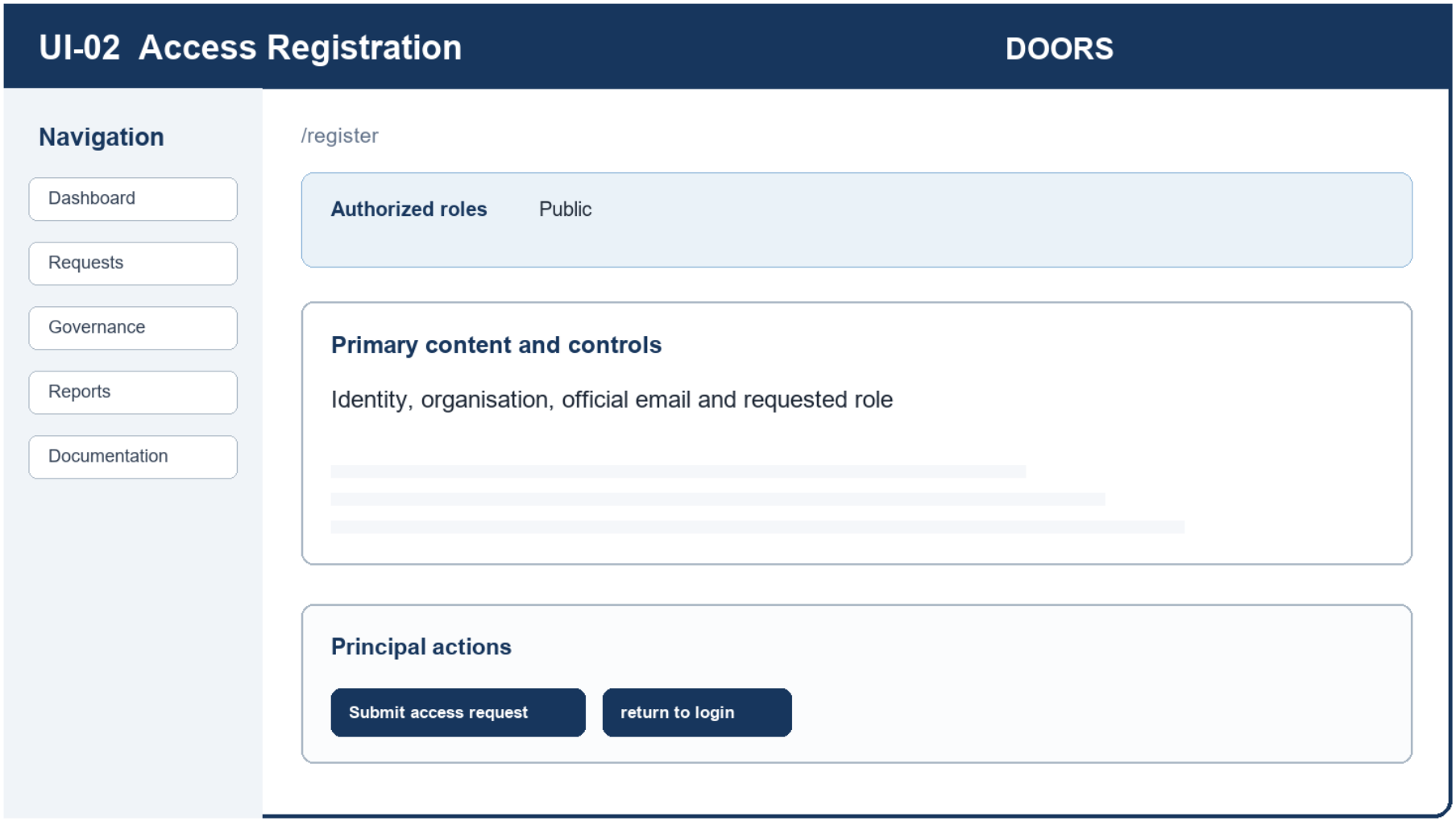


Figure G-2 UI-02 Access Registration wireframe

G.4 UI-03 Change Password

Route: /change-password **Authorized roles:** Authenticated user with reset requirement

Design intent: Current password, new password and confirmation. Principal actions: Update password.

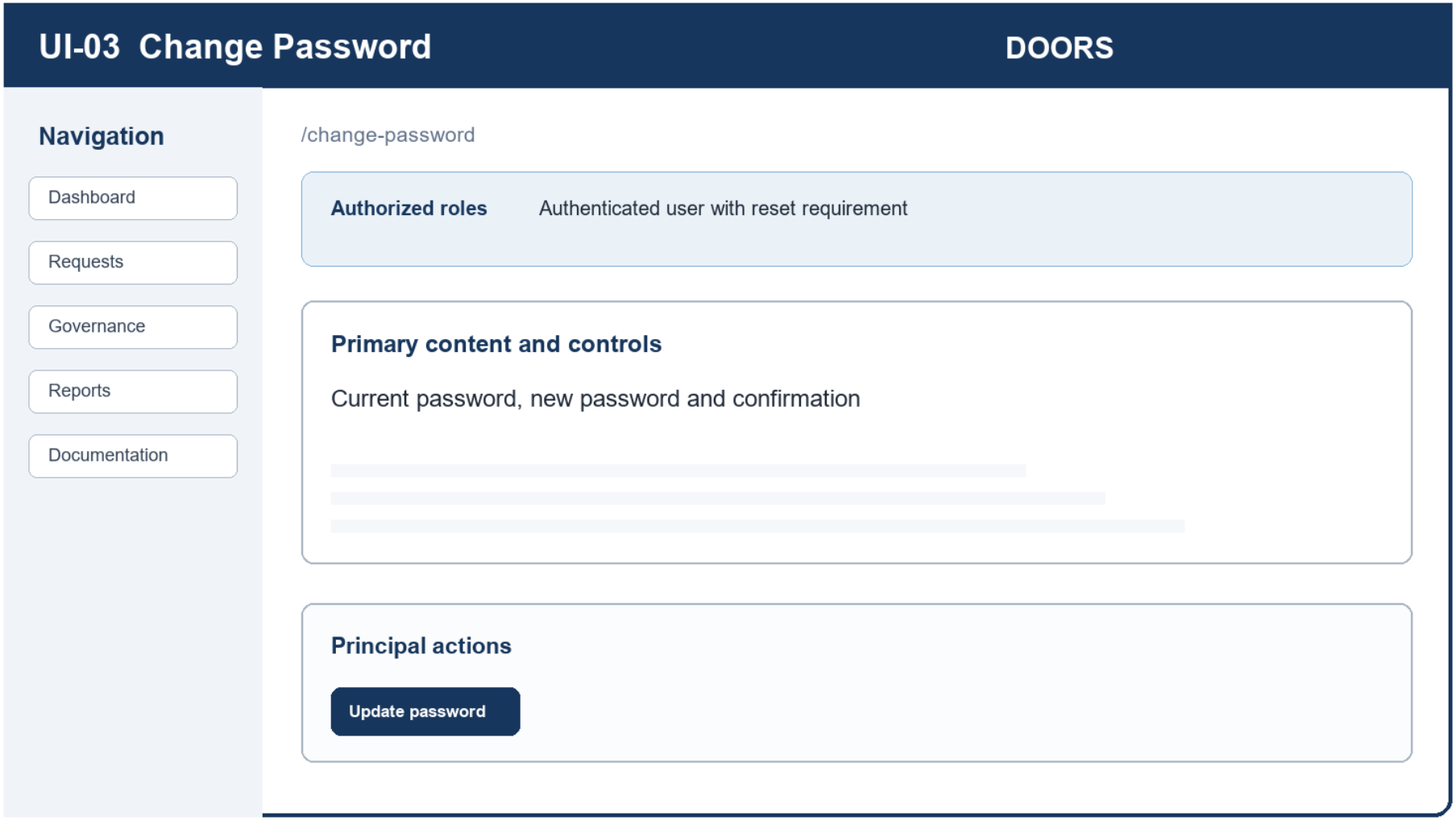


Figure G-3 UI-03 Change Password wireframe

G.5 UI-04 Data Manager Dashboard

Route: /dashboard **Authorized roles:** DataManager

Design intent: Operational summaries, governance insights and recent activity. Principal actions: Open governance, subscriptions or audit.

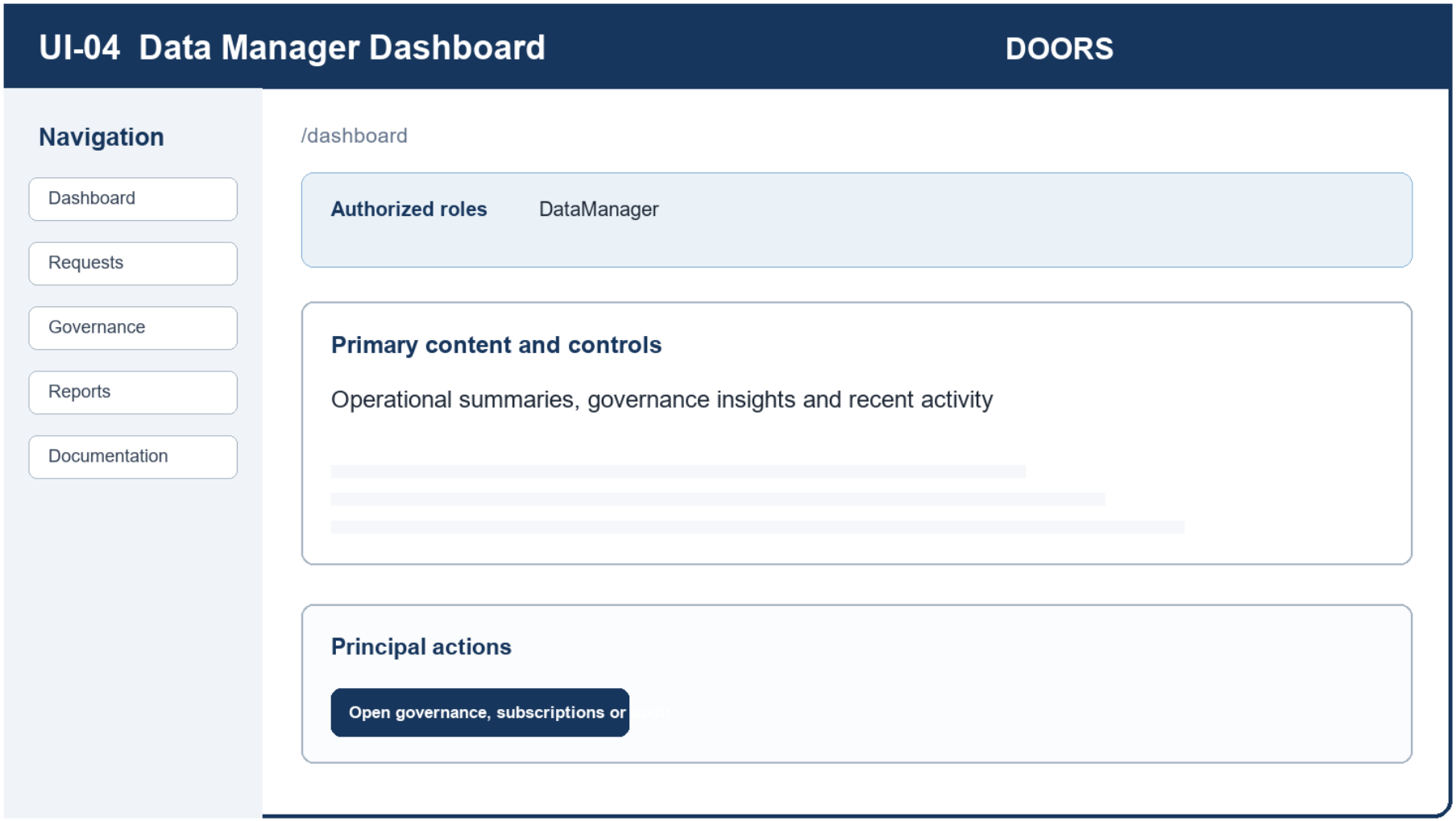


Figure G-4 UI-04 Data Manager Dashboard wireframe

G.6 UI-05 Data Viewer Dashboard

Route: /dashboard/dataviewer/dashboard **Authorized roles:** DataViewer

Design intent: Read-only service and reporting entry points. Principal actions: Open reports or documentation.

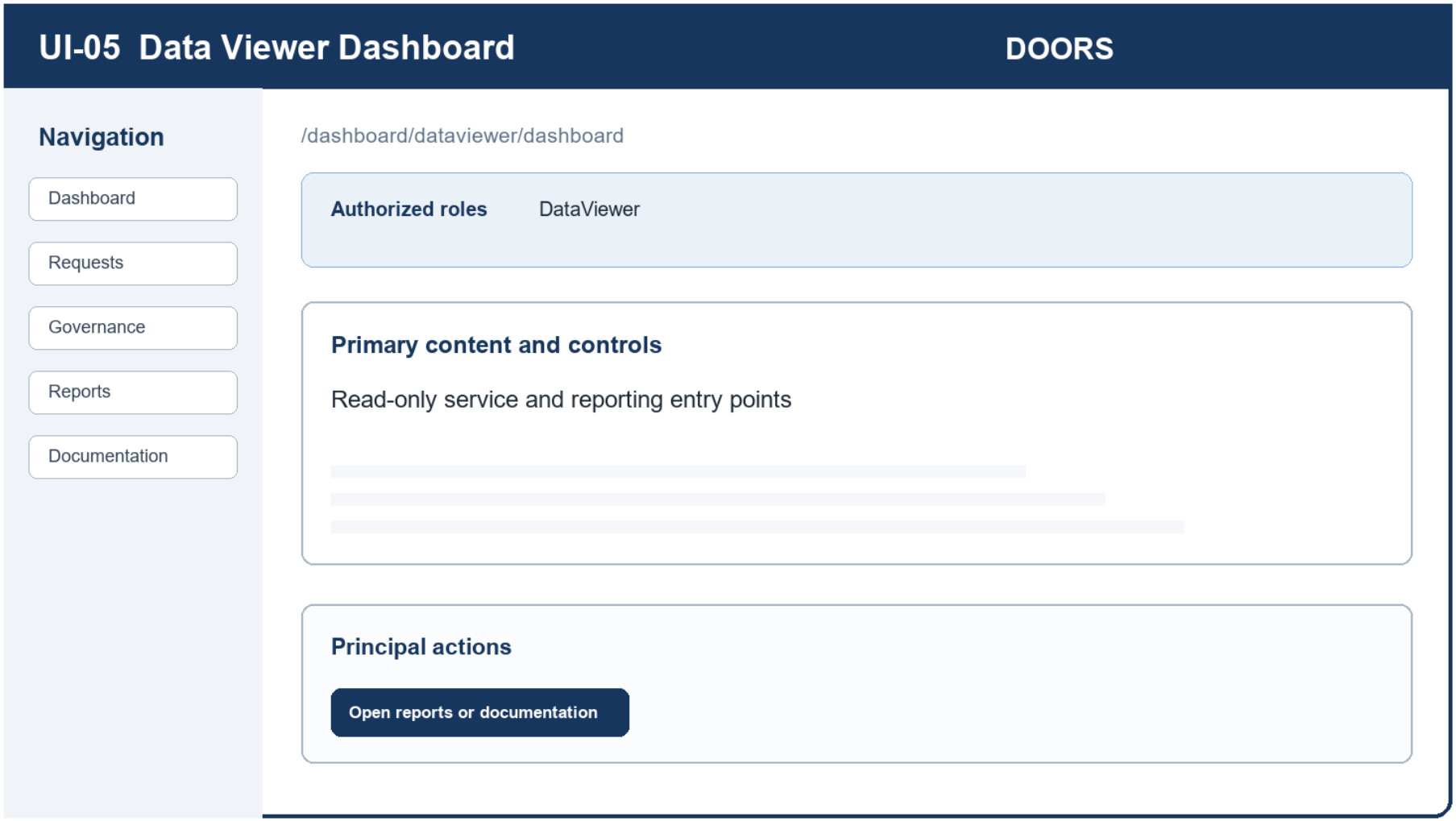


Figure G-5 UI-05 Data Viewer Dashboard wireframe

G.7 UI-06 Developer Dashboard

Route: /dashboard/developer/dashboard **Authorized roles:** Developer

Design intent: Submission status and query-authoring entry points. Principal actions: Create query; review submissions.

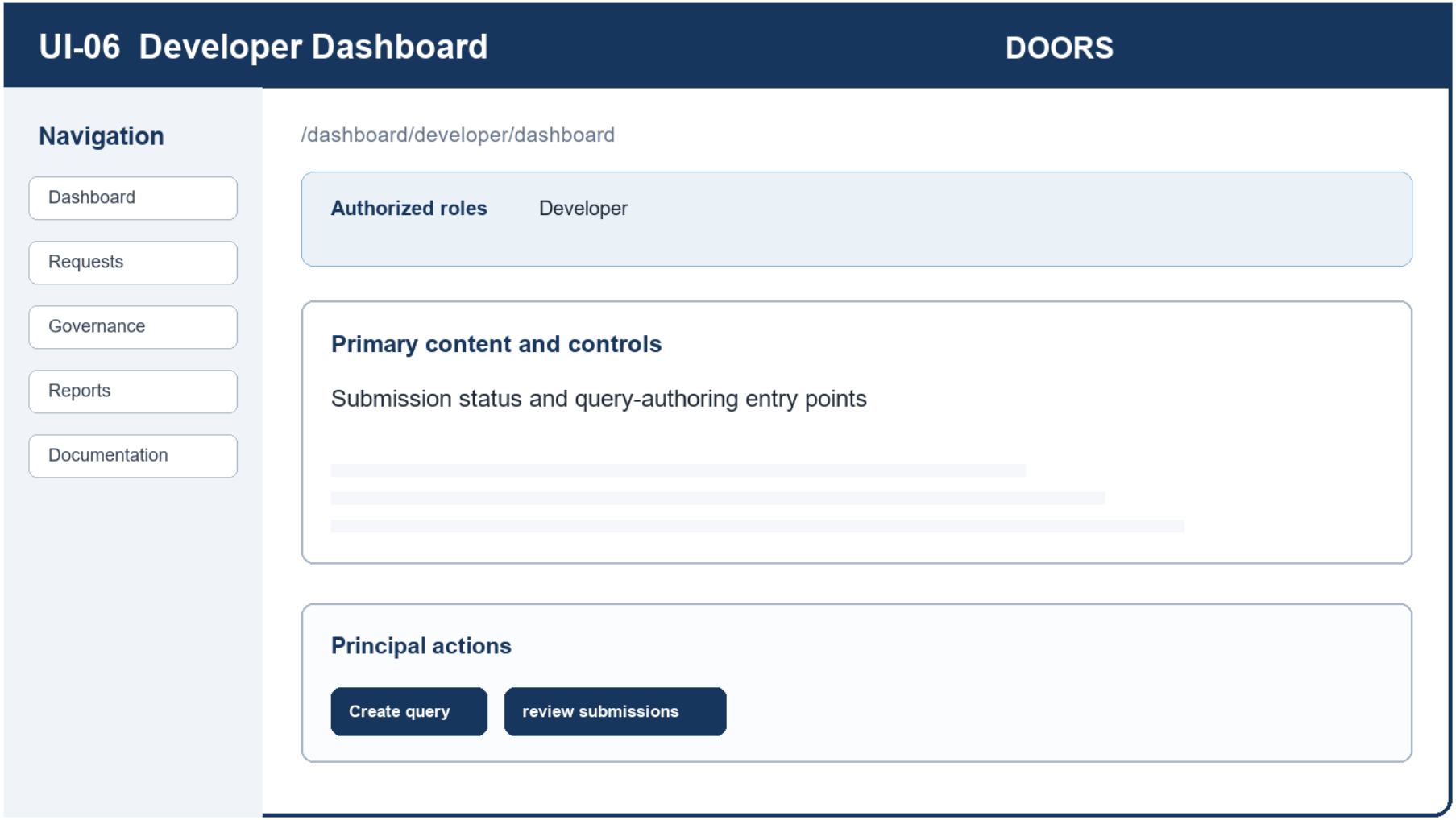


Figure G-6 UI-06 Developer Dashboard wireframe

G.8 UI-07 API User Dashboard

Route: /dashboard/api-user/dashboard **Authorized roles:** ApiUser

Design intent: Subscription, licence, schedule, usage and download status. Principal actions: Inspect subscription; open assigned clients.

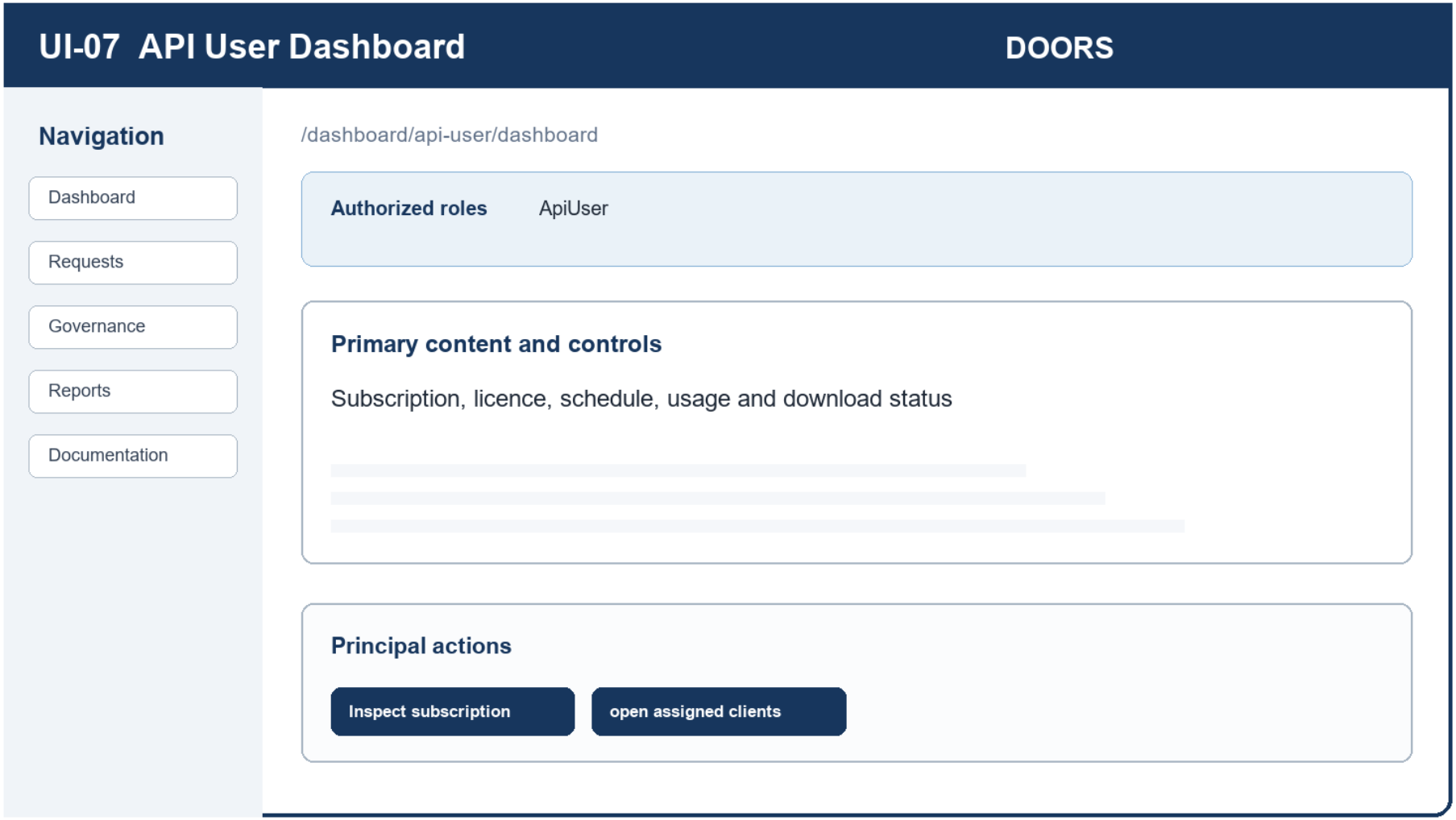


Figure G-7 UI-07 API User Dashboard wireframe

G.9 UI-08 API User Client Management

Route: /dashboard/api-user/clients **Authorized roles:** ApiUser

Design intent: Assigned clients, credentials, SDK and integration material. Principal actions: Download assets; inspect configuration.

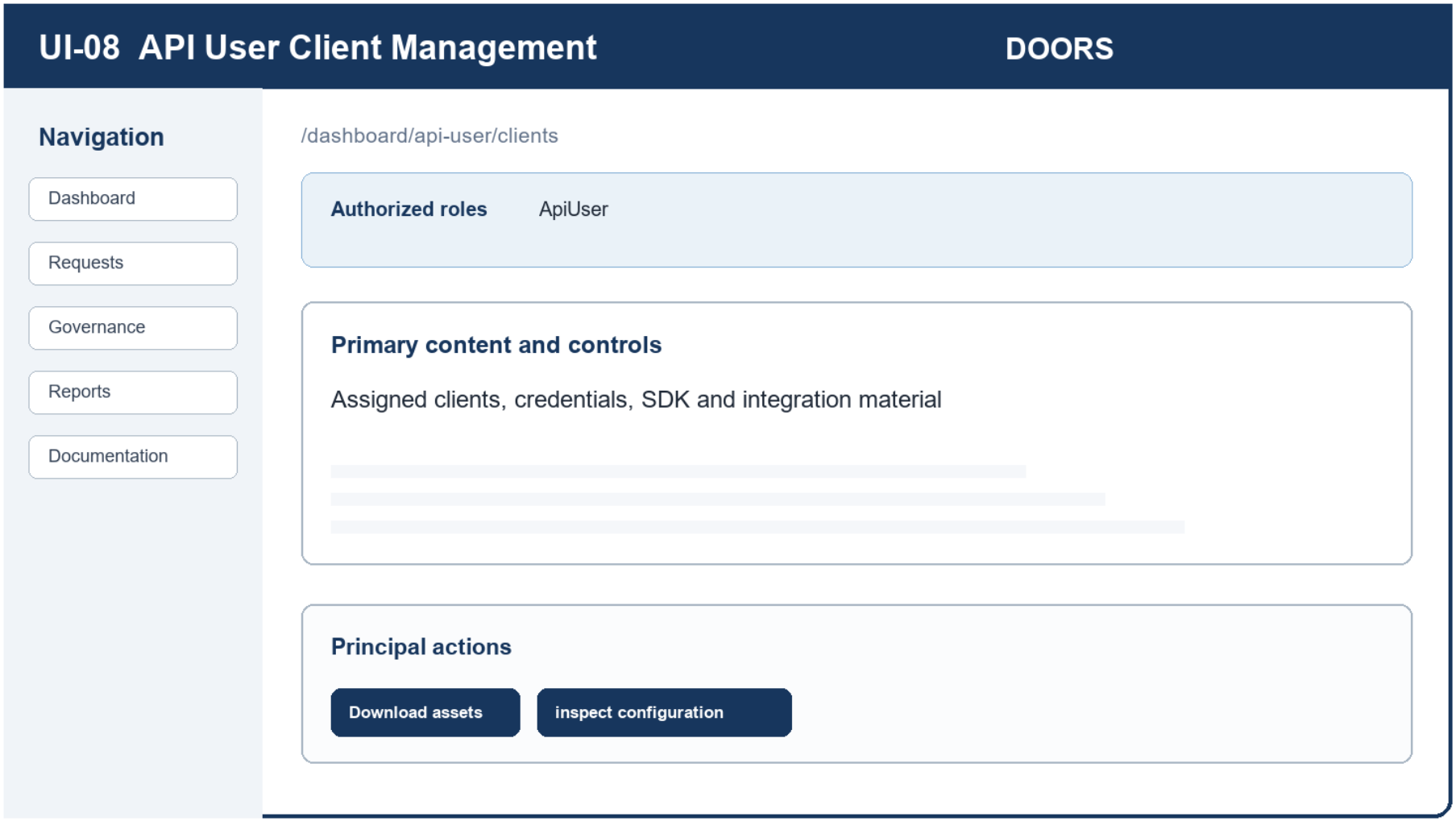


Figure G-8 UI-08 API User Client Management wireframe

G.10 UI-09 External User Dashboard

Route: /dashboard/external/dashboard **Authorized roles:** External

Design intent: Data-request summary and self-service entry points. Principal actions: Create request; view own requests.

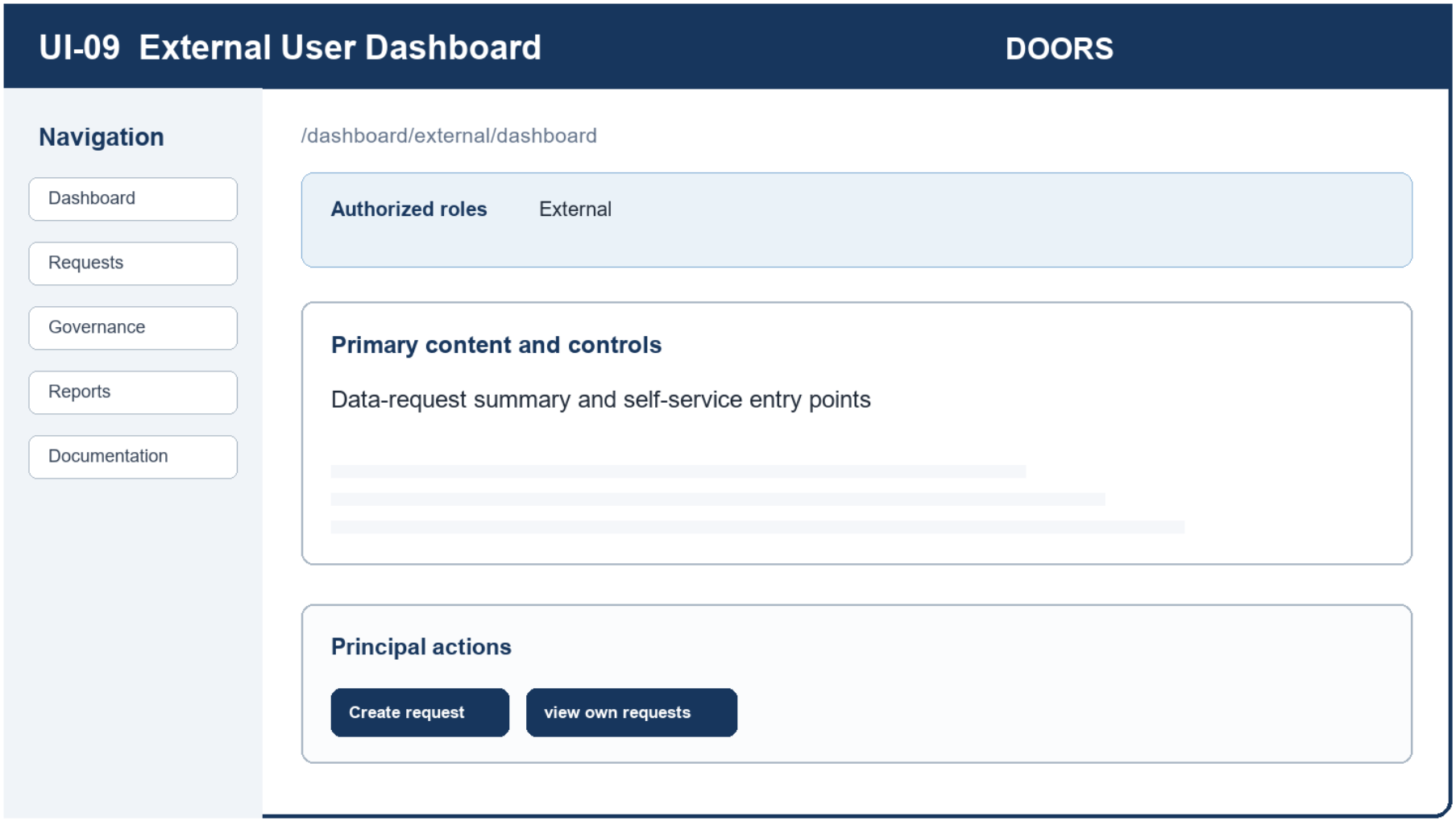


Figure G-9 UI-09 External User Dashboard wireframe

G.11 UI-10 External Data Request

Route: /dashboard/external-request **Authorized roles:** External; ApiUser

Design intent: Request purpose, category, data need and supporting details. Principal actions: Save or submit request.

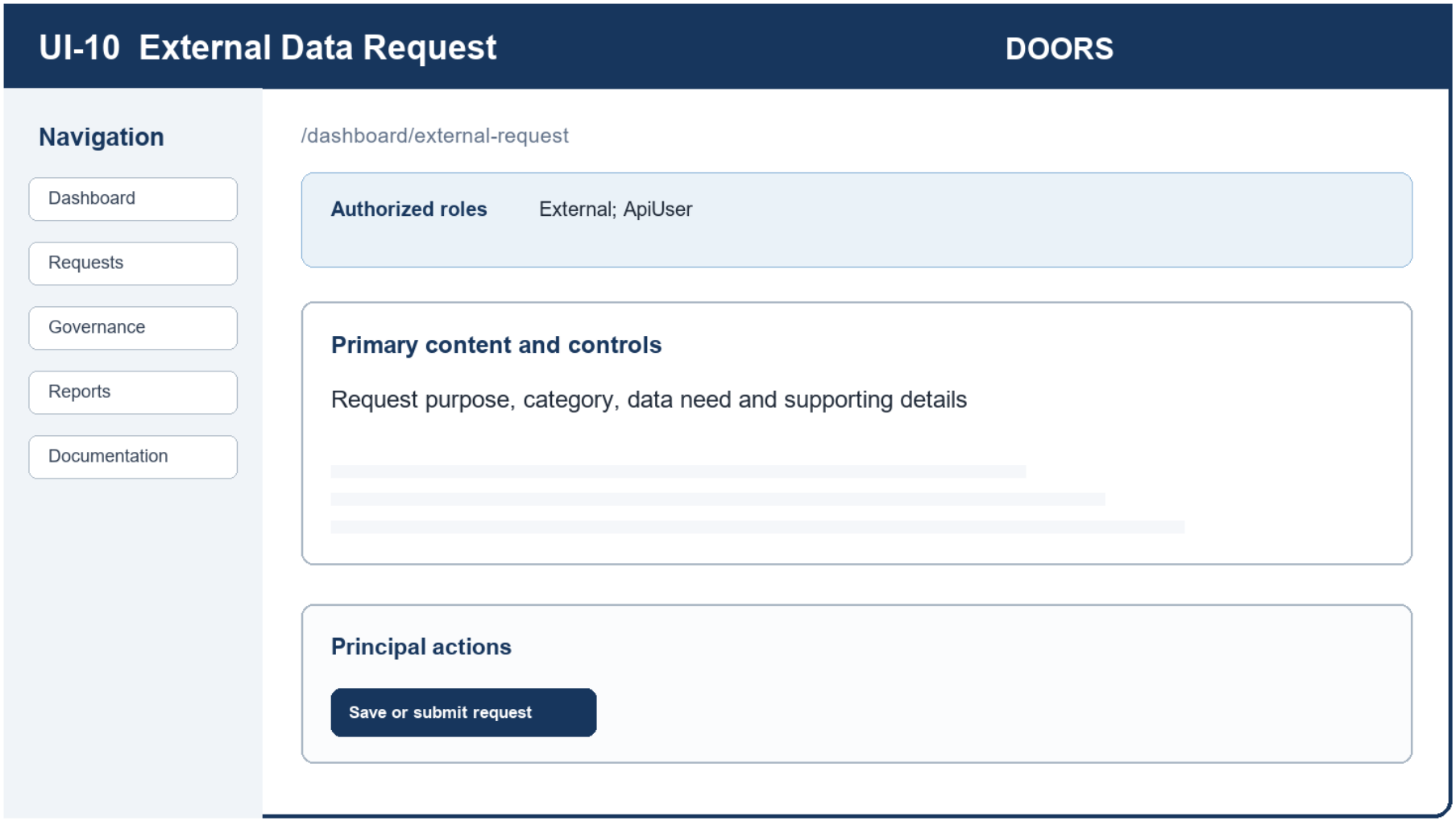


Figure G-10 UI-10 External Data Request wireframe

G.12 UI-11 My Data Requests

Route: /dashboard/my-requests **Authorized roles:** External; ApiUser

Design intent: Submitted requests, decisions and fulfilment status. Principal actions: View detail; correct and resubmit where allowed.

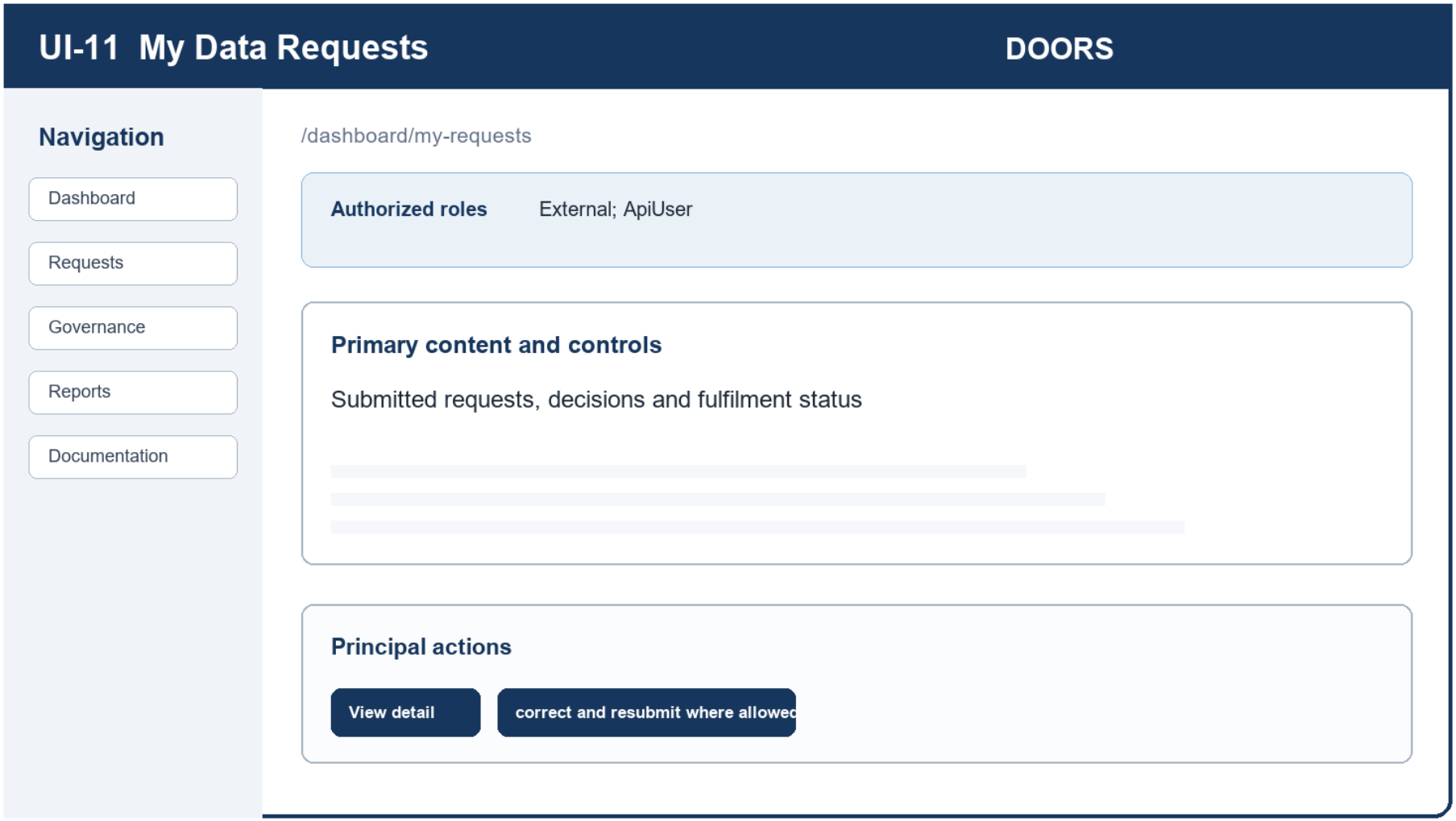


Figure G-11 UI-11 My Data Requests wireframe

G.13 UI-12 Request Submission

Route: /dashboard/query-submission **Authorized roles:** DataManager; Developer; ADMIN

Design intent: Approved assignment, SELECT definition or write JSON proposal. Principal actions: Validate; test; submit for approval.

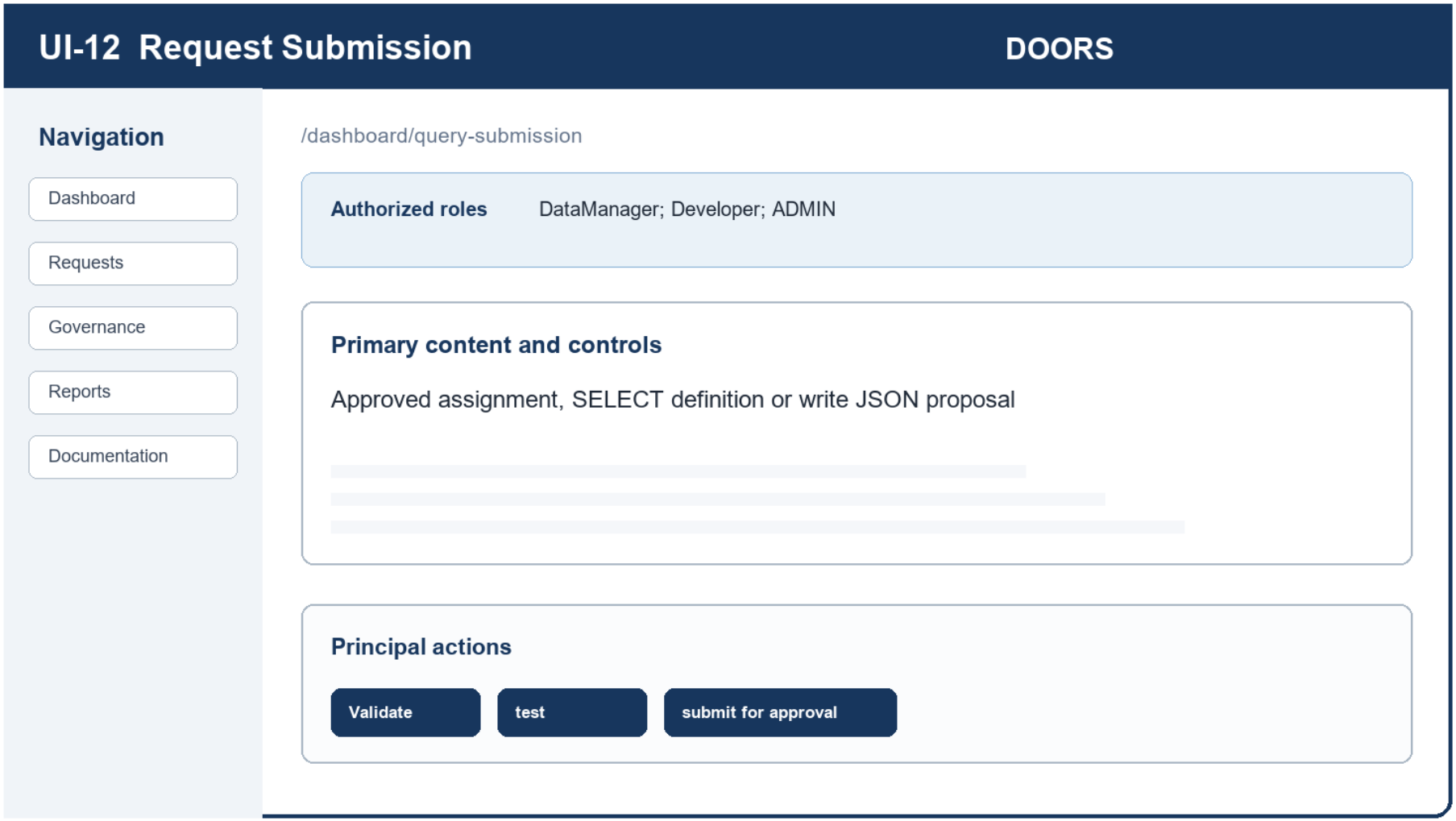


Figure G-12 UI-12 Request Submission wireframe

G.14 UI-13 Submission History

Route: /dashboard/my-submissions **Authorized roles:** Authenticated author

Design intent: Own submissions, state, remarks and revision history. Principal actions: View; revise; resubmit.

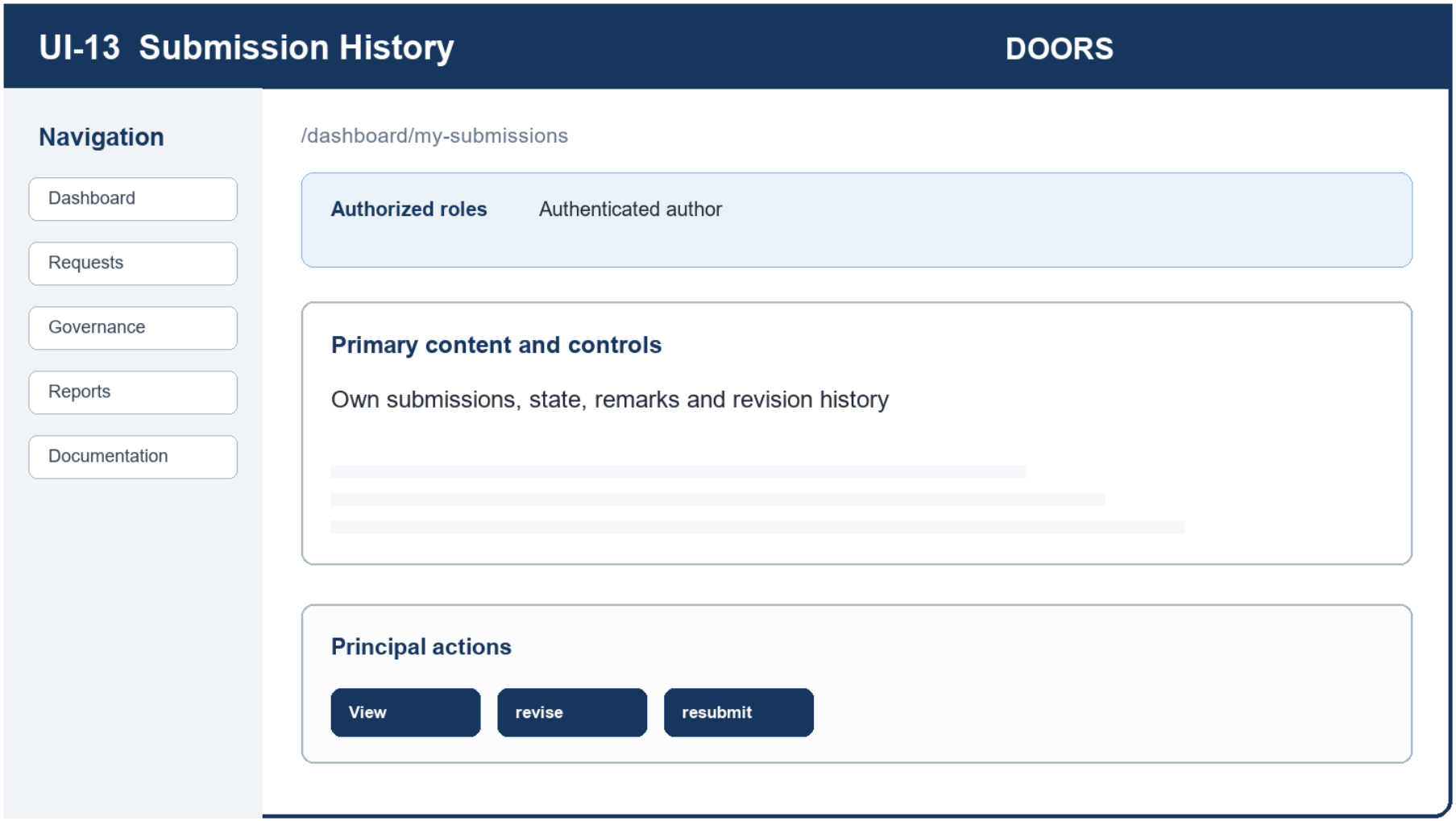


Figure G-13 UI-13 Submission History wireframe

G.15 UI-14 Pending Approvals

Route: /dashboard/governance/pending **Authorized roles:** DataManager

Design intent: Pending read-query or request review queue. Principal actions: Inspect; approve; reject; return for correction.

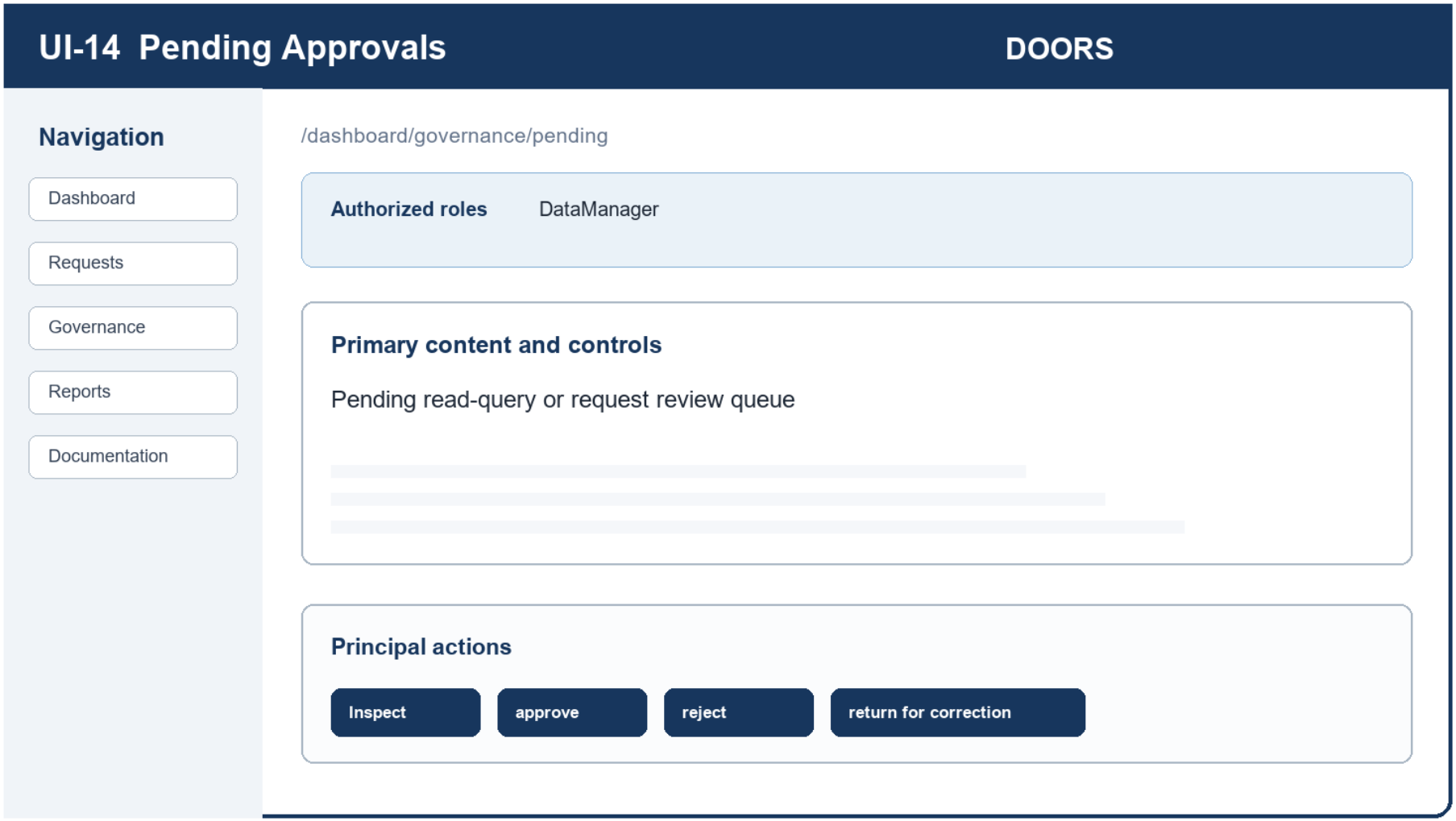


Figure G-14 UI-14 Pending Approvals wireframe

G.16 UI-15 Query Library

Route: /dashboard/governance/approved **Authorized roles:** DataManager

Design intent: Approved and published read services. Principal actions: Inspect; map; disable or restore under policy.

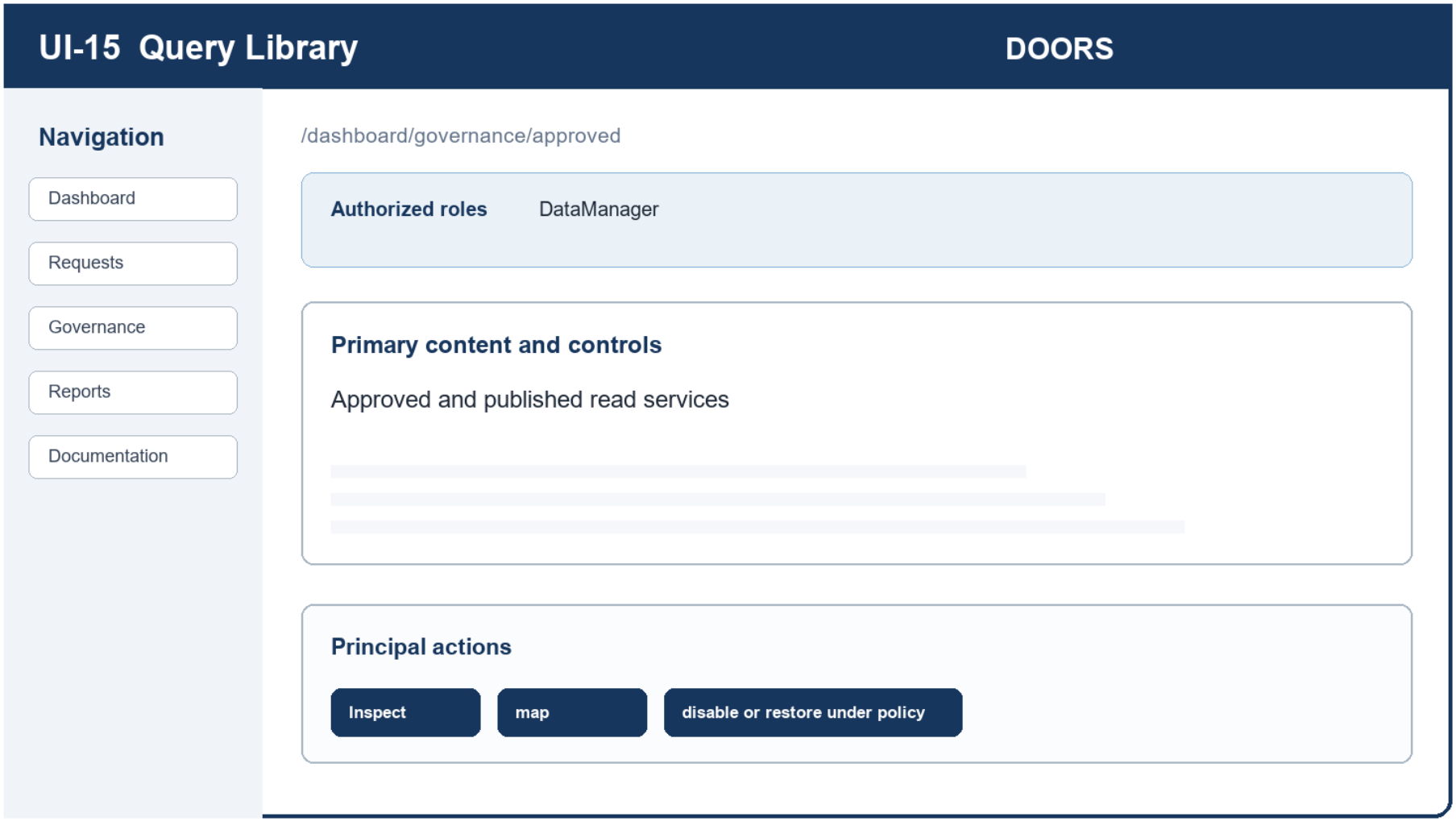


Figure G-15 UI-15 Query Library wireframe

G.17 UI-16 Infrastructure and Agents

Route: /dashboard/governance/infrastructure **Authorized roles:** DataManager

Design intent: Agent registry, health, connectivity and ownership. Principal actions: Register; test; update mappings.

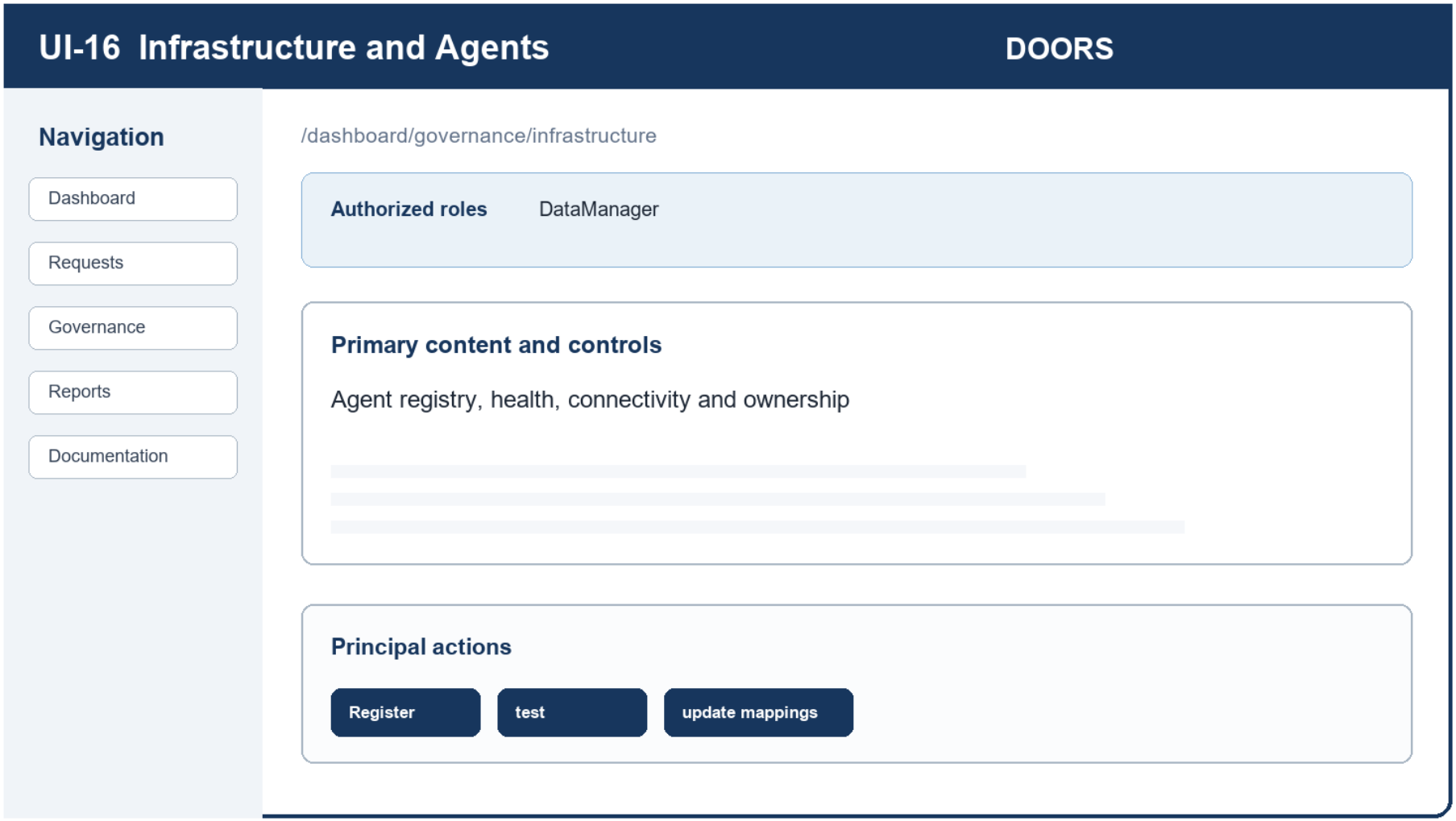


Figure G-16 UI-16 Infrastructure and Agents wireframe

G.18 UI-17 API Client Registry

Route: /dashboard/governance/api-clients **Authorized roles:** DataManager

Design intent: Read and write clients, credentials, IPs, mappings and security. Principal actions: Create; activate; rotate; map; disable.

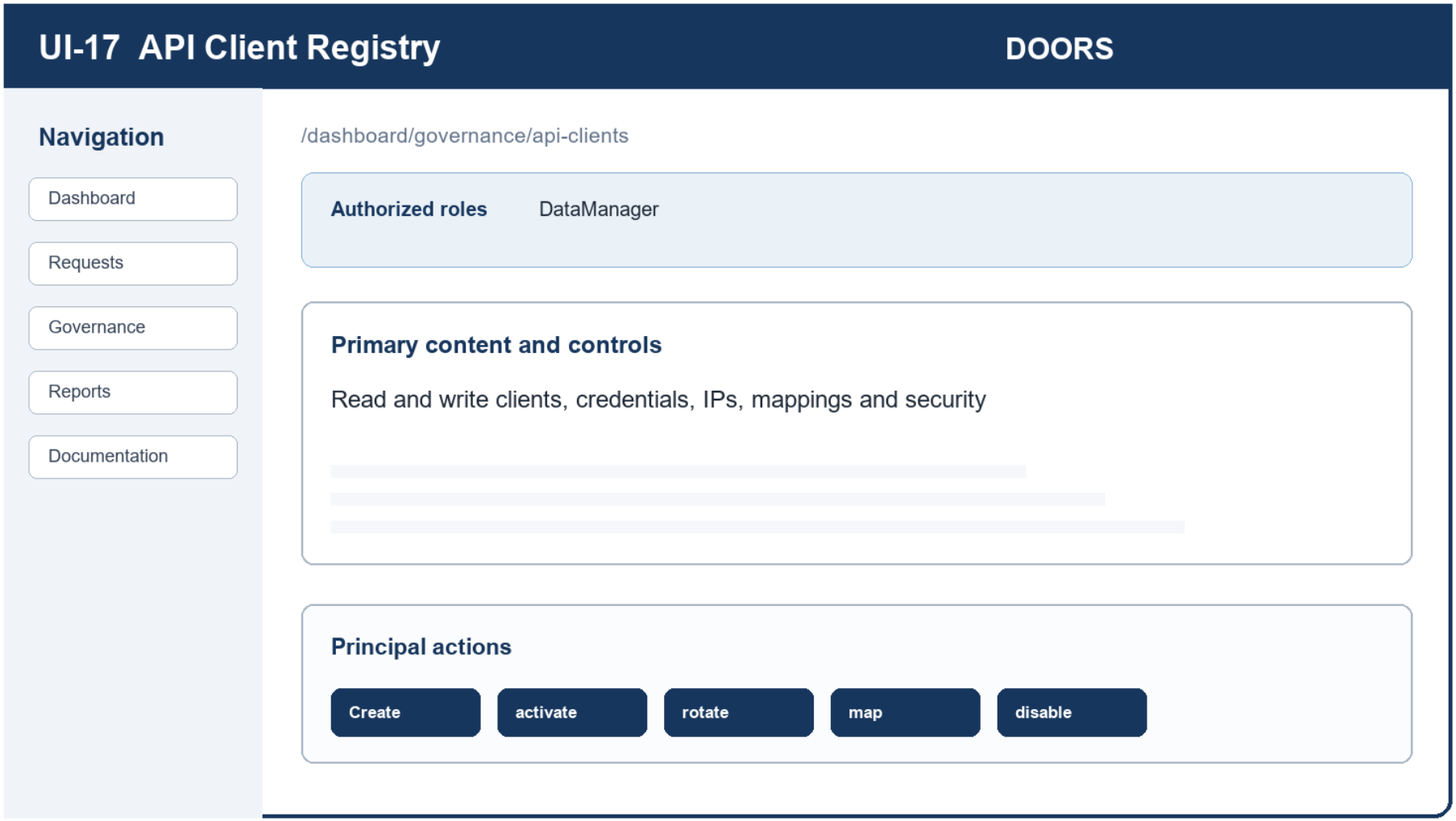


Figure G-17 UI-17 API Client Registry wireframe

G.19 UI-18 Template Contract Governance

Route: /dashboard/governance/template-contracts **Authorized roles:** DataManager

Design intent: Schema discovery, drafts, revisions and decisions. Principal actions: Submit; approve; reject; restore.

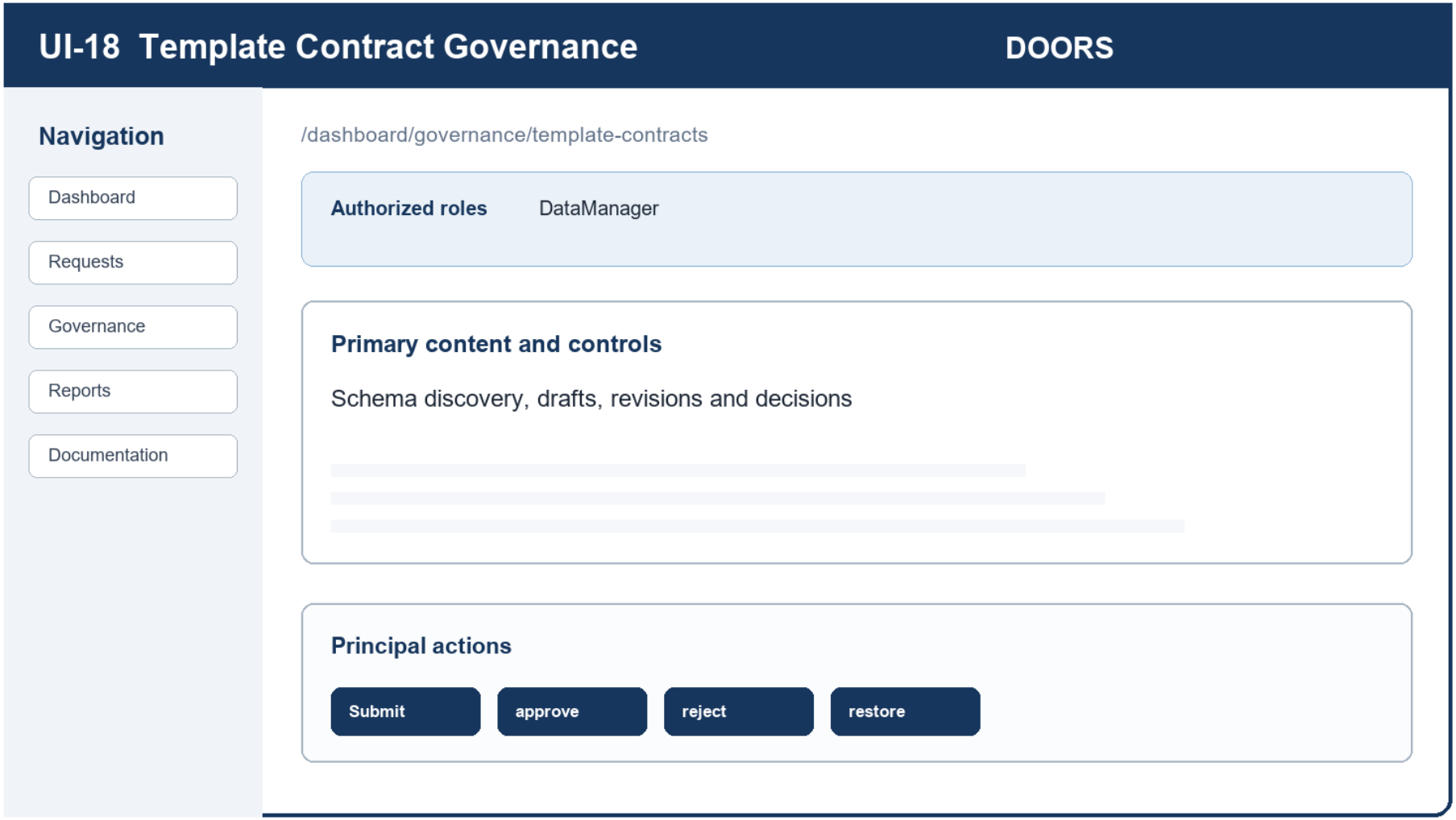


Figure G-18 UI-18 Template Contract Governance wireframe

G.20 UI-19 API Subscriptions

Route: /dashboard/governance/api-subscriptions **Authorized roles:** DataManager

Design intent: Licence, periods, schedules, usage and notices. Principal actions: Configure; activate; suspend; renew.

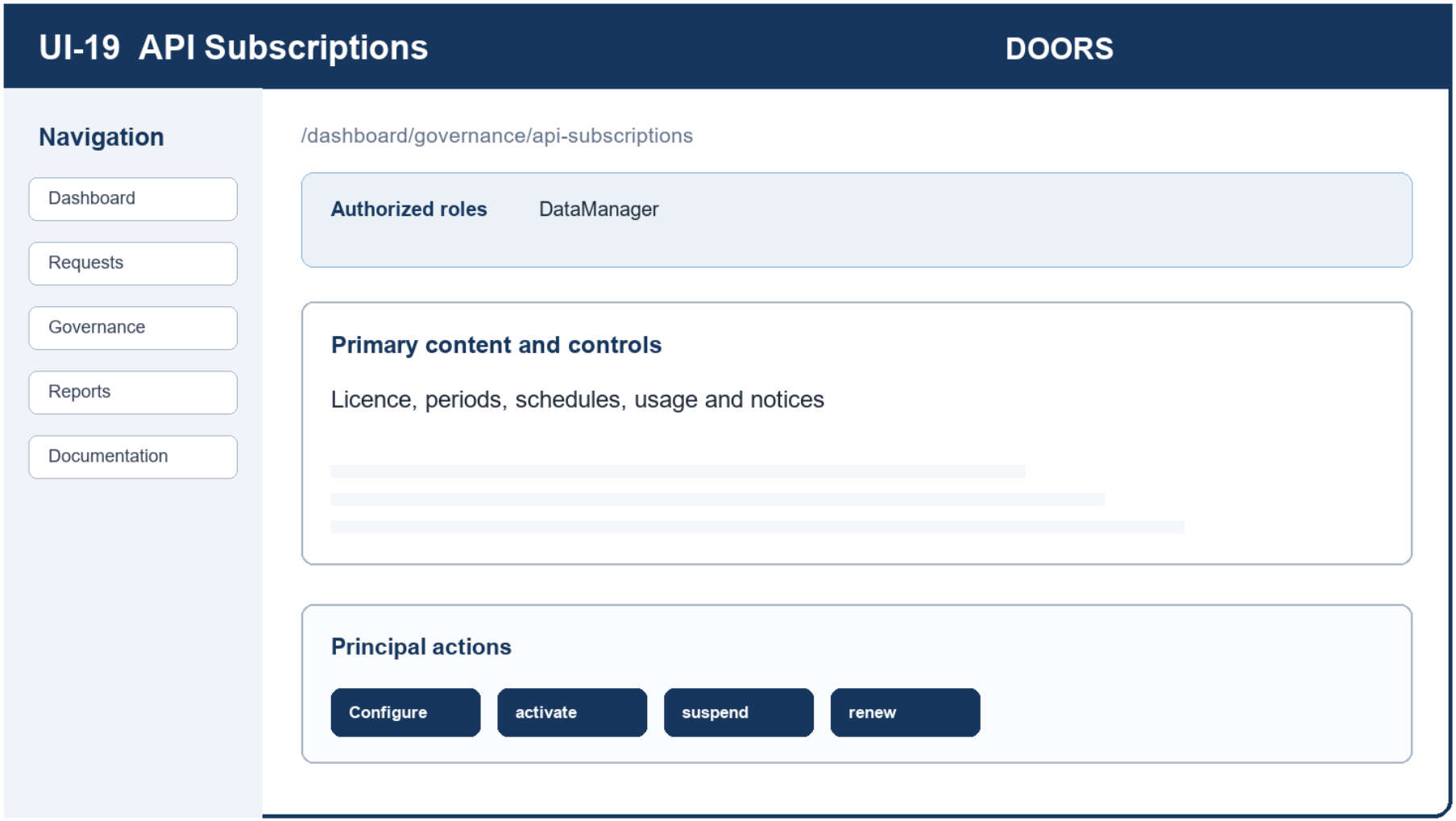


Figure G-19 UI-19 API Subscriptions wireframe

G.21 UI-20 Document Download Policies

Route: /dashboard/governance/document-policies; /dashboard/documents/policies **Authorized roles:** DataManager

Design intent: Eligibility, limits, expiry and download controls. Principal actions: Create; update; activate; revoke.

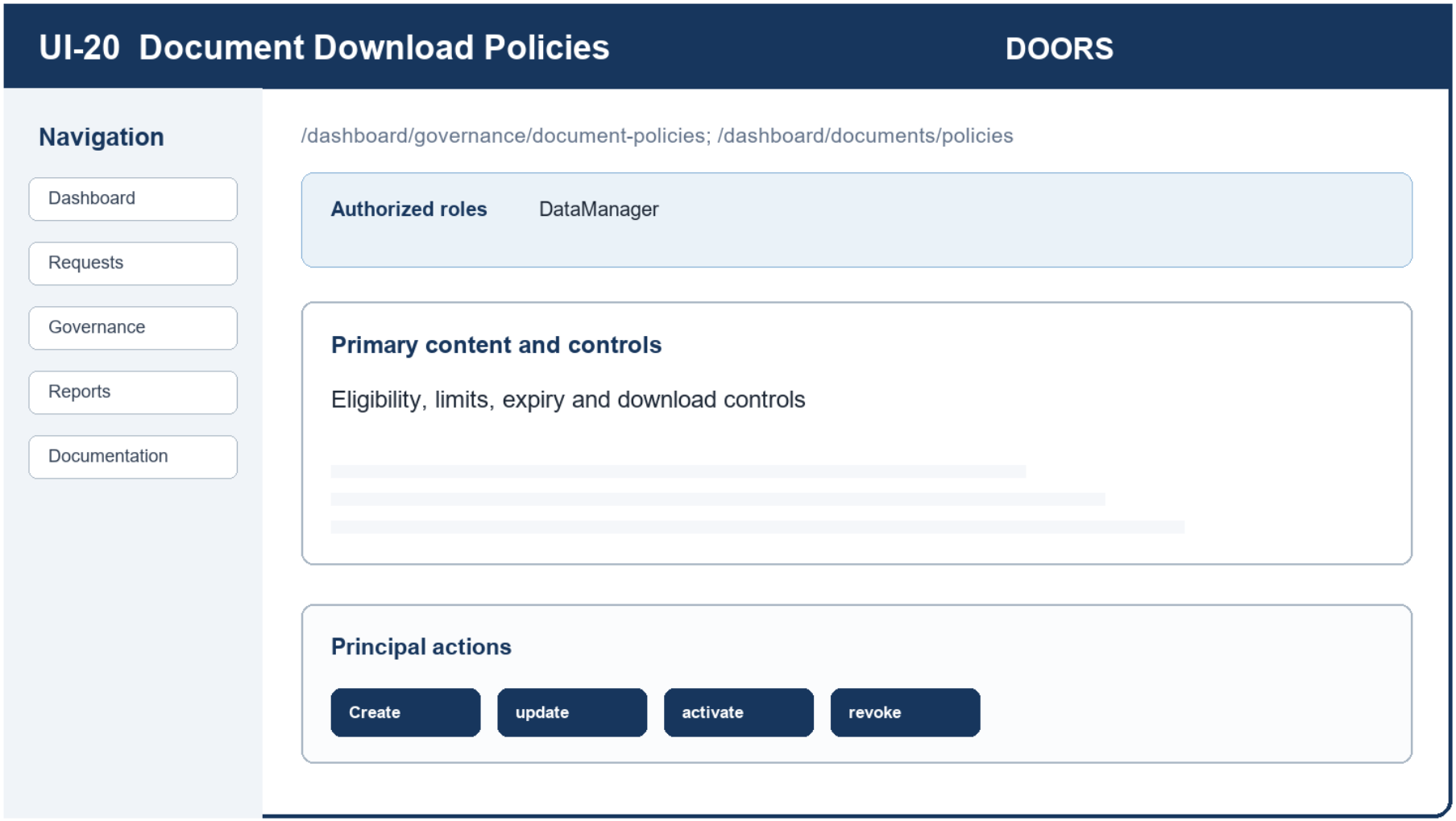


Figure G-20 UI-20 Document Download Policies wireframe

G.22 UI-21 Write Library

Route: /dashboard/governance/write-library (alias inbound-services) **Authorized roles:** DataManager

Design intent: Approved write contracts, revisions, readiness and integration assets. Principal actions: Inspect; provision; verify; revise; disable.

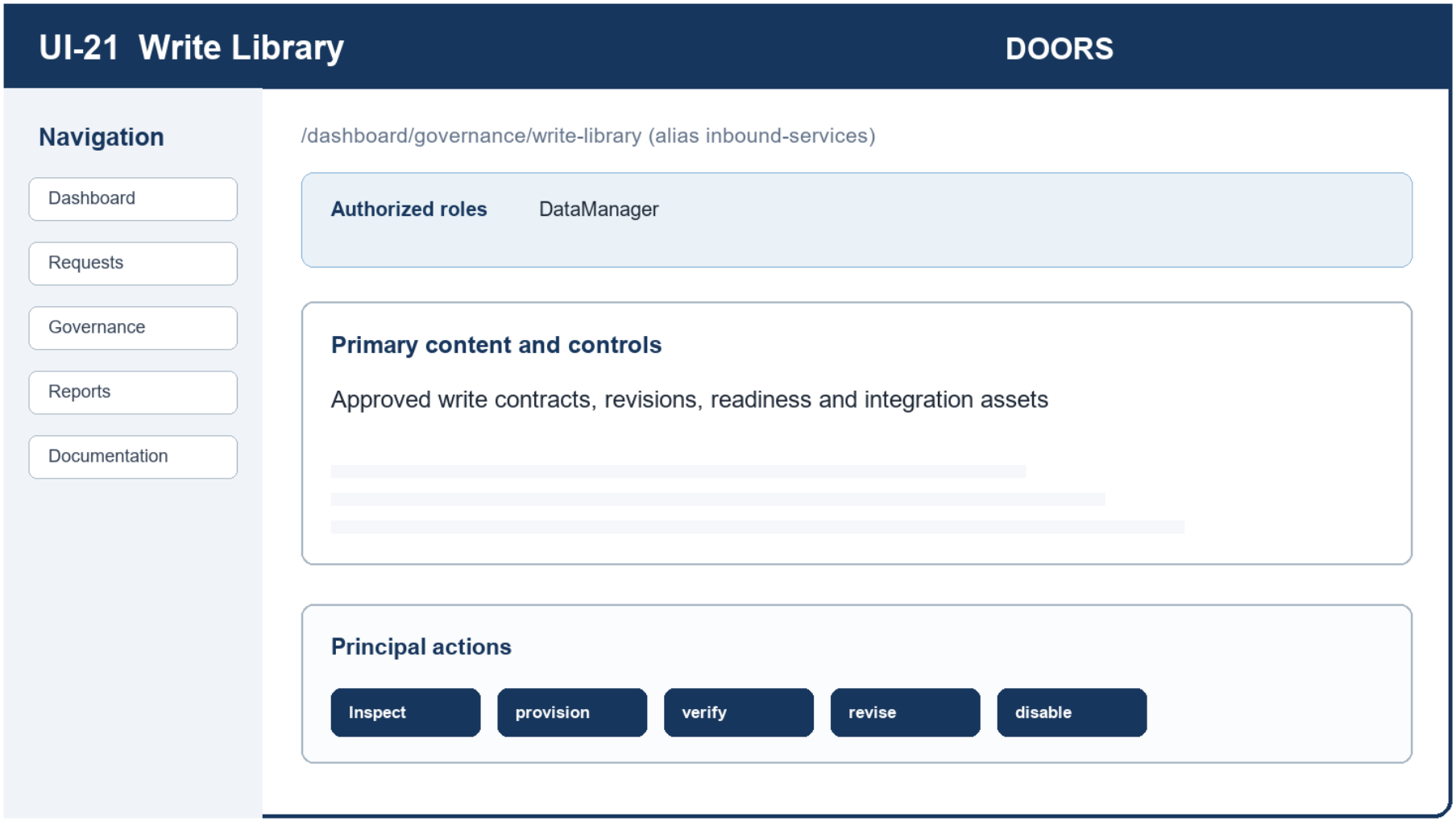


Figure G-21 UI-21 Write Library wireframe

G.23 UI-22 Document Service Registry

Route: /dashboard/governance/document-services; /dashboard/documents/services **Authorized roles:** DataManager

Design intent: Document definitions, Agent mappings and availability. Principal actions: Create; test; approve or activate; disable.

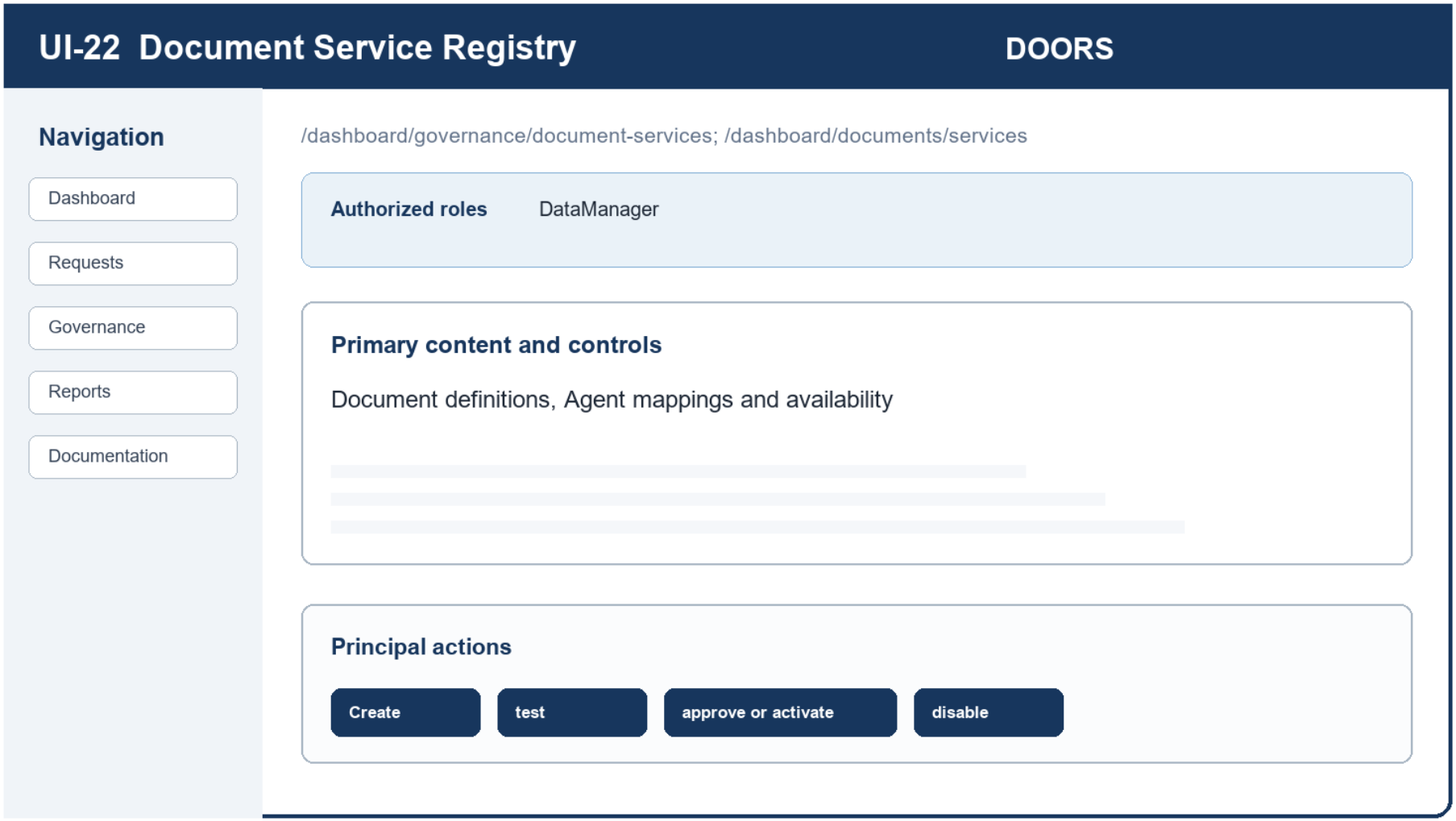


Figure G-22 UI-22 Document Service Registry wireframe

G.24 UI-23 Category Management

Route: /dashboard/governance/categories **Authorized roles:** DataManager

Design intent: Request and service categories. Principal actions: Create; edit; activate or deactivate.

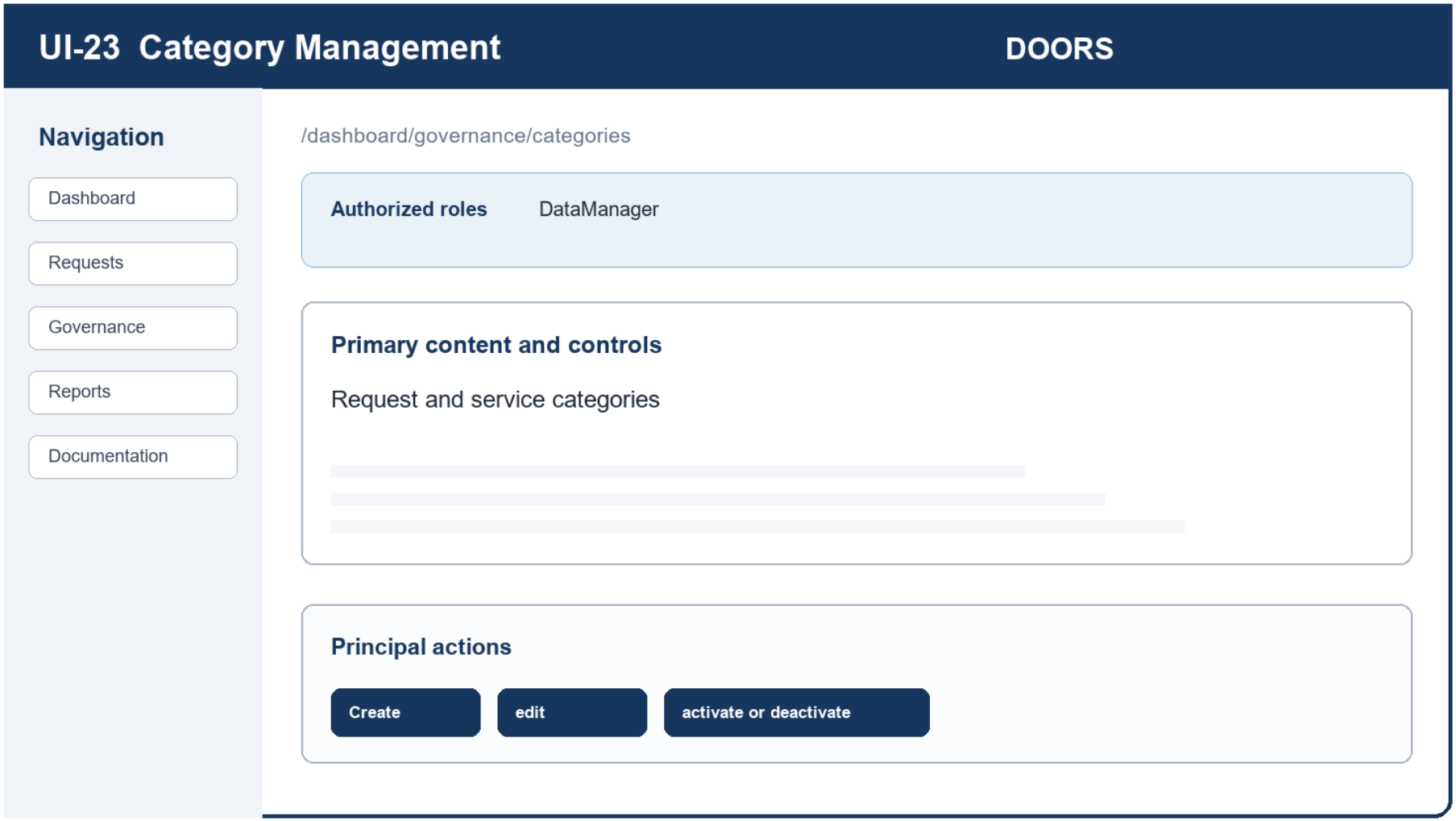


Figure G-23 UI-23 Category Management wireframe

G.25 UI-24 Data Request Management

Route: /dashboard/governance/data-requests **Authorized roles:** DataManager

Design intent: External request review, assignment and decision. Principal actions: Approve; reject; return; assign.

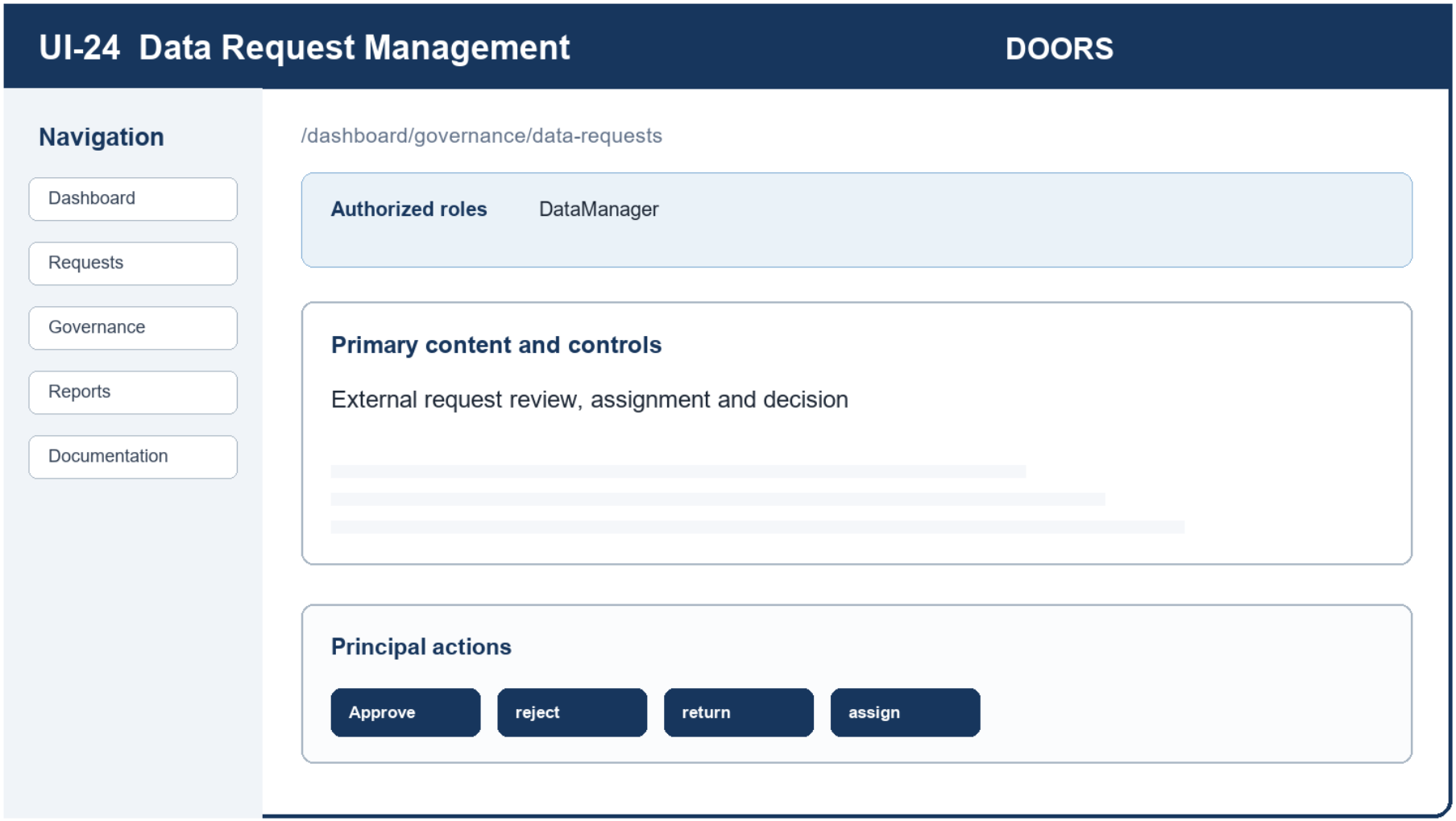


Figure G-24 UI-24 Data Request Management wireframe

G.26 UI-25 Signing Key Management

Route: /dashboard/governance/signing-keys **Authorized roles:** DataManager

Design intent: Signing certificates, validity and rotation status. Principal actions: Register; activate; rotate; revoke.

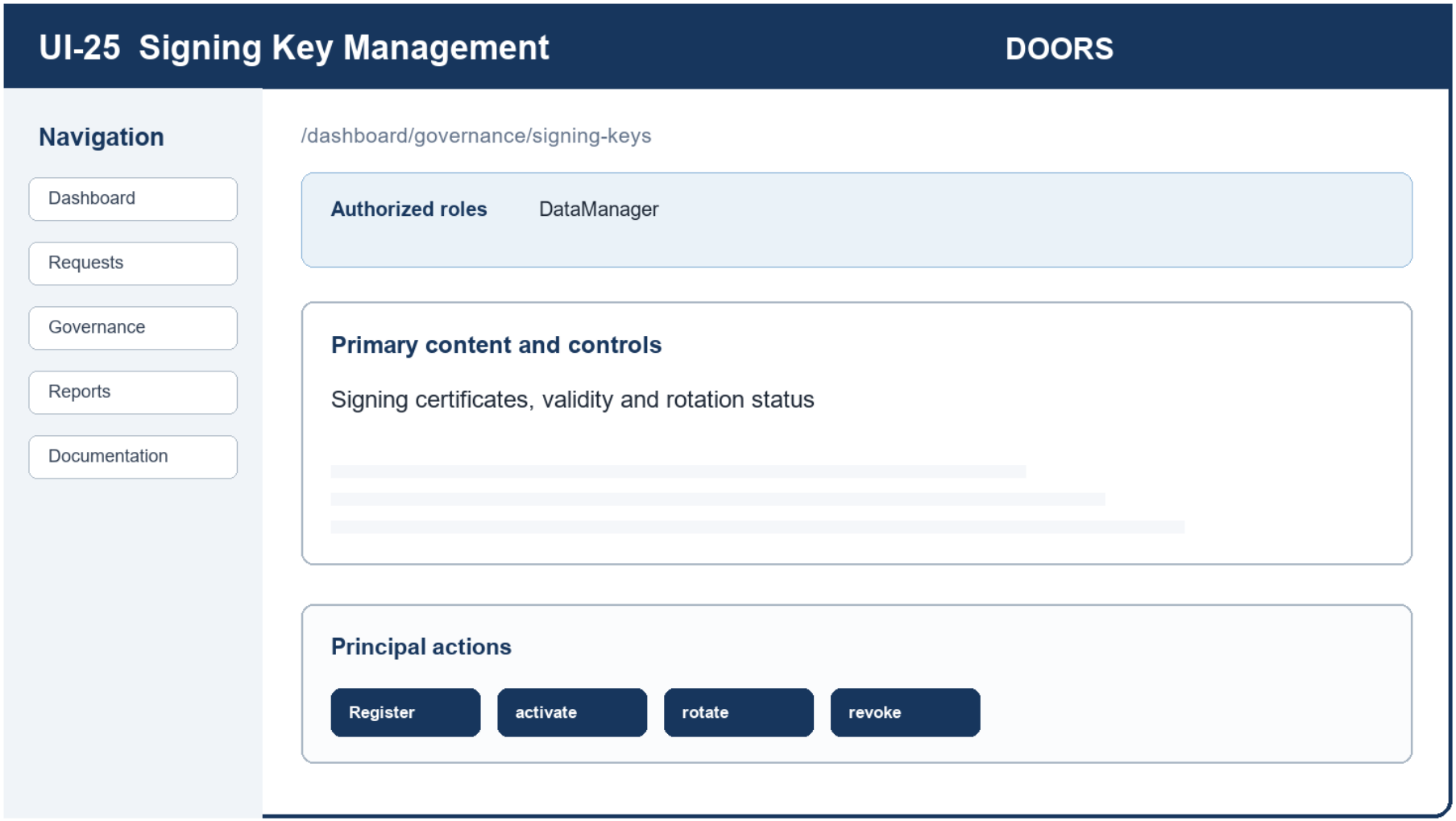


Figure G-25 UI-25 Signing Key Management wireframe

G.27 UI-26 User Management

Route: /dashboard/governance/users; /dashboard/security/users **Authorized roles:** DataManager; SecurityAdmin

Design intent: Pending, active and rejected users with role and access controls. Principal actions: Approve; reject; reapprove; change role; reset; disable.

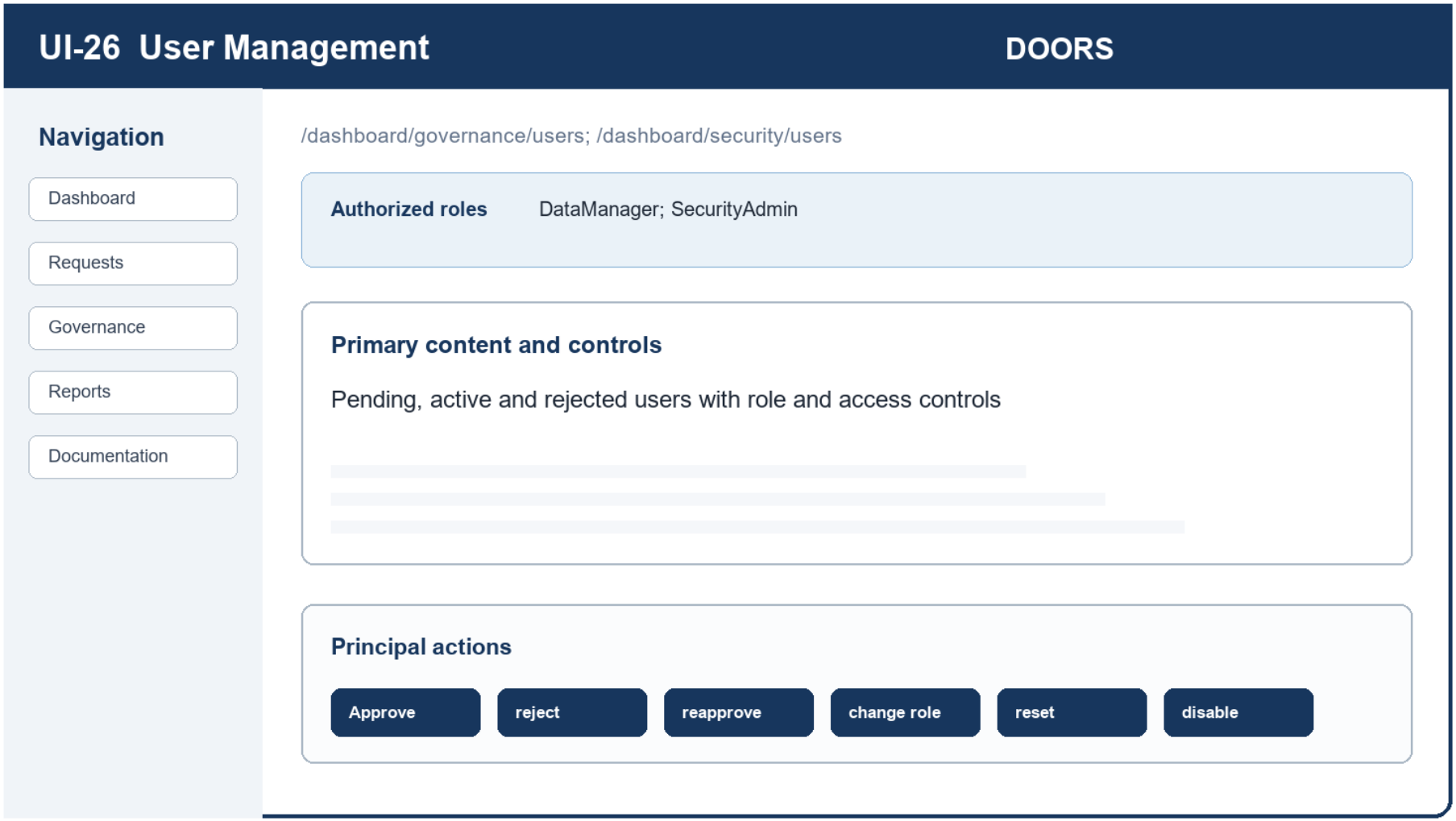


Figure G-26 UI-26 User Management wireframe

G.28 UI-27 User Agent Matrix

Route: /admin/users/matrix **Authorized roles:** DataManager

Design intent: User-to-Agent authorization matrix. Principal actions: Grant; revoke; save mappings.

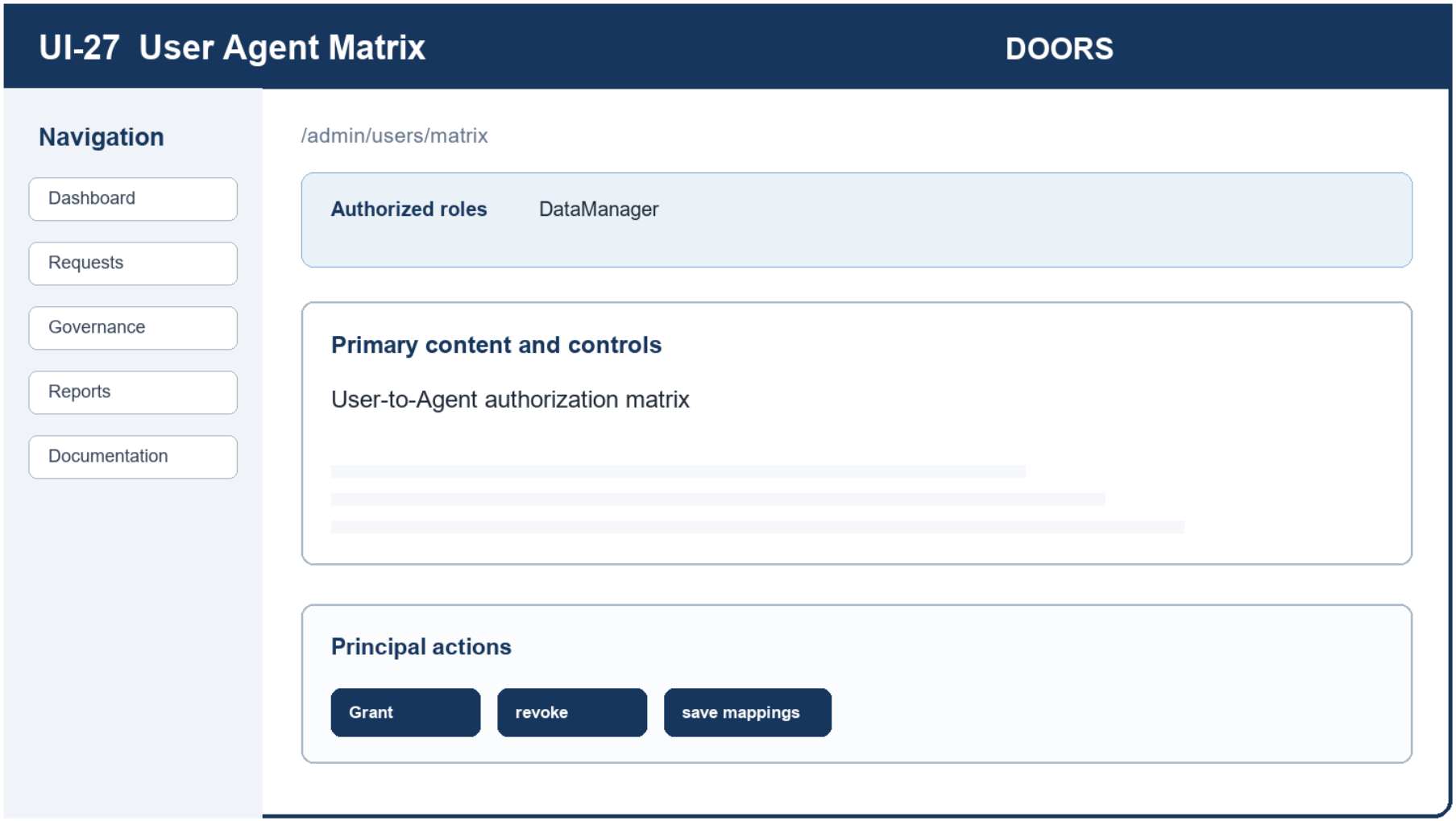


Figure G-27 UI-27 User Agent Matrix wireframe

G.29 UI-28 System Documentation

Route: /dashboard/documentation **Authorized roles:** All authenticated roles

Design intent: Role-filtered manuals, guides and downloads. Principal actions: View or download approved documents.

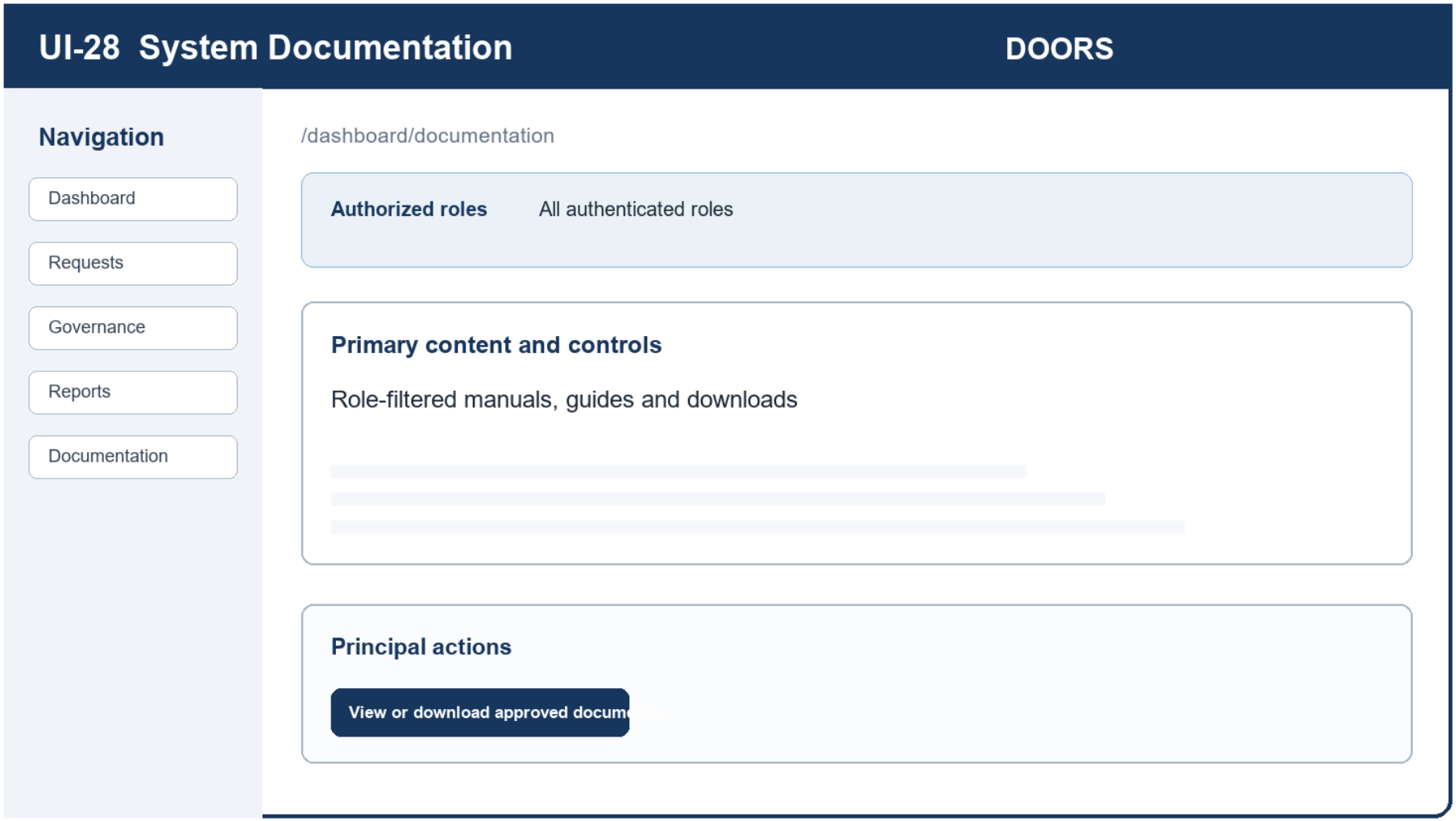


Figure G-28 UI-28 System Documentation wireframe

G.30 UI-29 Document Dashboard

Route: /dashboard/documents/dashboard **Authorized roles:** DataManager

Design intent: Document-service counts, policy status and delivery activity. Principal actions: Open services, policies or logs.

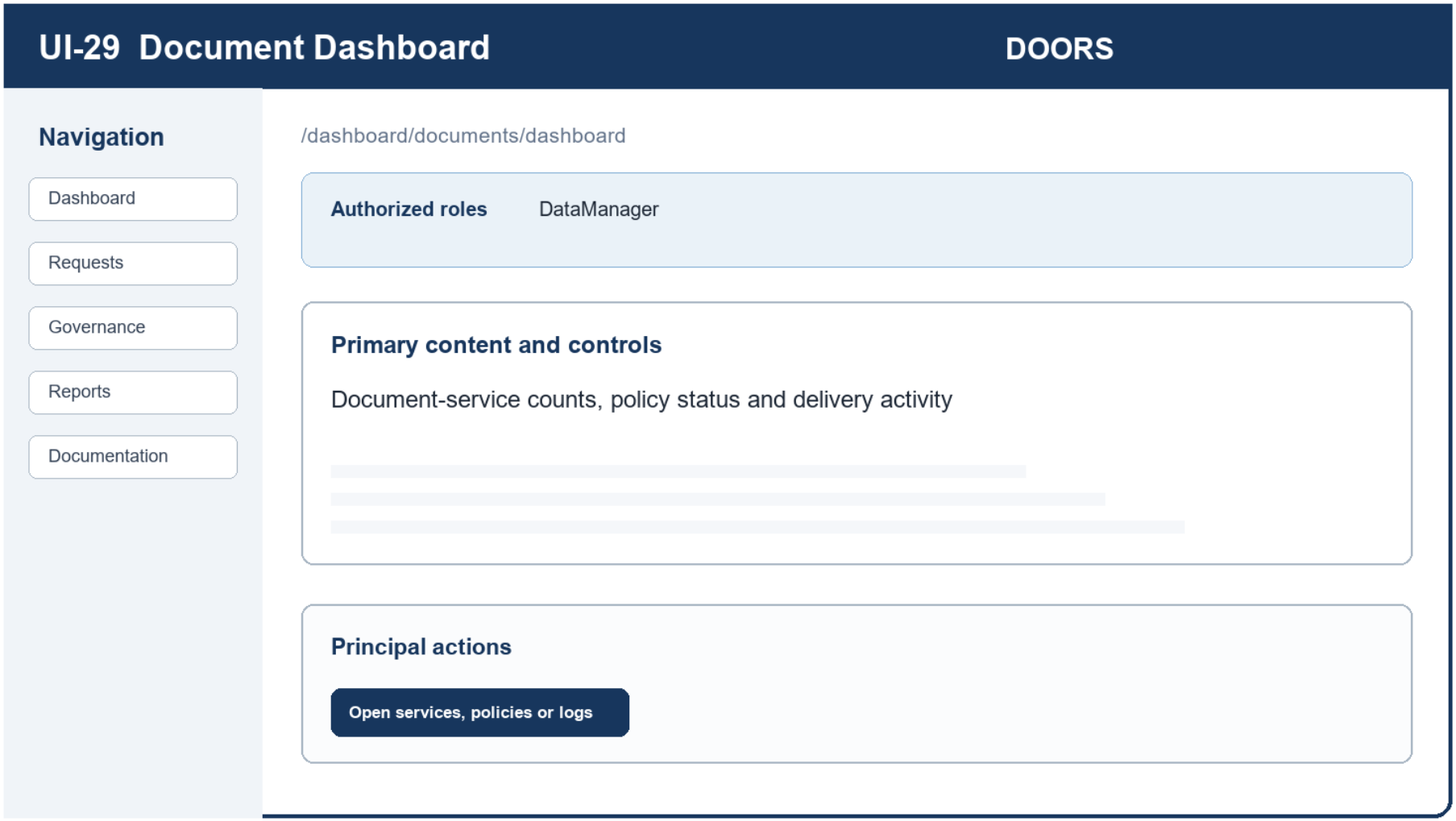


Figure G-29 UI-29 Document Dashboard wireframe

G.31 UI-30 Document Download Logs

Route: /dashboard/documents/logs **Authorized roles:** DataManager

Design intent: Delivery attempts, outcomes, actors and trace evidence. Principal actions: Filter; inspect; export bounded evidence.

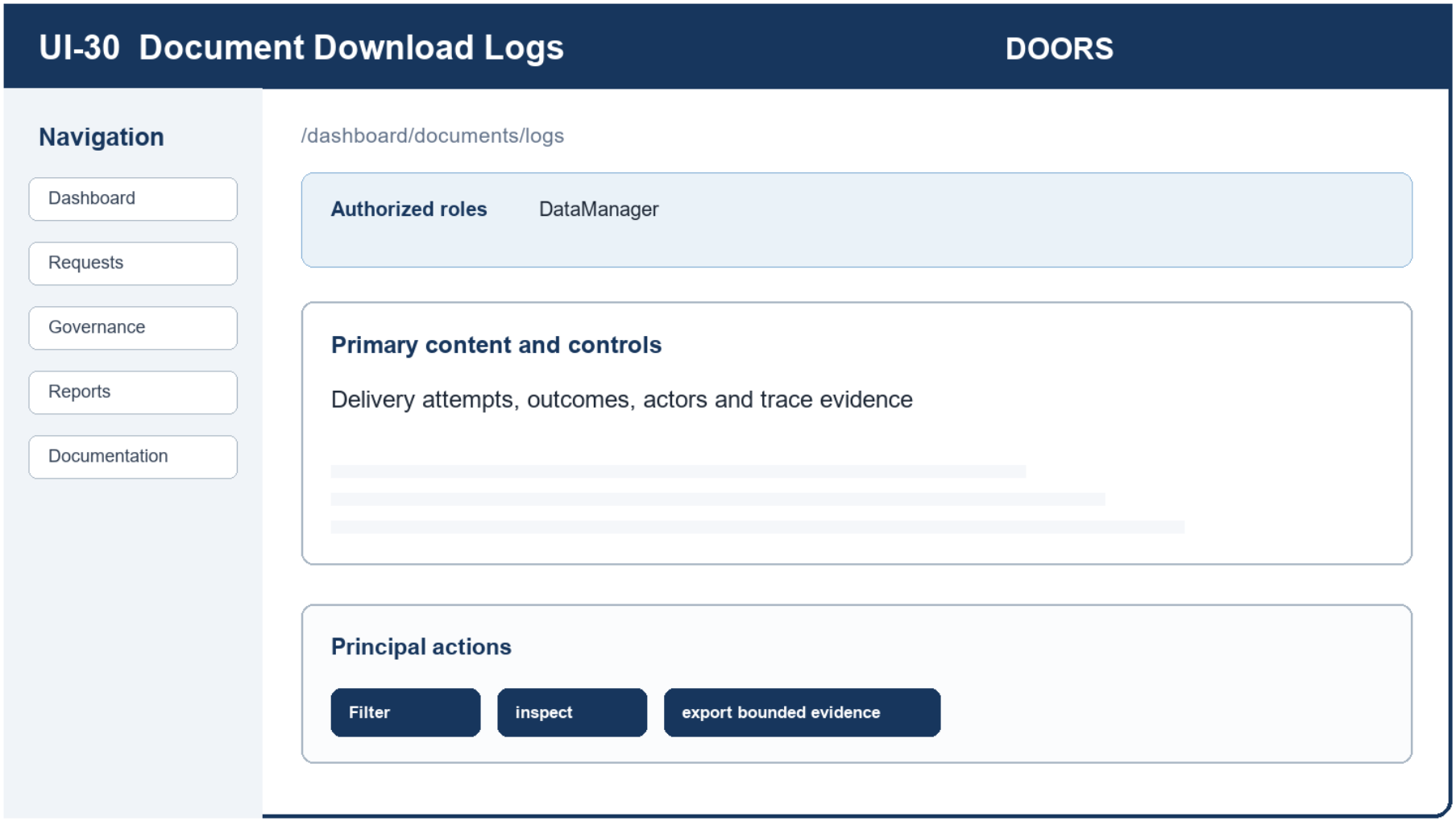


Figure G-30 UI-30 Document Download Logs wireframe

G.32 UI-31 Audit Logs

Route: /dashboard/audit **Authorized roles:** DataManager

Design intent: Governance and execution events with status, trace and latency. Principal actions: Search; filter; inspect diagnostics.

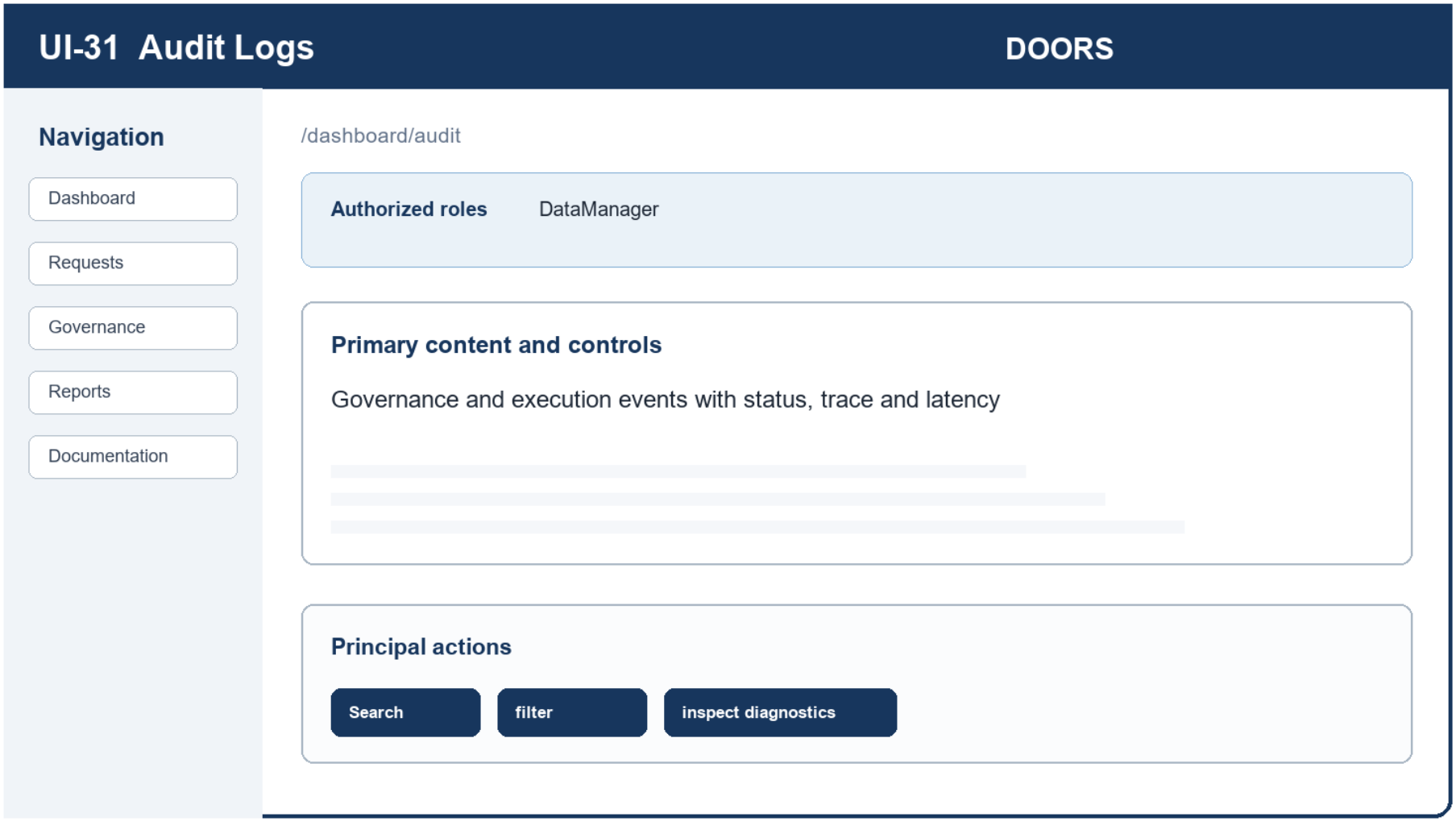


Figure G-31 UI-31 Audit Logs wireframe

G.33 UI-32 Report Library

Route: /dashboard/reporting/report-library **Authorized roles:** DataManager; DataViewer; External; ApiUser

Design intent: Authorized report templates and descriptions. Principal actions: Select and open report.

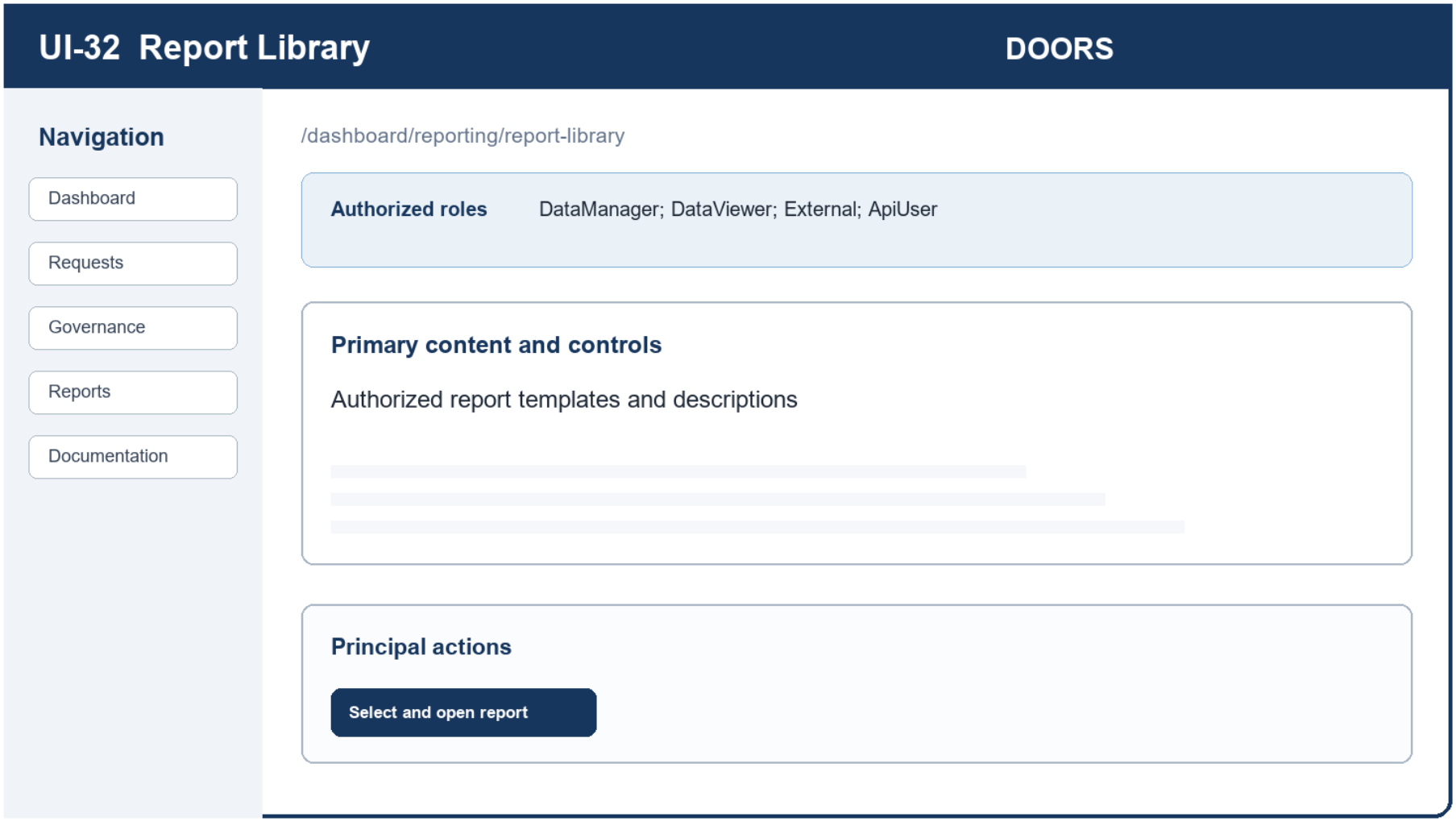


Figure G-32 UI-32 Report Library wireframe

G.34 UI-33 Report Viewer

Route: /dashboard/reporting/report-viewer **Authorized roles:** DataManager; DataViewer; External; ApiUser

Design intent: Parameters, execution status, results and export. Principal actions: Run; page; export; download.

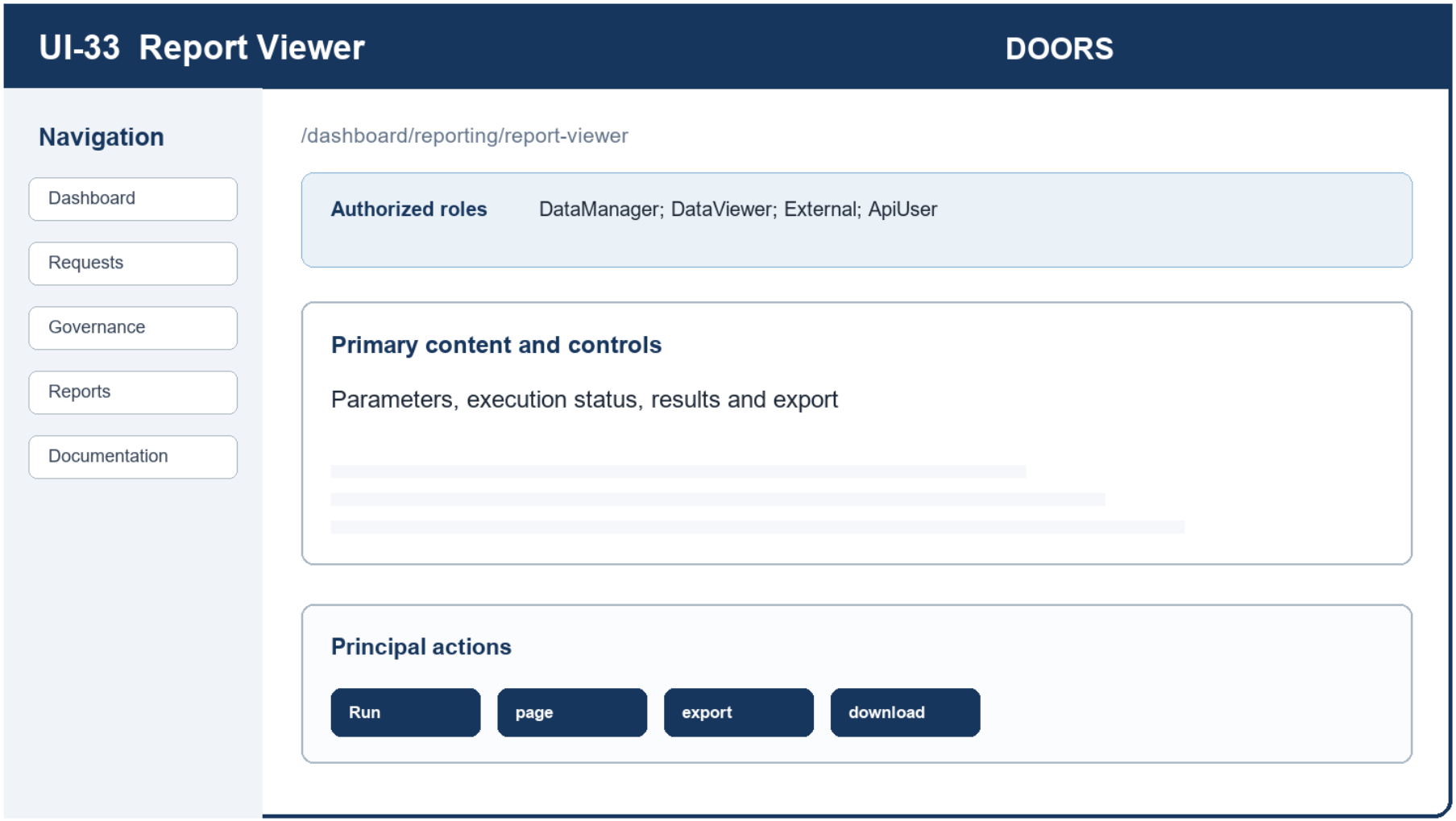


Figure G-33 UI-33 Report Viewer wireframe

Appendix H Controlled State Workflow Swimlanes

Every workflow involving approval or another controlled state transition shall have a swimlane diagram identifying the responsible actors and system components. Each diagram covers the normal path and, where applicable, approval, rejection, return for correction, resubmission, authorization checks, notification, audit evidence, exception handling and final state.

ID	Workflow	Controlled decisions and evidence
WF-01	Registration and user approval	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-02	External data-request approval and assignment	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-03	Read-query governance	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-04	Template-contract governance	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-05	Write-service governance and provisioning	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-06	Document-service and policy lifecycle	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-07	API-client activation and security change	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-08	Subscription lifecycle	Independent authorization, decision remarks, state transition and correlated audit evidence
WF-09	Signing-key lifecycle	Independent authorization, decision remarks, state transition and correlated audit evidence

H.1 WF-01 Registration and user approval

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

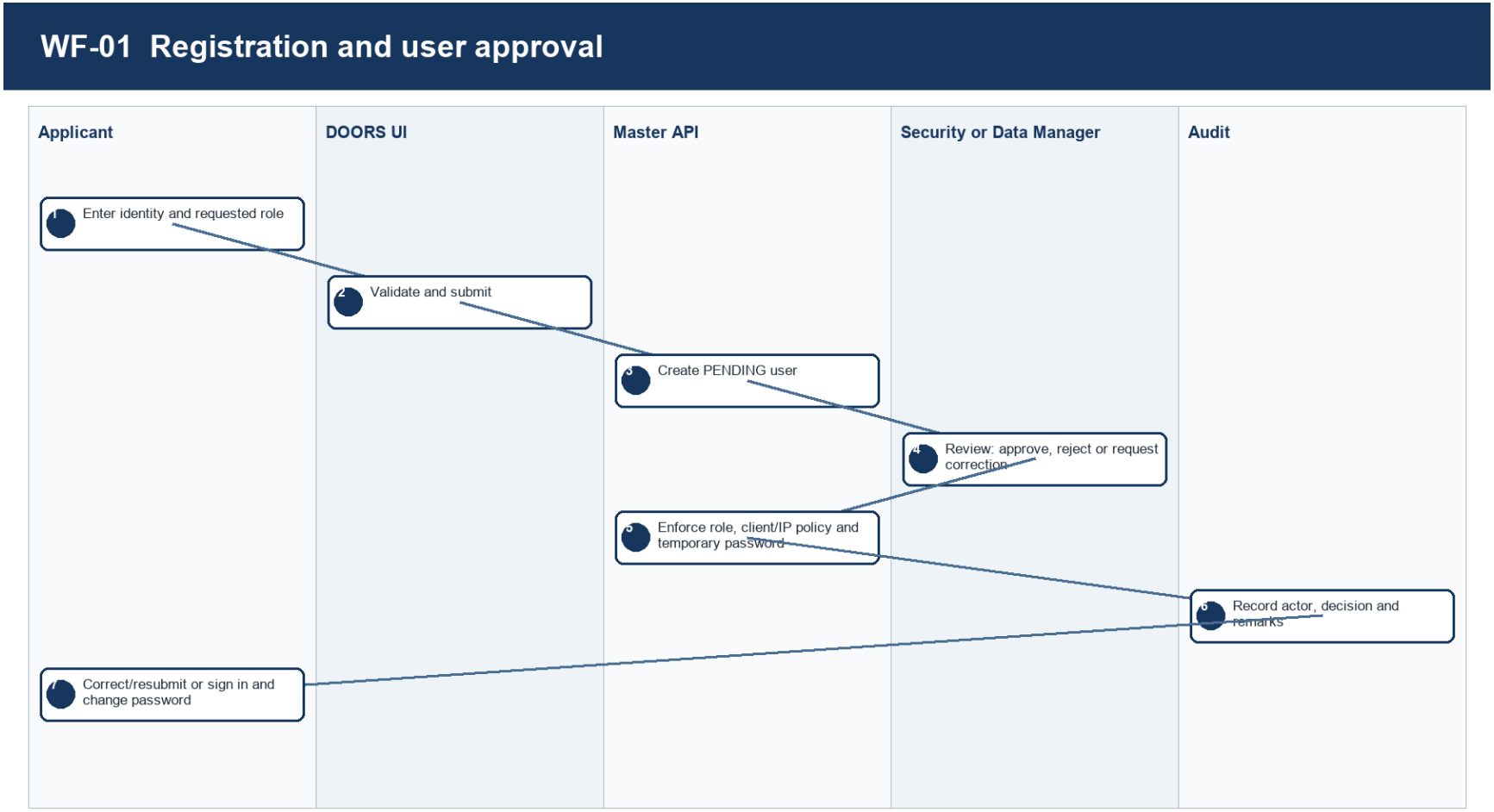


Figure H-1 WF-01 Registration and user approval

H.2 WF-02 External data-request approval and assignment

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

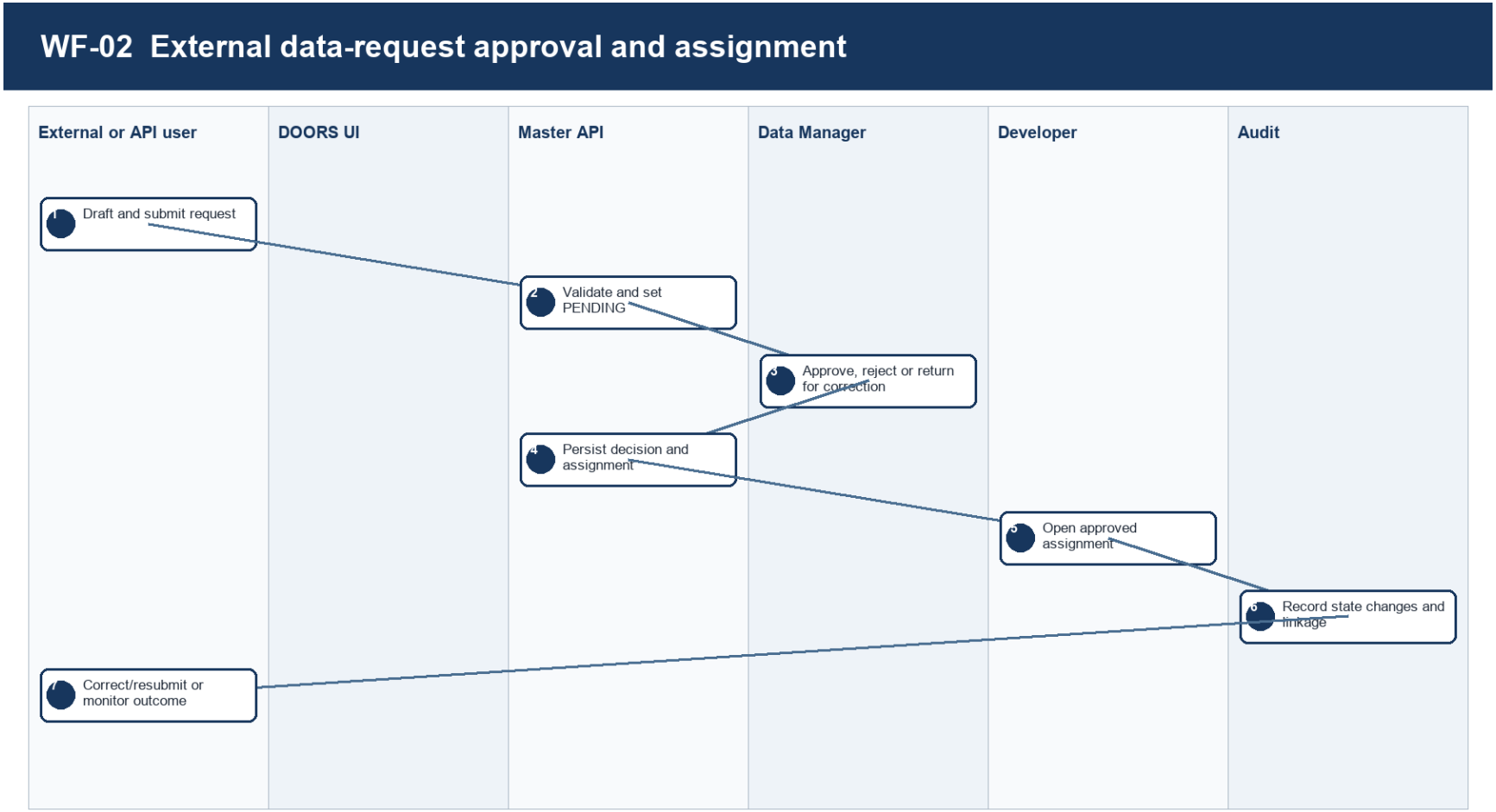


Figure H-2 WF-02 External data-request approval and assignment

H.3 WF-03 Read-query governance

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

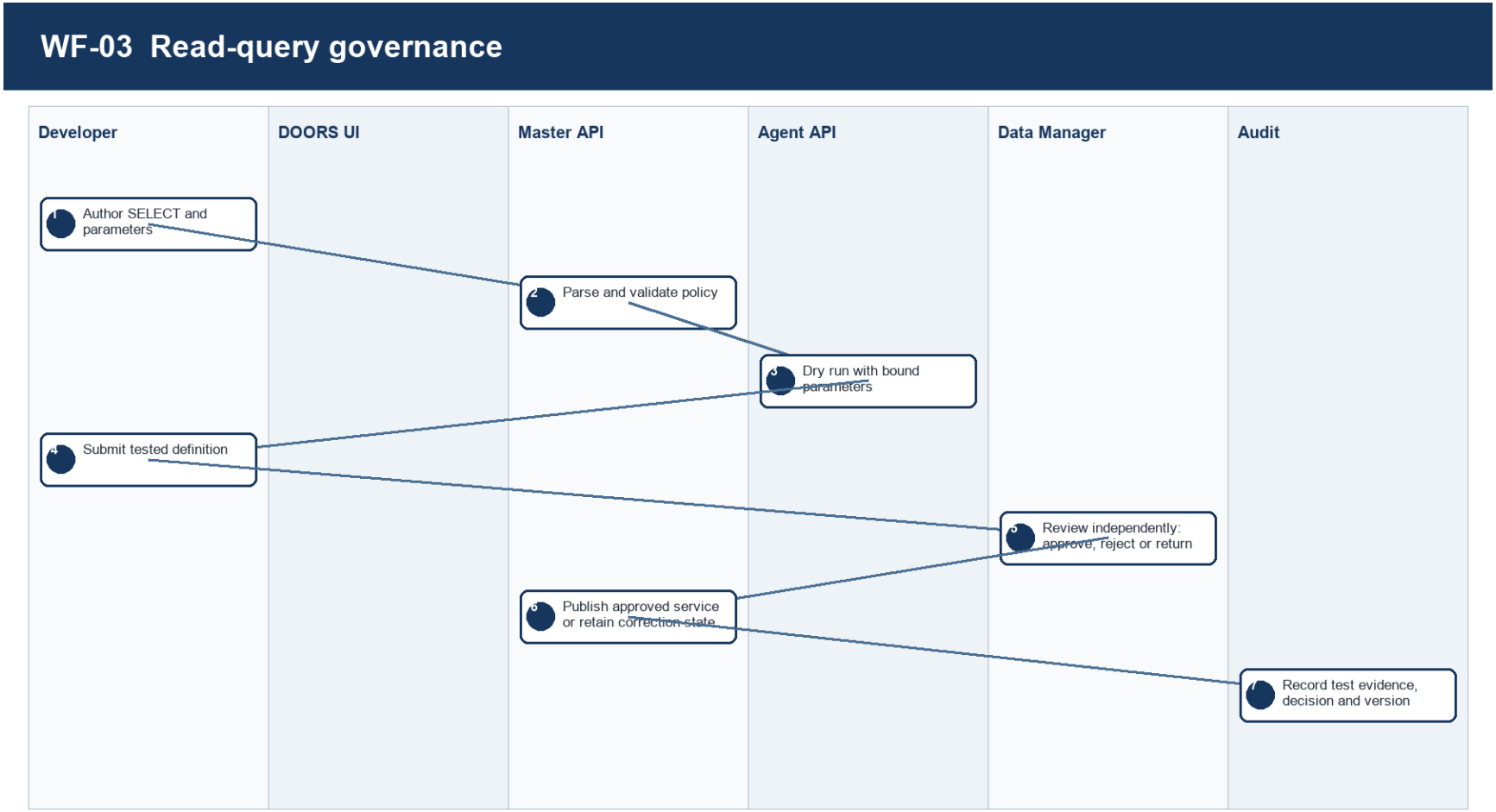


Figure H-3 WF-03 Read-query governance

H.4 WF-04 Template-contract governance

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

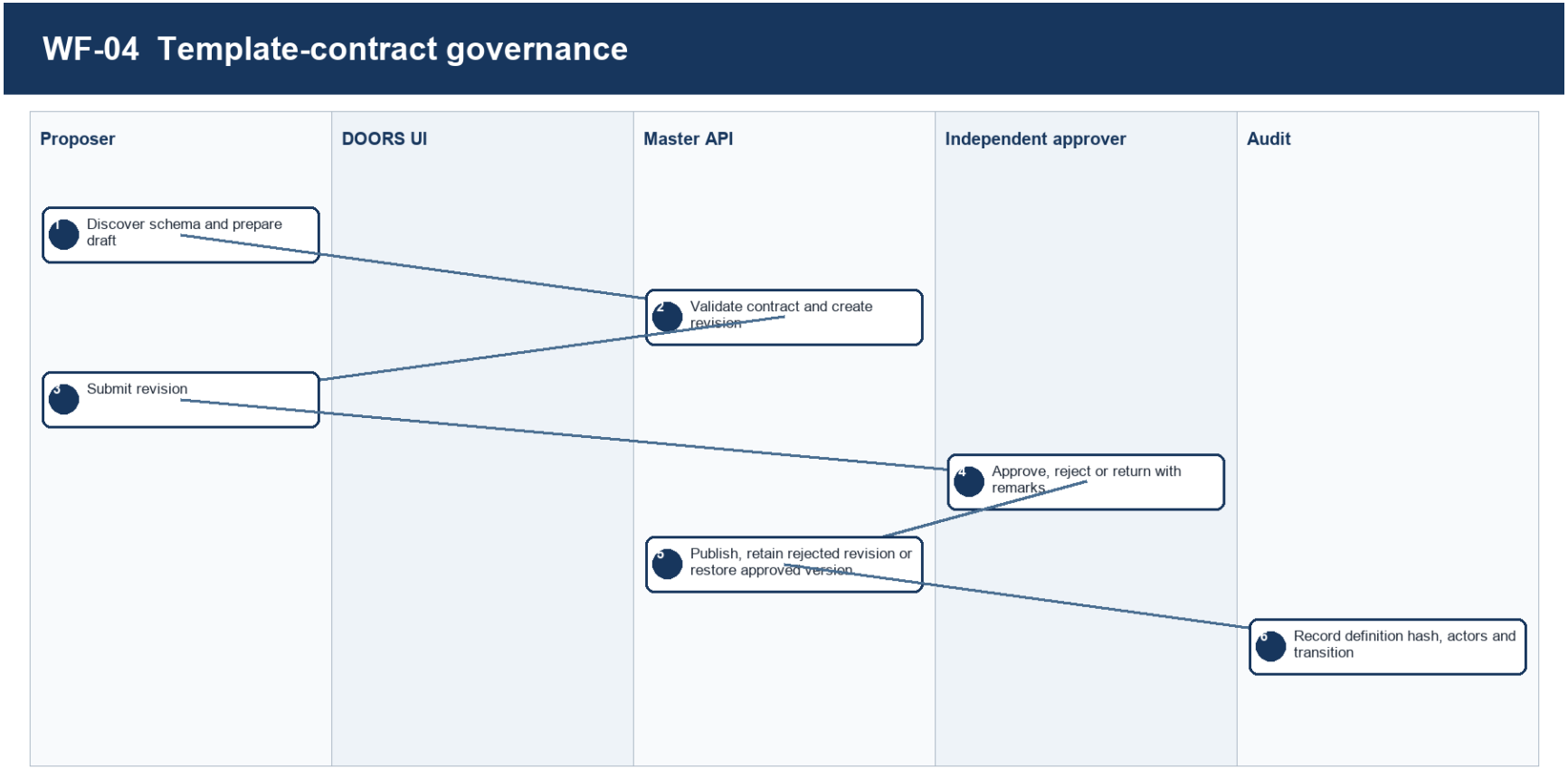


Figure H-4 WF-04 Template-contract governance

H.5 WF-05 Write-service governance and provisioning

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

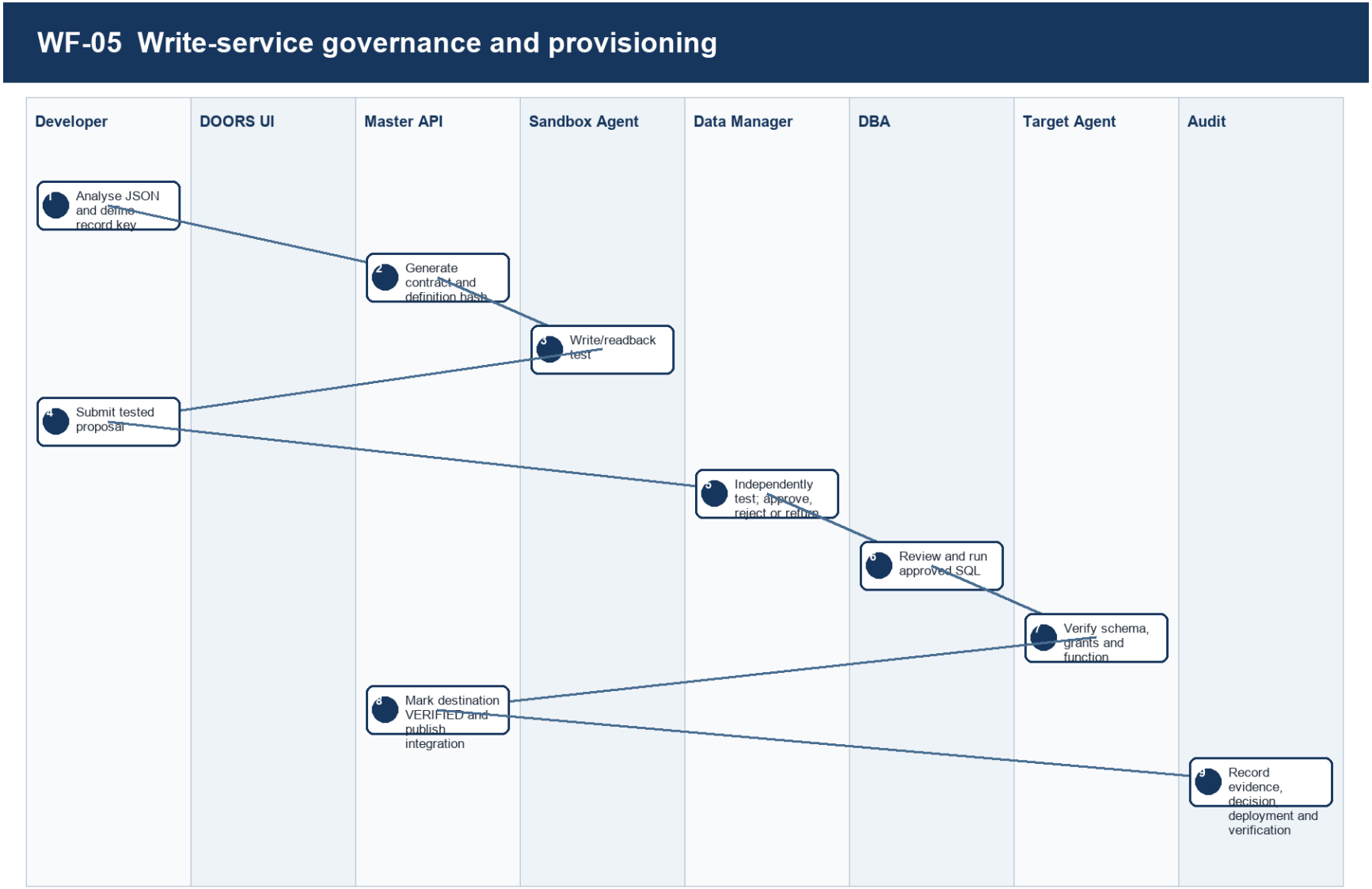


Figure H-5 WF-05 Write-service governance and provisioning

H.6 WF-06 Document-service and policy lifecycle

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

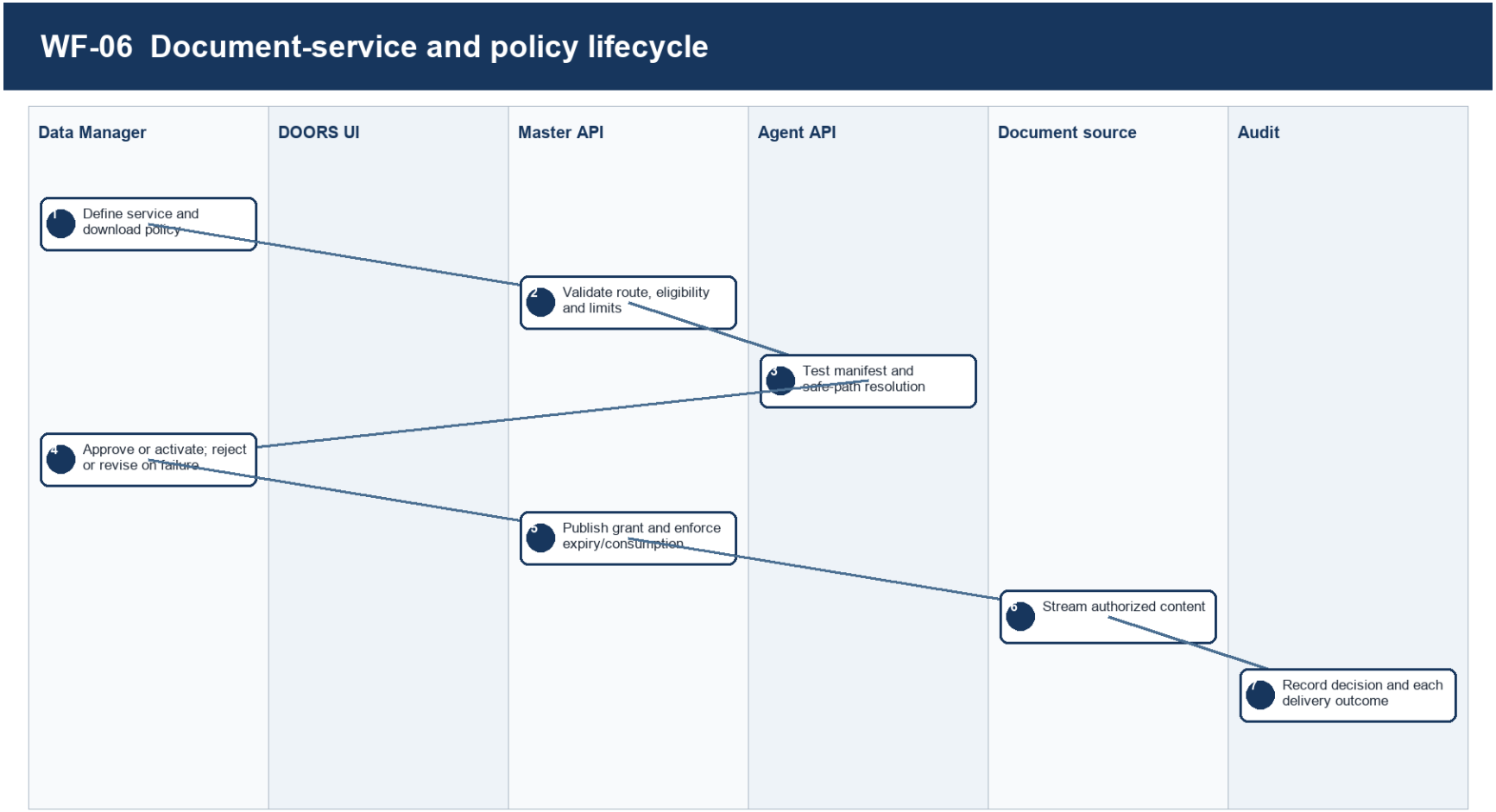


Figure H-6 WF-06 Document-service and policy lifecycle

H.7 WF-07 API-client activation and security change

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

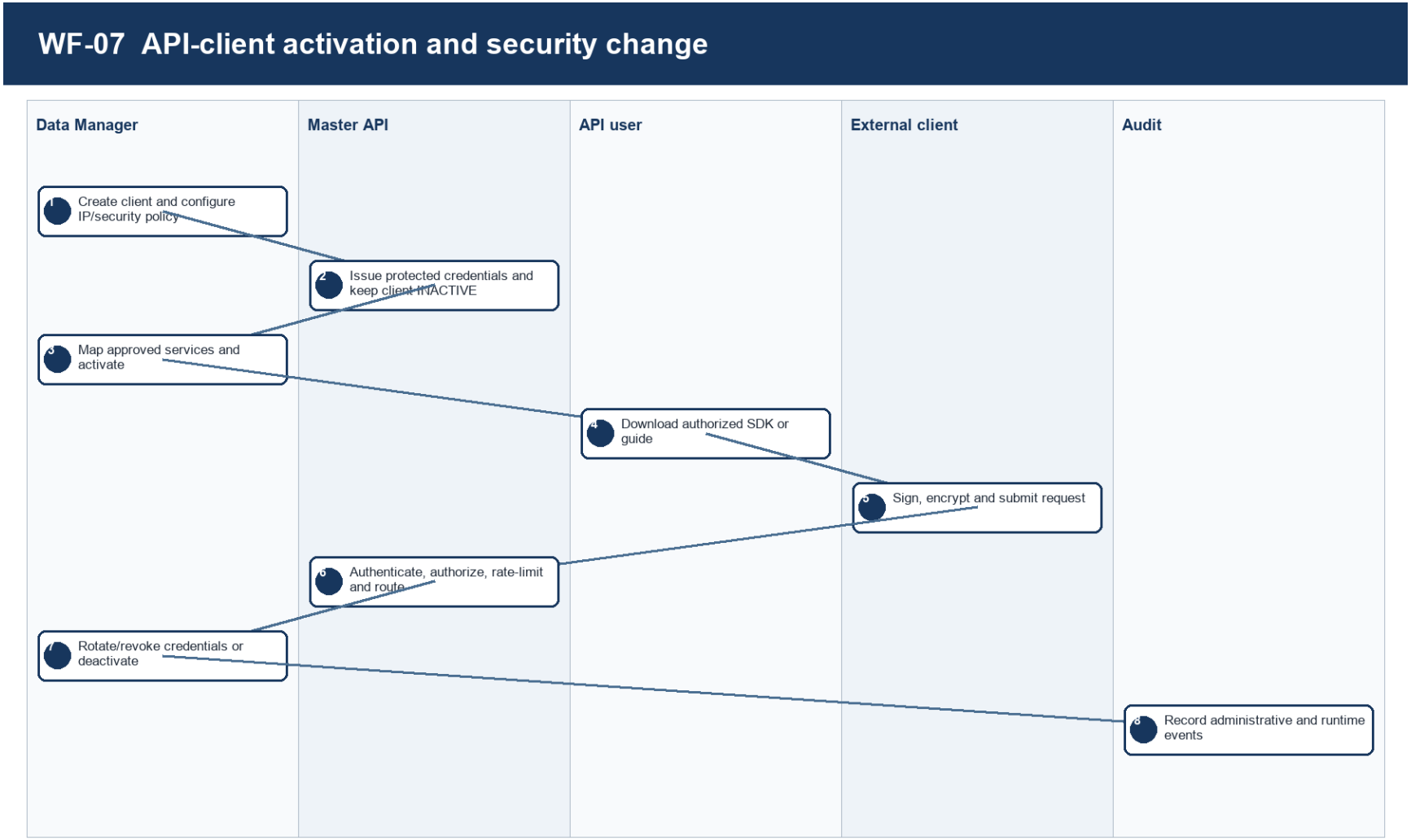


Figure H-7 WF-07 API-client activation and security change

H.8 WF-08 Subscription lifecycle

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

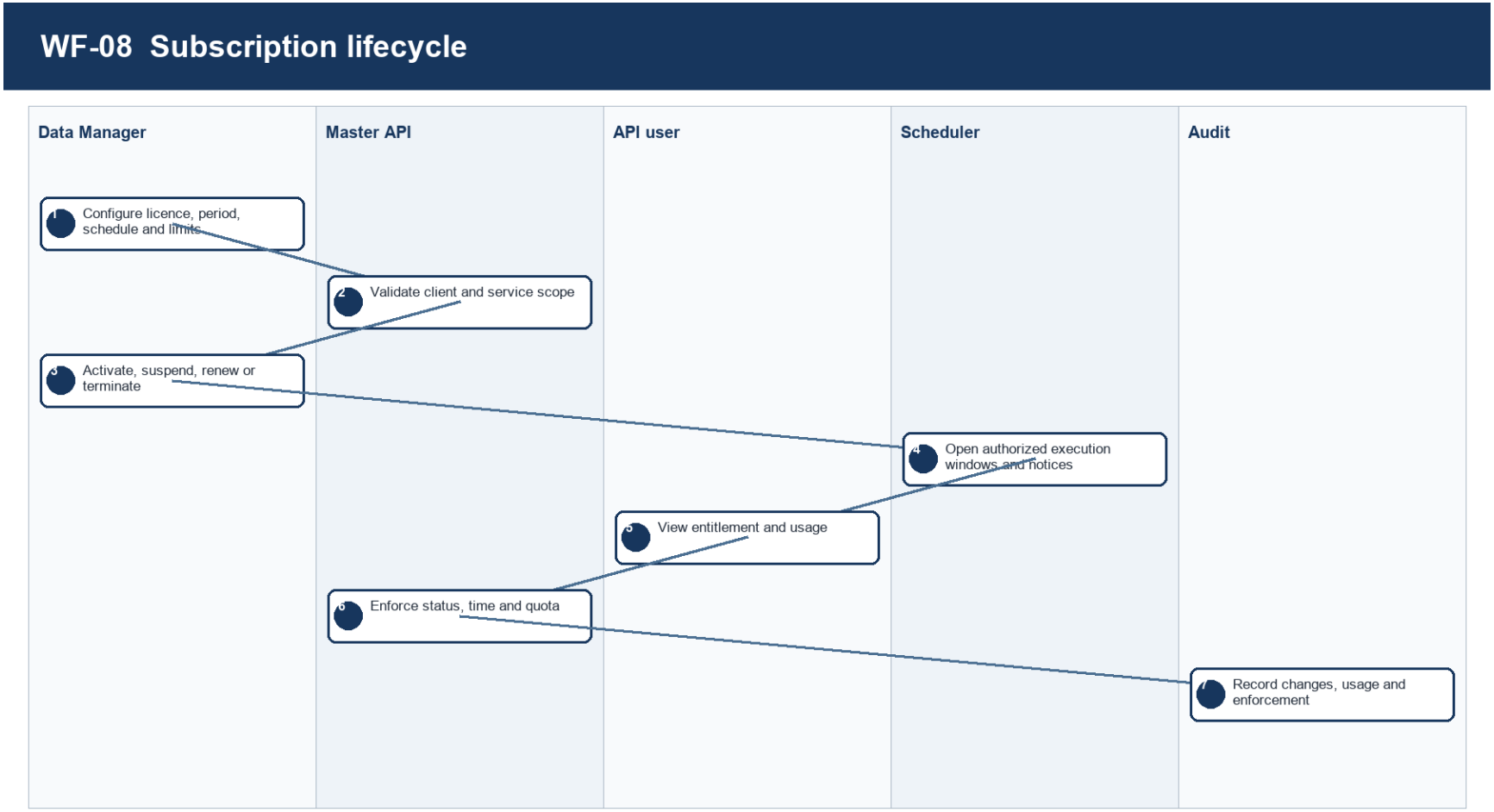


Figure H-8 WF-08 Subscription lifecycle

H.9 WF-09 Signing-key lifecycle

Decision handling: failed validation remains unsubmitted; a return or rejection retains reviewer remarks and permits authorized correction/resubmission where policy allows; approval requires an authorized independent actor when separation of duties applies. Every material transition records actor, time, object/version, decision and trace context.

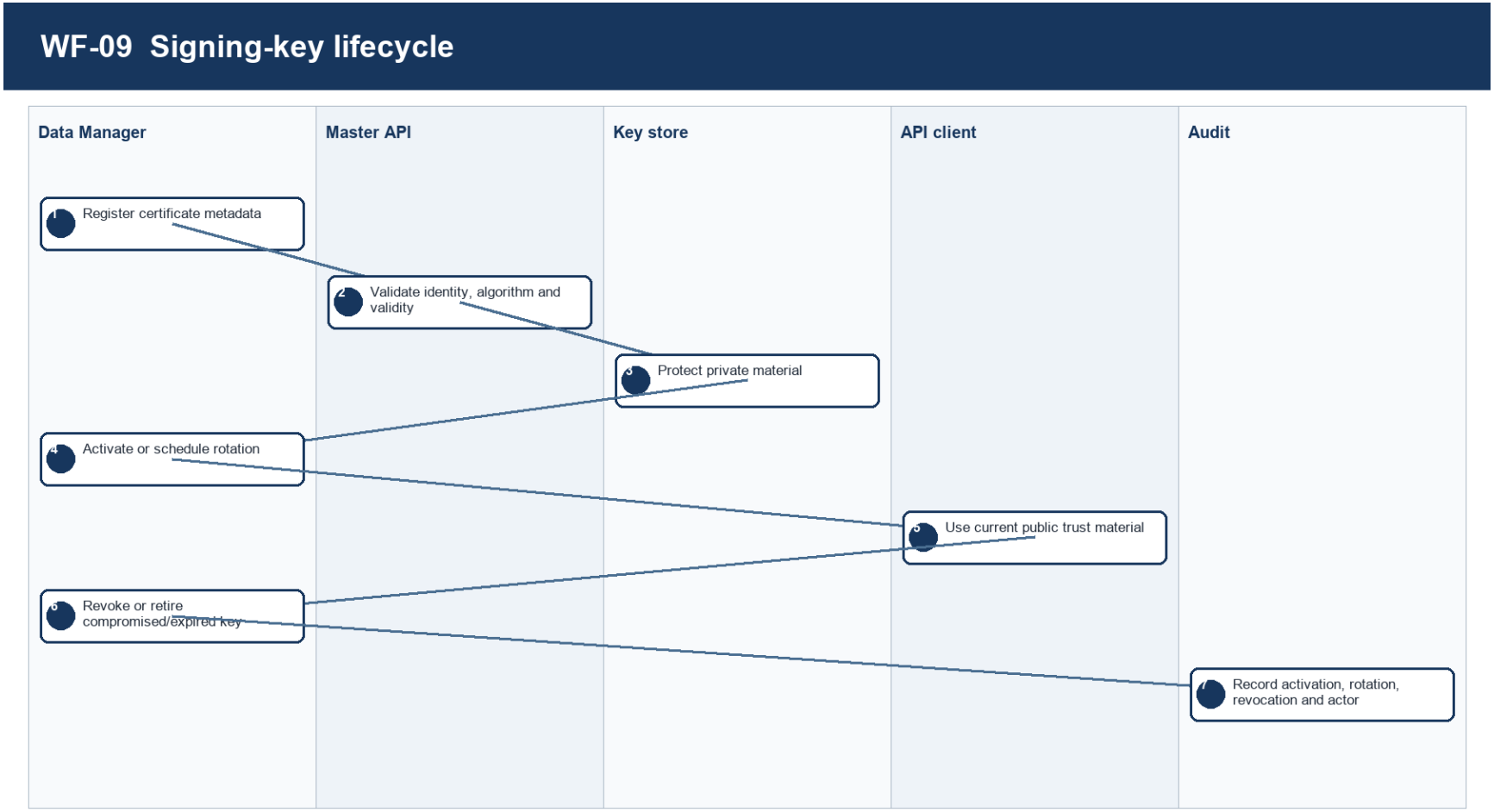


Figure H-9 WF-09 Signing-key lifecycle

Appendix I Architecture Decision Register

These decisions define the approved design intent. A material change requires a replacement decision with rationale, impact analysis and approval rather than silent implementation drift.

ID	Decision	Rationale	Consequence
ADR-01	Separate Master governance from Agent execution	Keeps policy central while data and database authority remain near each GePNIC source	Requires authenticated Master-to-Agent protocol, health monitoring and version compatibility
ADR-02	Separate Admin, External and API request planes	Reduces accidental route exposure and distinguishes human from machine admission	Requires exact proxy host rules and fail-closed plane routing
ADR-03	Use approved SELECT definitions with bound parameters	Provides reusable read services without caller-selected SQL	Requires parser validation, governance, row/time bounds and Agent dry run
ADR-04	Use DBA-provisioned SECURITY DEFINER functions for writes	Preserves DBA control and removes runtime DDL/direct table DML	Requires generated SQL review, controlled ownership, grants and post-install verification
ADR-05	Retain immutable write history, head and receipts	Provides auditability, deterministic retry and latest-state access	Consumes additional storage and requires retention/capacity planning
ADR-06	Use signed AES-256-GCM envelopes over HTTPS	Protects content, authenticates origin and detects tampering across the external boundary	Requires certificate lifecycle, key rotation, replay controls and client interoperability tests
ADR-07	Enforce authorization on the server and database	Browser controls improve usability but cannot be trusted as security boundaries	Requires repeated policy checks and least-privilege database identities
ADR-08	Use deterministic Alra answers before retrieval or generation	Reduces hallucination and preserves authorization boundaries for operational questions	Requires curated knowledge, provenance and safe fallback
ADR-09	Version migrations, contracts, SDKs and documentation	Makes releases reproducible and auditable	Requires one release manifest and controlled promotion evidence
ADR-10	Treat deployment values as controlled evidence, not prose defaults	Avoids publishing secrets or accepting unverified workstation values	Requires environment owners to approve limits, RPO/RTO, retention and monitoring thresholds

Appendix J Residual Risk and Operational Dependency Register

ID	Residual risk or dependency	Required treatment	Owner role	Release position
R-01	Configuration or database catalogue drifts from the approved source baseline	Compare manifest, Flyway history, object definitions, ownership, grants and effective configuration before release	Release manager and DBA	Block on unexplained variance
R-02	Certificate, key or transit secret expires, is revoked or is exposed	Monitor expiry; rehearse rotation/revocation; maintain custody and dependent-client communication	Security authority	Block when trust is invalid
R-03	Agent or network outage interrupts services mapped to one environment	Bound timeouts; isolate failure; alert; retry only safe operations; test recovery	Operations owner	Accept only within approved availability objective
R-04	Production capacity is inferred from development testing	Execute representative load and storage-growth tests and approve limits with headroom	Technical authority	Block without capacity evidence
R-05	Retention, purge, RPO or RTO values remain undefined	Obtain business, security, legal and operations approval; configure and rehearse	Project owner and operations	Block production approval
R-06	Generated write package differs from installed function or grants	Compare definition hash, function signature, owner, search path and grants through Agent verification	DBA and Data Manager	Block affected service
R-07	Sensitive data is exposed through diagnostic logging or export	Apply logging exclusions, privileged access, bounded exports, retention and review	Security and operations	Block on secret exposure
R-08	Alra knowledge becomes stale or unsupported content is generated	Use approved sources, deterministic answers, provenance, reindex controls and safe fallback	Knowledge owner	Disable affected generated answer path
R-09	Approval roles are concentrated or self-approval becomes possible	Enforce independent proposer/reviewer rules and periodically review role assignments	Project owner and security	Block governed publication
R-10	Documentation and implemented routes diverge	Regenerate route/screen catalogue and protocol assets as part of release verification	Development and QA	Block final document approval on material mismatch

Appendix K Final Design Acceptance Checklist

The following evidence must be attached to the release or approval record. Status remains Required until an accountable reviewer records Pass, an evidence reference and a date.

Gate	Required evidence	Approver	Status
Scope and baseline	Source commits, artefact hashes, migrations, generated SQL and documentation identify one baseline	Technical authority	Required
Architecture	Trust boundaries, request planes, Master/Agent responsibilities and ADRs match deployment	Technical authority	Required
UI and workflows	All routed screens and controlled-state flows are reviewed against Appendices G and H	Product owner and QA	Required
Security	Threat controls, DAST/SAST evidence, certificates, secrets, key rotation, encryption flows and negative tests pass	Security authority	Required
Database	Migrations, functions, owners, controlled search paths, roles and grants match the Database Design Document	Database authority	Required
Functional verification	Representative identity, read, write, document, report, subscription and AIra scenarios pass	QA lead	Required
Performance and capacity	Approved workload, percentiles, saturation point, limits and operational headroom are recorded	Technical authority and operations	Required
Availability and recovery	RPO, RTO, backup success, restore rehearsal and restart/failure behaviour are approved	Project owner and operations	Required
Observability	Health, expiry, security, database, capacity, backup and audit-pipeline alerts reach accountable owners	Operations and security	Required
Data governance	Classification, retention, legal hold, purge and destruction evidence are approved	Project owner and security	Required
Deployment and rollback	Runbook, configuration manifest, change window, rollback/forward-fix and smoke tests are approved	Release manager	Required
Formal approval	Approval Record contains names, decisions and dates for all mandatory authorities	Project owner	Required

Head of Department review conclusion: the SDD is technically complete for formal approval. Production release remains conditional on the evidence gates above; documentation completion does not substitute for environment verification or accountable sign-off.